

Software Analysis

Control Flow Analysis

Gordon Fraser
Lehrstuhl für Software Engineering II

Outline

1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

SpotBugs

+

← → ↺

https://spotbugs.github.io

☆ 🛑 🗨️ ⬆️ 🔍 🏠 📺 🛡️ 👤 ⋮

SpotBugs



Check it out on
GitHub

Find bugs in Java Programs

SpotBugs is a program which uses static analysis to look for bugs in Java code. It is free software, distributed under the terms of the [GNU Lesser General Public License](#).

SpotBugs is the spiritual successor of [FindBugs](#), carrying on from the point where it left off with support of its community. Please check [official manual site](#) for details.

SpotBugs requires JRE (or JDK) 1.8.0 or later to run. However, it can analyze programs compiled for any version of Java, from 1.0 to 1.9.

Bug Descriptions

SpotBugs checks for more than 400 bug patterns. Bug descriptions can be found [here](#)

Descriptions are also available in [Japanese](#)

Using SpotBugs

SpotBugs can be used standalone and through several integrations, including:

Bug Descriptions

Using SpotBugs

Extensions

License of SpotBugs Logo

Support or Contact

ⓧ Bug descriptions — spotbugs

← → ↺ 🔒 https://spotbugs.readthedocs.io/en/latest/bugDescript... ☆ 🛑 🗨️ ⬆️ ⚙️ 🏠 📄 🛡️ 👤 ⋮

Search docs

Introduction

Requirements

Installing

Running SpotBugs

Using the SpotBugs GUI

Using the SpotBugs Eclipse plugin

Using the SpotBugs Ant task

Using the SpotBugs Maven Plugin

Using the SpotBugs Gradle Plugin

Filter file

Analysis Properties

Effort

Implement SpotBugs plugin

Use SpotBugs Plugin on SonarQube

SpotBugs FAQ

SpotBugs Links

☐ Bug descriptions

⊕ Bad practice (BAD_PRACTICE)

⊕ Correctness (CORRECTNESS)

⊕ Experimental (EXPERIMENTAL)

⊕ Internationalization (I18N)

📖 Read the Docs

v: latest ▾

Examples include hash code and equals problems, cloneable idiom, dropped exceptions, Serializable problems, and misuse of finalize. We strive to make this analysis accurate, although some groups may not care about some of the bad practices.

CNT: Rough value of known constant found (CNT_ROUGH_CONSTANT_VALUE)

It's recommended to use the predefined library constant for code clarity and better precision.

NP: Method with Boolean return type returns explicit null (NP_BOOLEAN_RETURN_NULL) 🔗

A method that returns either Boolean.TRUE, Boolean.FALSE or null is an accident waiting to happen. This method can be invoked as though it returned a value of type boolean, and the compiler will insert automatic unboxing of the Boolean value. If a null value is returned, this will result in a NullPointerException.

SW: Certain swing methods needs to be invoked in Swing thread (SW_SWING_METHODS_INVOKED_IN_SWING_THREA

(From JDC Tech Tip): The Swing methods show(), setVisible(), and pack() will create the associated peer for the frame. With the creation of the peer, the system creates the event dispatch thread. This makes things problematic because the event dispatch thread could be notifying listeners while pack and validate are still processing. This situation could result in two threads going

Example

```
boolean booleanBug(int x) {  
    Boolean ret = null;  
  
    // ...  
  
    return ret;  
}
```

Example

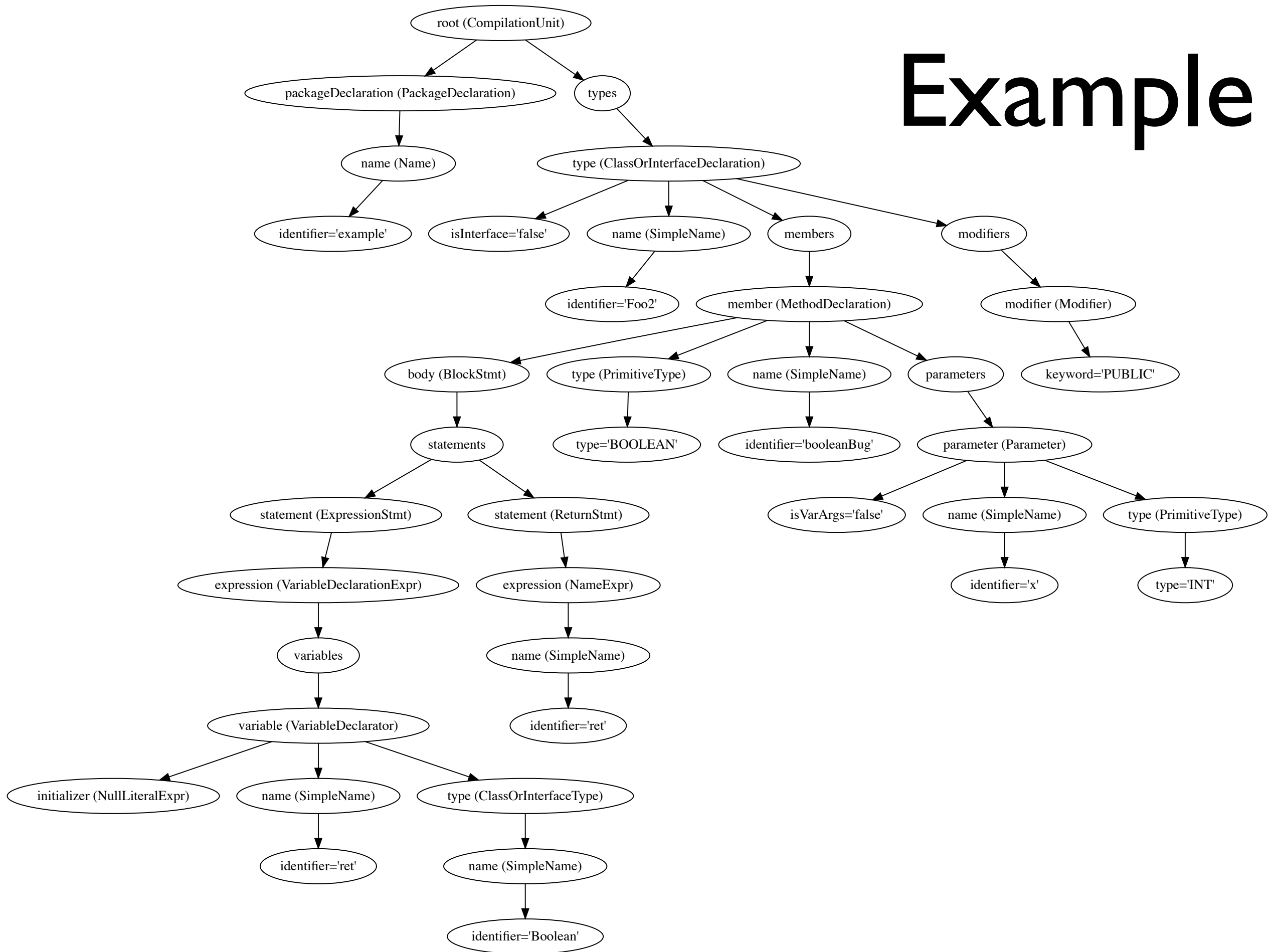
```
Primitive boolean booleanBug(int x) {  
    Object Boolean ret = null;  
  
    // ...  
  
    return ret; Boolean.TRUE -> true  
    Boolean.FALSE -> false  
    null -> ?  
}
```

Example

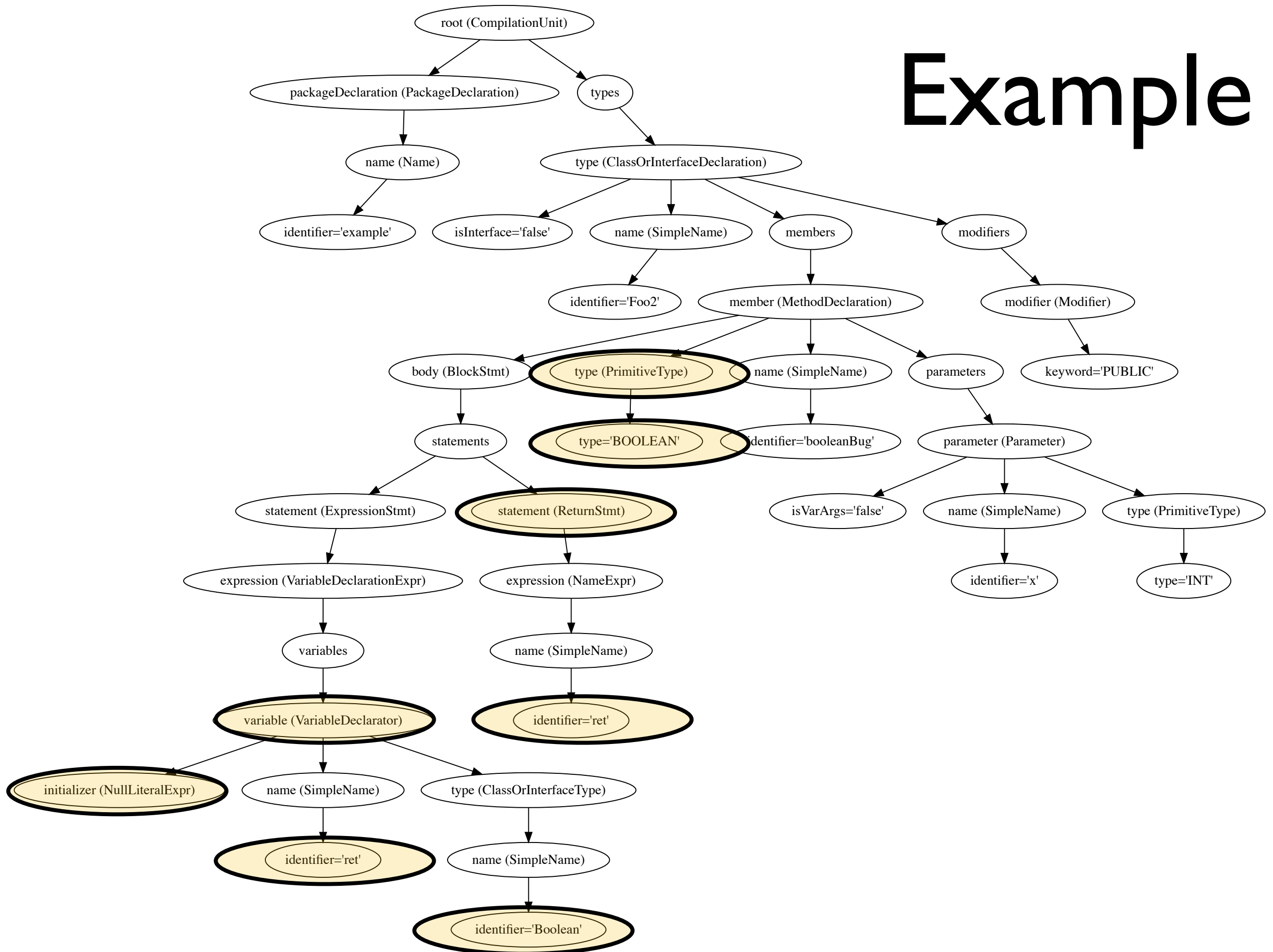
```
boolean booleanBug(int x) {  
    Boolean ret = null;  
  
    // ...  
  
    return ret;  
}
```

The diagram illustrates the flow of a variable named `ret`. A curved arrow originates from the parameter `x` in the function signature `booleanBug(int x)` and points to the variable `ret` in the assignment `Boolean ret = null;`. Another curved arrow originates from the variable `ret` in the `return ret;` statement and points back to the `ret` variable in the assignment. The `return` keyword is underlined with a red wavy line.

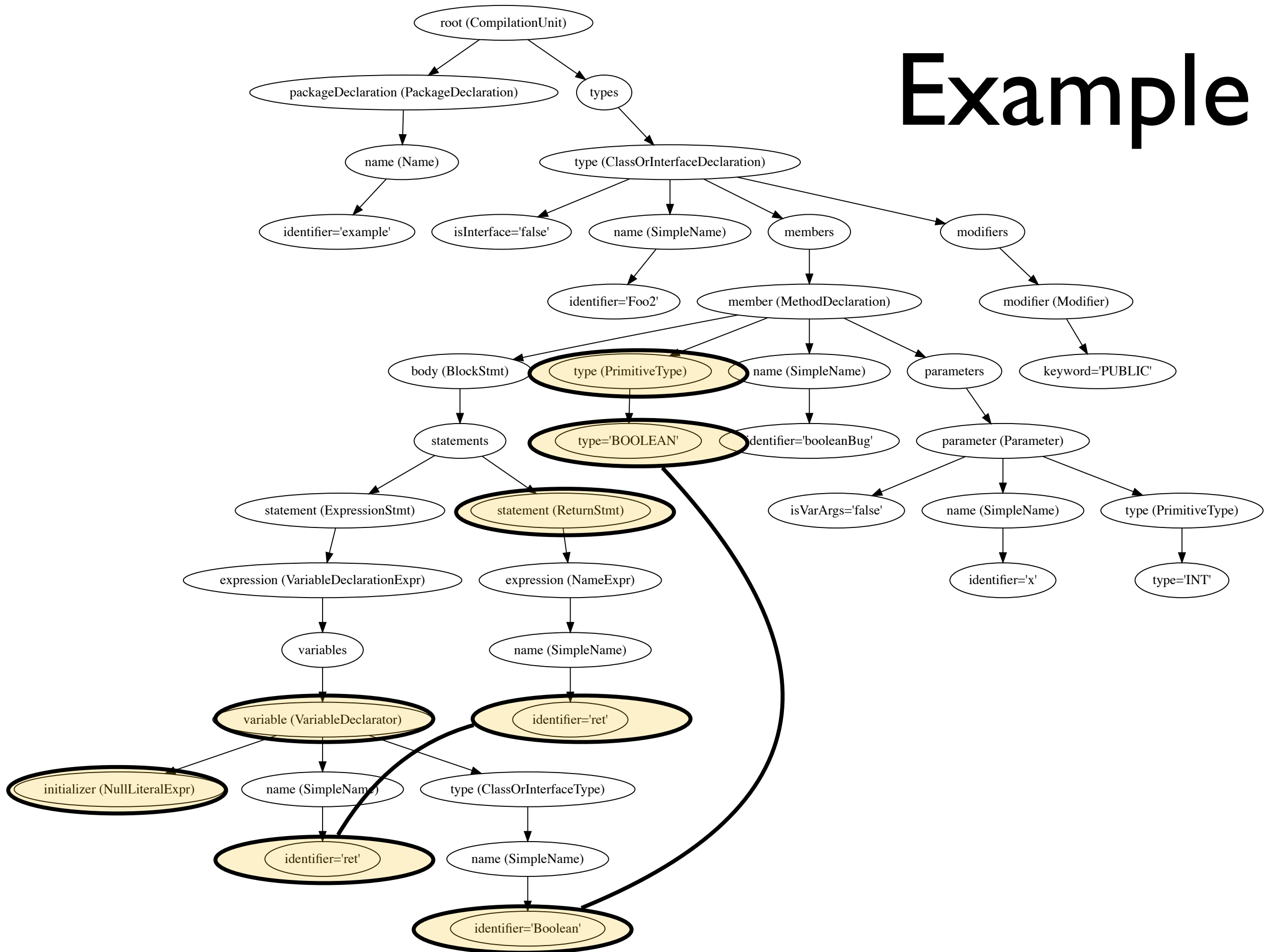
Example



Example



Example



Outline

1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

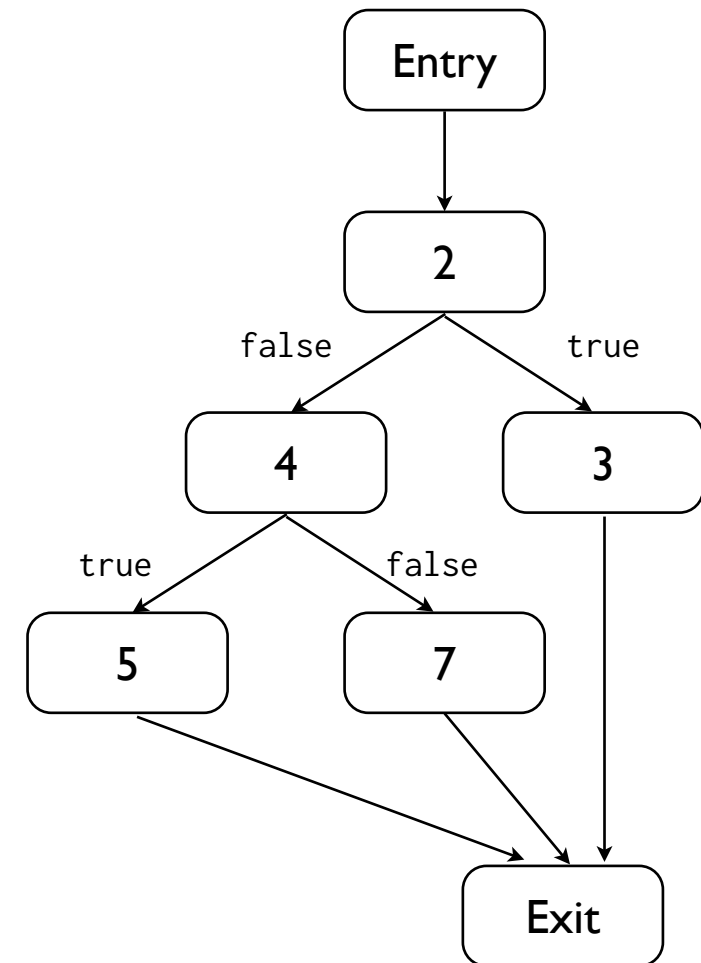
Control Flow Graph

- Source code statements are executed in sequence.
This sequence is known as the **control flow**.
- Control Flow Graphs (CFGs) model flow of control in programs (or procedures)
- $G = (N, E)$ as a directed graph
 - Node $n \in N$ are statements (or blocks of statements)
 - Edge $e = (n_i, n_j) \in E$ model the transfer of control from node n_i to node n_j
- Simplifying assumptions:
 - Unique entry node (n_0)
 - Unique exit node (n_f)

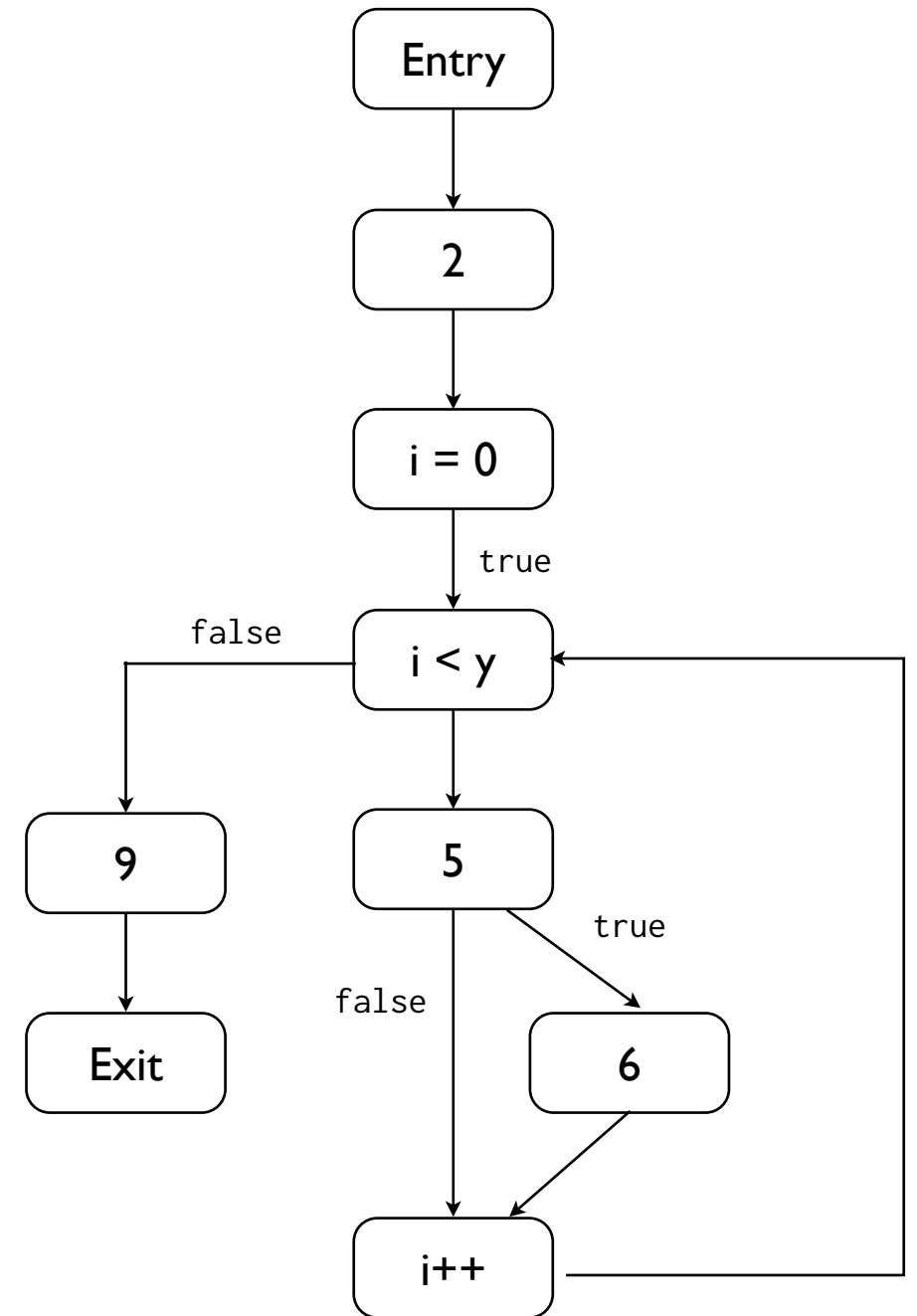
Definitions

- Given a CFG $G = (N, E)$
- If there is an edge $n_i \rightarrow n_j$ in E
 - n_i is a *predecessor* of n_j
 - n_j is a *successor* of n_i
- For any node n in N :
 - $\text{Pred}(n)$: The set of predecessor nodes of n
 - $\text{Succ}(n)$: The set of successor nodes of n
- A *branch node* is just a node that has more than one successor
- A *join node* is a node that has more than one predecessor

```
1 public int foo(int x) {  
2     if(x == 5) {  
3         return 1;  
4     } else if(x == 6) {  
5         return 2;  
6     } else {  
7         return 3;  
8     }  
9 }
```



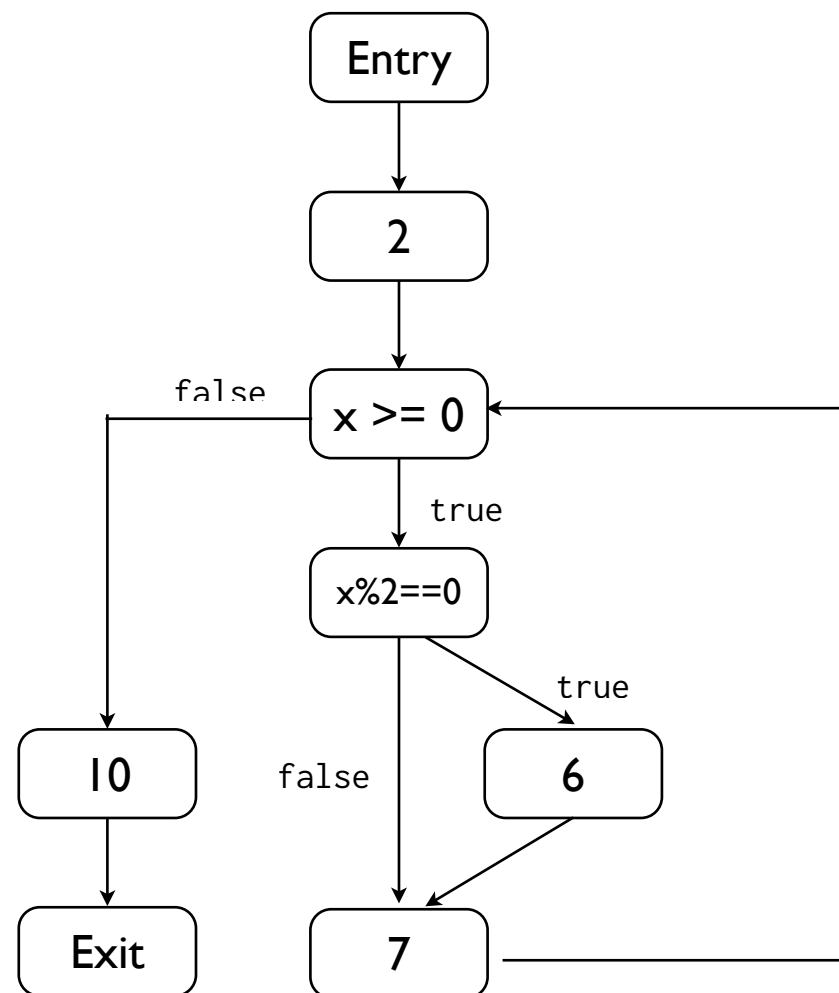
```
1 public int foo(int y) {  
2     int x = 0;  
3  
4     for(int i = 0; i < y; i++) {  
5         if(i % 2 == 0)  
6             x += i;  
7     }  
8  
9     return x;  
10 }
```




```

1  public int foo(int x) {
2      int y = 0;
3
4      while(x >= 0) {
5          if(x % 2 == 0)
6              y += x;
7          x--;
8      }
9
10     return y;
11 }

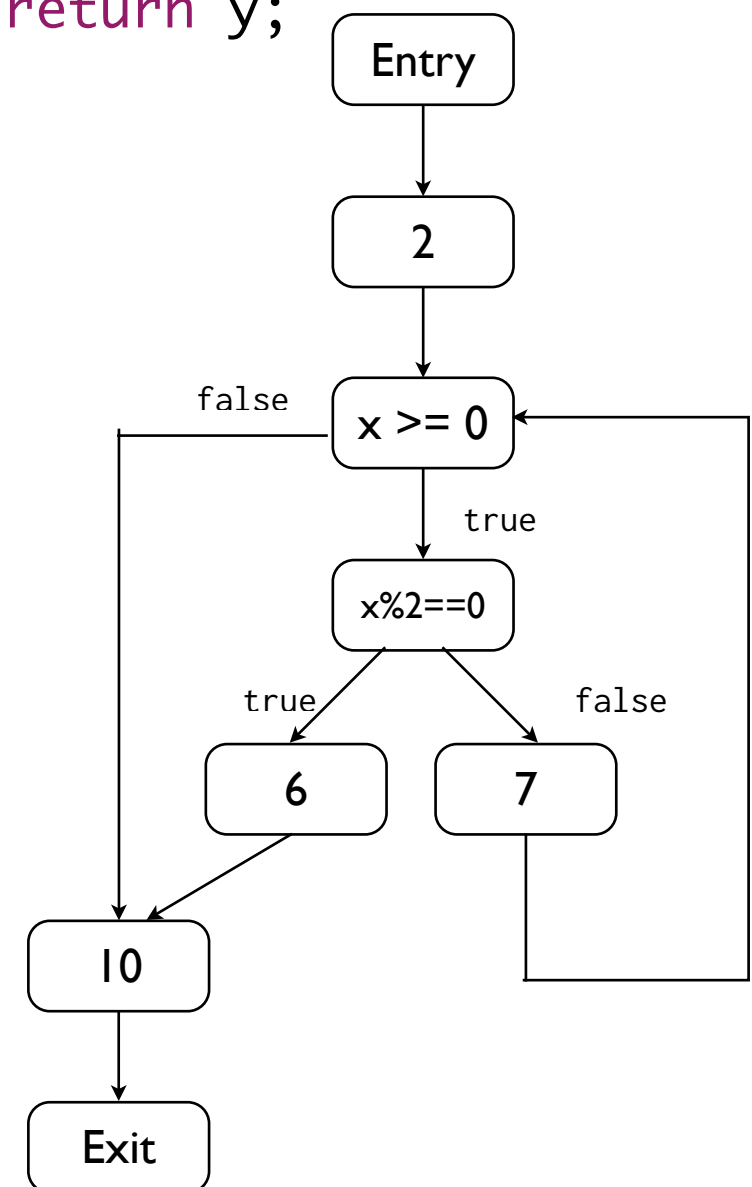
```



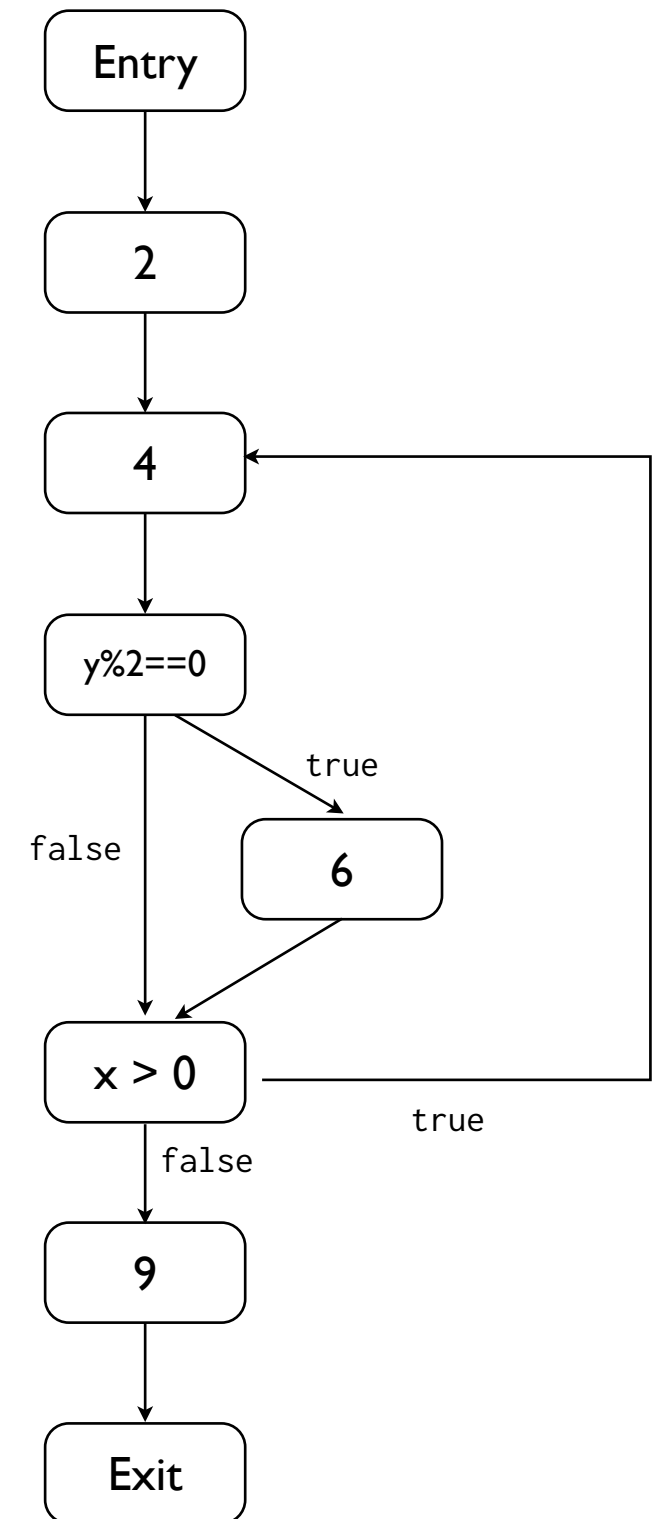
```

1  public int foo(int x) {
2      int y = 0;
3
4      while(x >= 0) {
5          if(x % 2 == 0)
6              break;
7          x--;
8      }
9
10     return y;
11 }

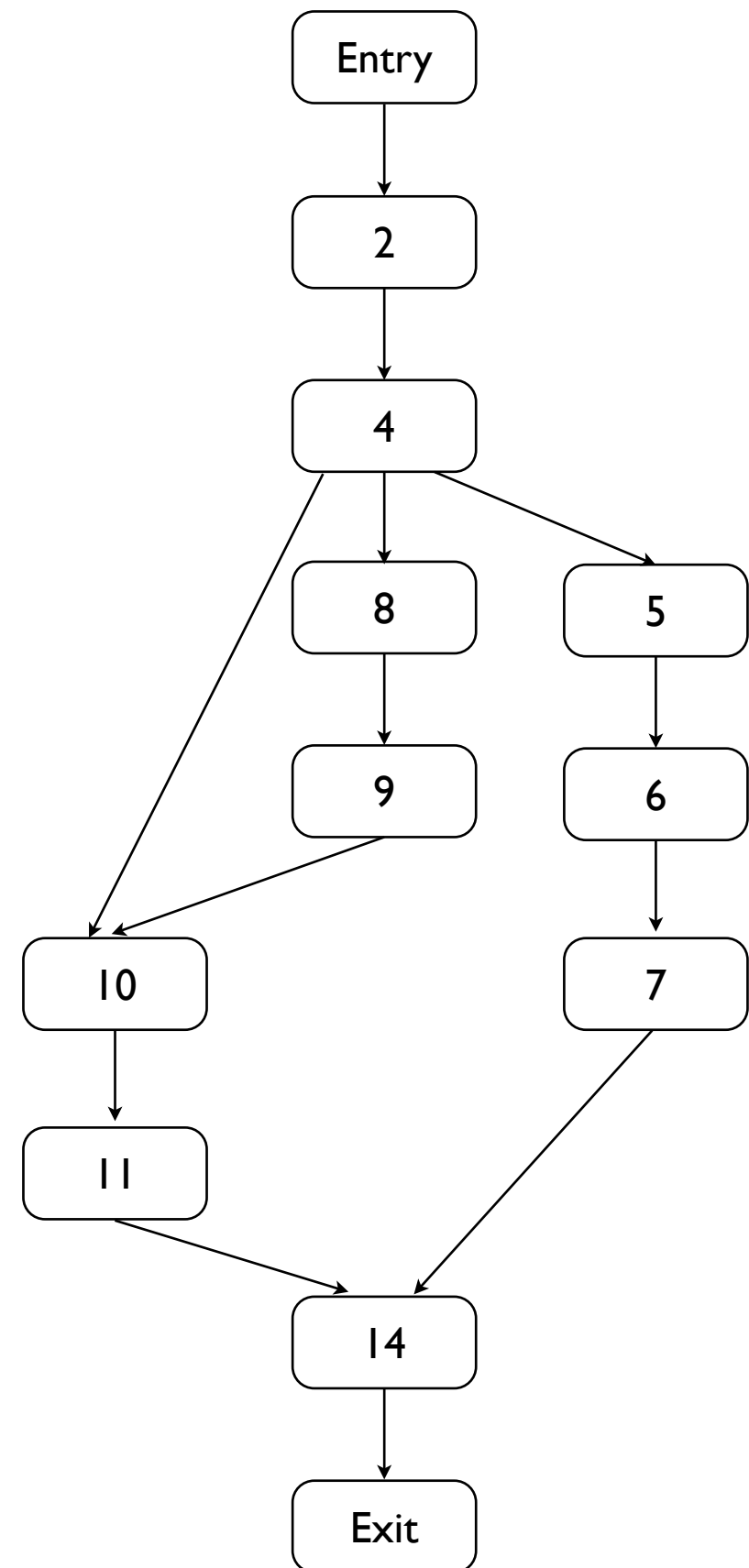
```



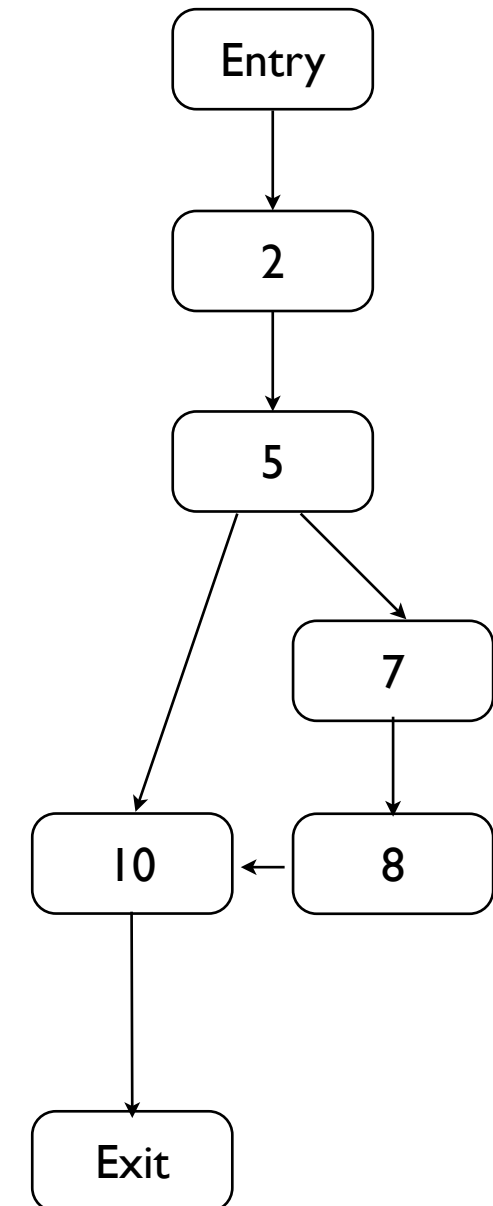

```
1 public int foo(int y) {  
2     int x = 0;  
3  
4     do {  
5         if(y % 2 == 0)  
6             x += y;  
7     } while(x > 0);  
8  
9     return x;  
10 }
```



```
1  public int foo(int x) {  
2      int y = 0;  
3  
4      switch(x) {  
5          case 0:  
6              y = 1;  
7              break;  
8          case 1:  
9              y = 2;  
10         default:  
11             y = 3;  
12     }  
13  
14     return y;  
15 }
```



```
1 public int foo(int y) {  
2     int x = 0;  
3  
4     try {  
5         foo();  
6     }  
7     catch(Throwable t) {  
8         System.exit(1);  
9     }  
10    return x;  
11 }
```



CFG vs. AST

- CFGs are much **simpler** than ASTs
 - Fewer forms, less redundancy, only simple expressions
- But...AST is a **more faithful** representation
 - CFGs loses block structure of program
- So for AST,
 - Easier to report error + other messages
 - Easier to explain to programmer
 - Easier to unparse to produce readable code

Constructing a CFG

- Associate with each Java statement a node in the CFG, along with an entry and exit node.
- Walk the nodes of the AST.
- If the node is a *non-jumping* statement, make an edge between it and the next statement.
- If the node is a “*jumping*” instruction (if statements, loops)
 - Add outgoing edges depending on conditions
 - Add back edges for loops

Possible Applications of CFG

- Finding dead code (e.g., using **reachability**)
- Understanding algorithms
- Measuring coverage (e.g., branch coverage)
- Measuring complexity (e.g., cyclomatic complexity)
- Foundation for data flow analysis

3-D Plot

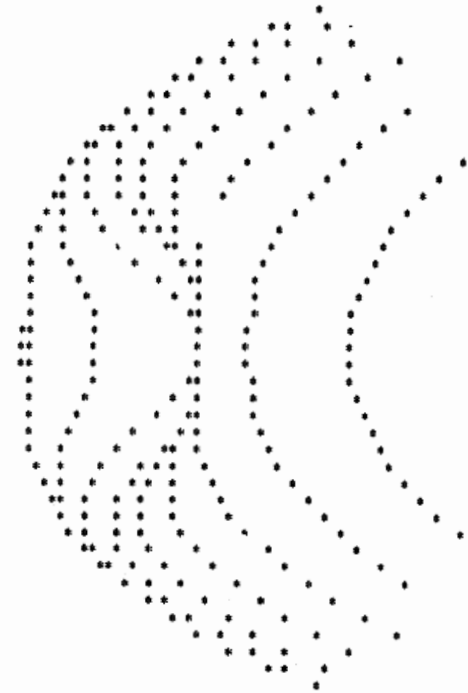
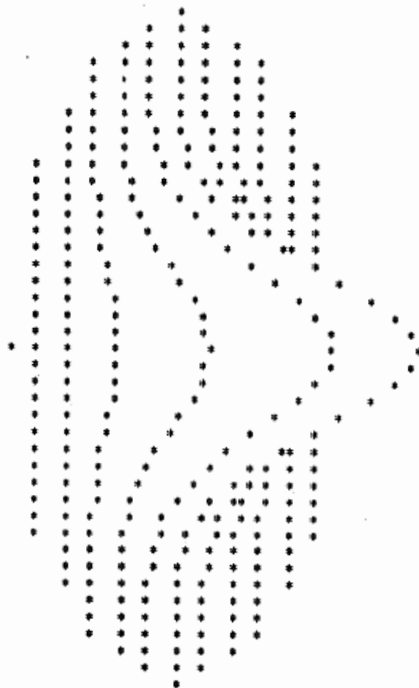
3-D PLOT will plot the family of curves of any function. The function Z is plotted as "rising" out of the x-y plane with x and y inside a circle of radius 30. The resultant plot looks almost 3-dimensional.

You set the function you want plotted in line 5. As with any mathematical plot, some functions come out "prettier" than others. Here are some that work nicely:

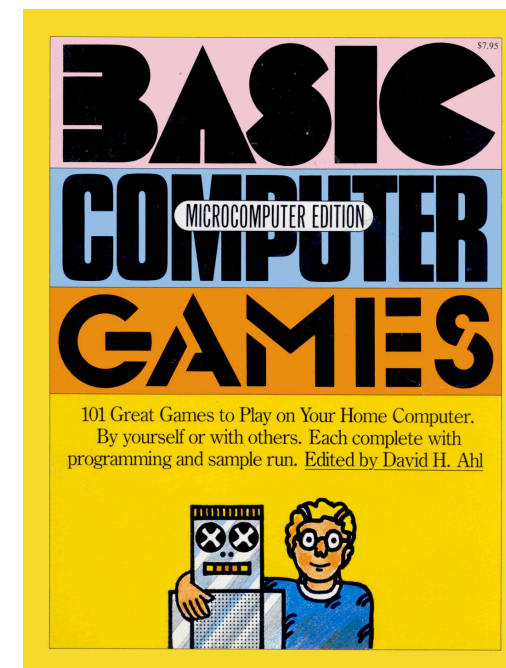
```
5 DEF FNA (Z) = 30*EXP (-Z*Z/100)
5 DEF FNA (Z) = SQR (900.01-Z*Z) *.9-2
5 DEF FNA (Z) = 30*(COS (Z/16)  2
5 DEF FNA (Z) = 30-30*SIN (Z/18)
5 DEF FNA (Z) = 30*EXP (-COS (Z/16) ) -30
  (Bessel function—Summerfeld's Integral)
5 DEF FNA (Z) = 30*SIN (Z/10)
```

The author of this amazingly clever program is Mark Bramhall of DEC.

3D PLOT
CREATIVE COMPUTING MORRISTOWN, NEW JERSEY



```
1 PRINT TAB(32); "3D PLOT"
2 PRINT TAB(15); "CREATIVE COMPUTING MORRISTOWN, NEW JERSEY"
3 PRINT:PRINT:PRINT
5 DEF FNA(Z)=30*EXP(-Z*Z/100)
100 PRINT
110 FOR X=-30 TO 30 STEP 1.5
120 L=0
130 Y1=5*INT(SQR(900-X*X)/5)
140 FOR Y=Y1 TO -Y1 STEP -.5
150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)
160 IF Z<=L THEN 190
170 L=Z
180 PRINT TAB(Z); "*";
190 NEXT Y
200 PRINT
210 NEXT X
300 END
```

3-D Plot

```

1 PRINT TAB(32);"3D PLOT"
2 PRINT TAB(15);"CREATIVE COMPUTING"
3 PRINT:PRINT:PRINT
5 DEF FNA(Z)=30*EXP(-Z*Z/100)
100 PRINT
110 FOR X=-30 TO 30 STEP 1.5
120 L=0
130 Y1=5*INT(SQR(900-X*X)/5)
140 FOR Y=Y1 TO -Y1 STEP -5
150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)
160 IF Z<=L THEN 190
170 L=Z
180 PRINT TAB(Z);"*";
190 NEXT Y
200 PRINT
210 NEXT X
300 END

```

3-D PLOT will plot the family of curves of any function. The function Z is plotted as "rising" out of the x-y plane with x and y inside a circle of radius 30. The resultant plot looks almost 3-dimensional.

You set the function you want plotted in line 5. As with any mathematical plot, some functions come out "prettier" than others. Here are some that work nicely:

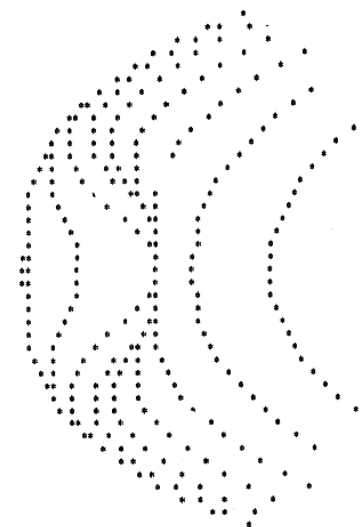
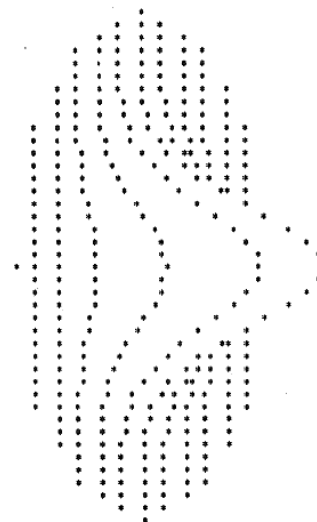
```

5 DEF FNA(Z) = 30*EXP (-Z*Z/100)
5 DEF FNA (Z)= SQR (900.01-Z*Z) ^ .9-2
5 DEF FNA (Z) = 30*(COS(Z/16) ^ 2
5 DEF FNA (Z) = 30-30*SIN (Z/18)
5 DEF FNA (Z) = 30*EXP (-COS (Z/16) ) -30
(Bessel function—Summerfeld's Integral)
5 DEF FNA (Z) = 30*SIN (Z/10)

```

The author of this amazingly clever program is Mark Bramhall of DEC.

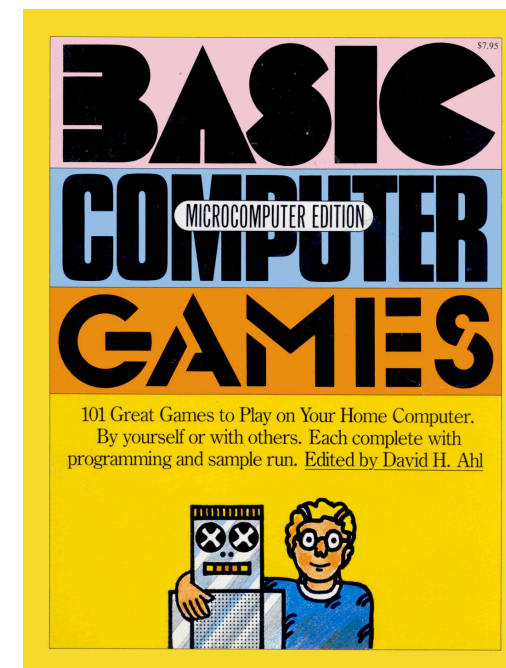
3D PLOT
CREATIVE COMPUTING HORRISTOWN, NEW JERSEY



```

1 PRINT TAB(32);"3D PLOT"
2 PRINT TAB(15);"CREATIVE COMPUTING HORRISTOWN, NEW JERSEY"
3 PRINT:PRINT:PRINT
5 DEF FNA(Z)=30*EXP(-Z*Z/100)
100 PRINT
110 FOR X=-30 TO 30 STEP 1.5
120 L=0
130 Y1=5*INT(SQR(900-X*X)/5)
140 FOR Y=Y1 TO -Y1 STEP -5
150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)
160 IF Z<=L THEN 190
170 L=Z
180 PRINT TAB(Z);"*";
190 NEXT Y
200 PRINT
210 NEXT X
300 END

```

3-D Plot

```

1 PRINT TAB(32);"3D PLOT"
2 PRINT TAB(15);"CREATIVE COMPUTING"
3 PRINT:PRINT:PRINT
5 DEF FNA(Z)=30*EXP(-Z*Z/100)
100 PRINT
110 FOR X=-30 TO 30 STEP 1.5
    120 L=0
    130 Y1=5*INT(SQR(900-X*X)/5)
    140 FOR Y=Y1 TO -Y1 STEP -5
        150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)
        160 IF Z<=L THEN 190
        170 L=Z
        180 PRINT TAB(Z);"*";
    190 NEXT Y
    200 PRINT
    210 NEXT X
300 END

```

3-D PLOT will plot the family of curves of any function. The function Z is plotted as "rising" out of the x-y plane with x and y inside a circle of radius 30. The resultant plot looks almost 3-dimensional.

You set the function you want plotted in line 5. As with any mathematical plot, some functions come out "prettier" than others. Here are some that work nicely:

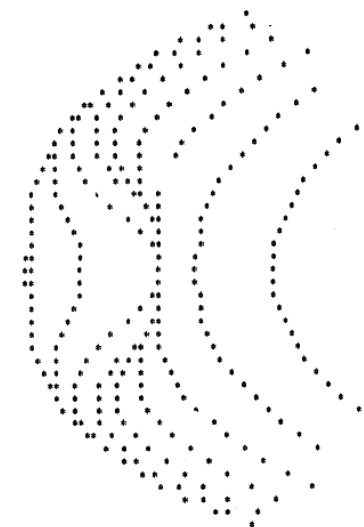
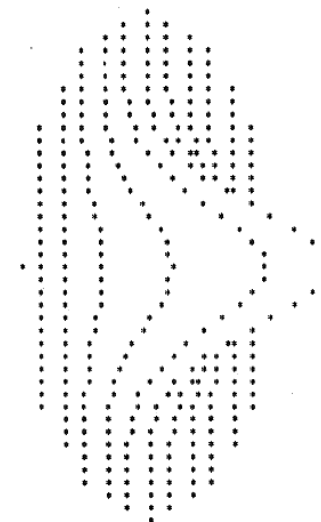
```

5 DEF FNA (Z) = 30*EXP (-Z*Z/100)
5 DEF FNA (Z) = SQR (900.01-Z*Z) ^ .9-2
5 DEF FNA (Z) = 30*(COS(Z/16)) ^ 2
5 DEF FNA (Z) = 30-30*SIN (Z/18)
5 DEF FNA (Z) = 30*EXP (-COS (Z/16)) -30
  (Bessel function—Summerfeld's Integral)
5 DEF FNA (Z) = 30*SIN (Z/10)

```

The author of this amazingly clever program is Mark Bramhall of DEC.

3D PLOT
CREATIVE COMPUTING HORRISTOWN, NEW JERSEY

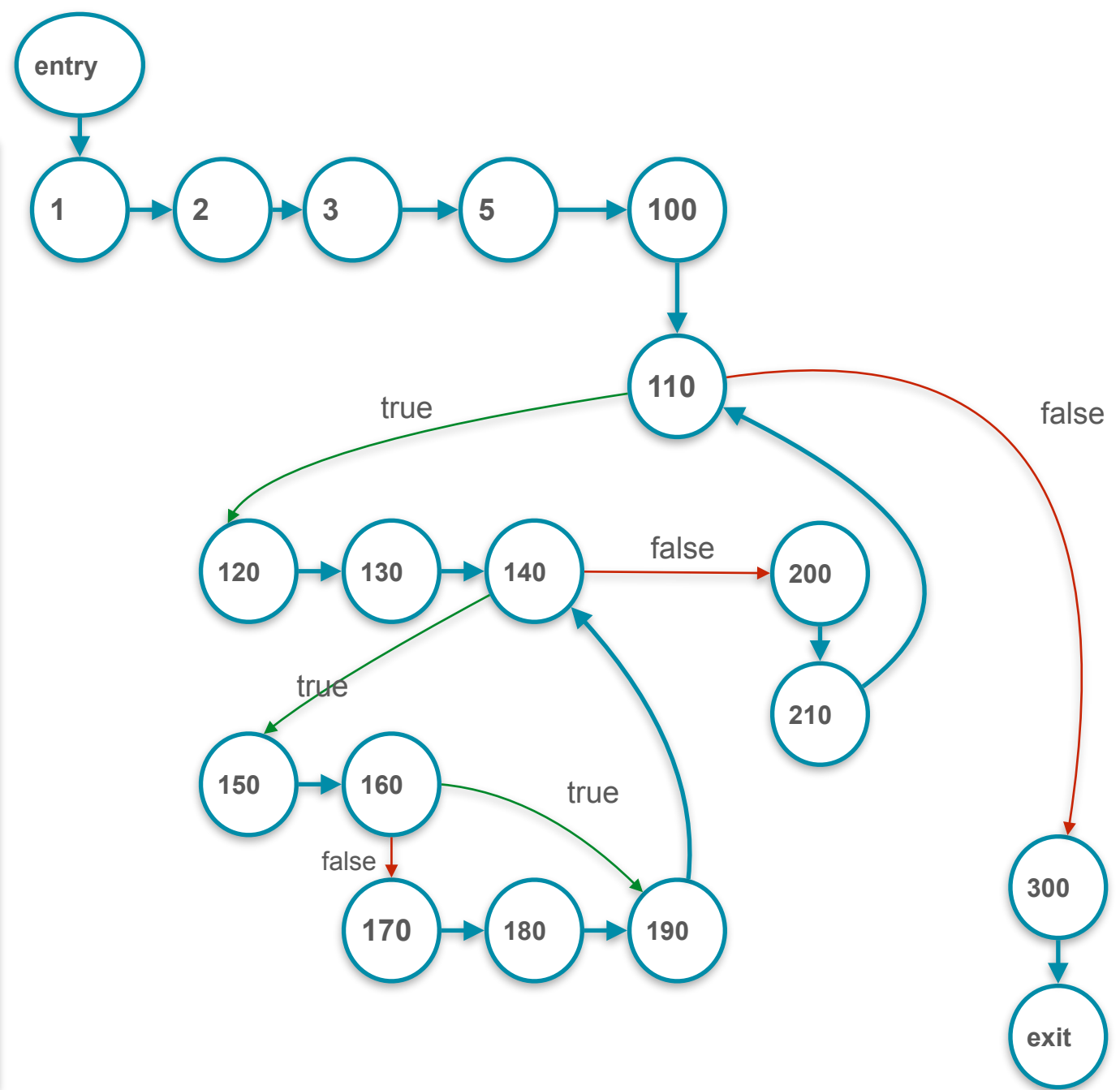


```

1 PRINT TAB(32);"3D PLOT"
2 PRINT TAB(15);"CREATIVE COMPUTING HORRISTOWN, NEW JERSEY"
3 PRINT:PRINT:PRINT
5 DEF FNA(Z)=30*EXP(-Z*Z/100)
100 PRINT
110 FOR X=-30 TO 30 STEP 1.5
    120 L=0
    130 Y1=5*INT(SQR(900-X*X)/5)
    140 FOR Y=Y1 TO -Y1 STEP -5
        150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)
        160 IF Z<=L THEN 190
        170 L=Z
        180 PRINT TAB(Z);"*";
    190 NEXT Y
    200 PRINT
    210 NEXT X
300 END

```

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
  190 NEXT Y  
200 PRINT  
210 NEXT X  
300 END
```



Outline

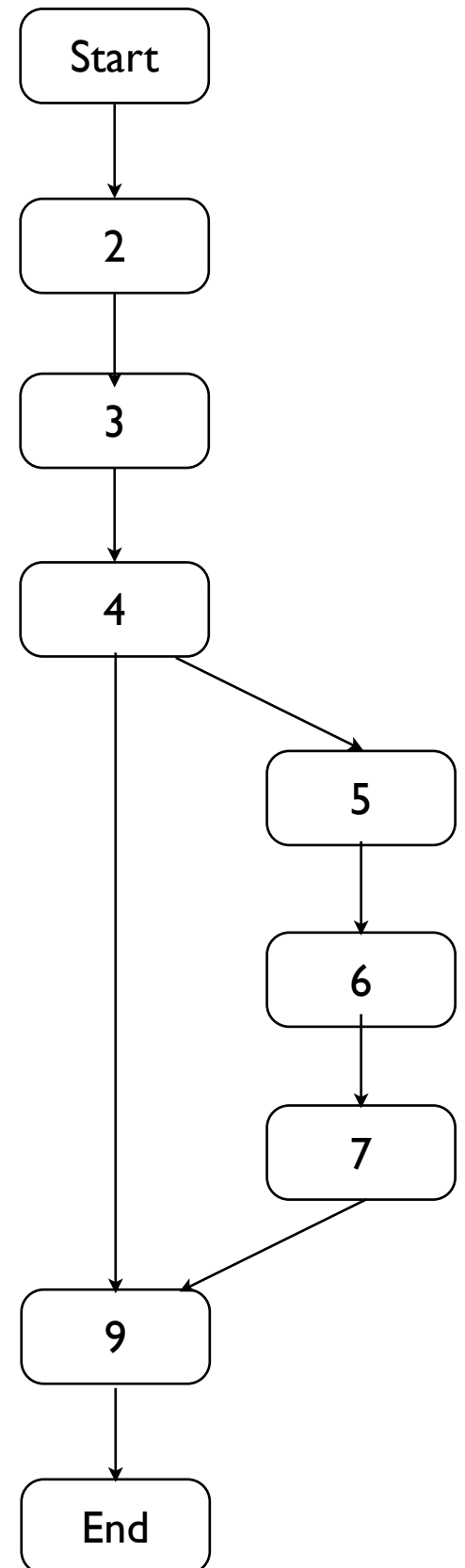
1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

Basic Blocks

- Many analyses only care about the **branching** behaviour of a function
- A **basic block** is a sequence of statements that must execute consecutively.
- Flow enters first statement in the block and leaves the last statement.
- No intermediate statements can exit the program or jump to another statement.
- Often basic unit in control flow analysis

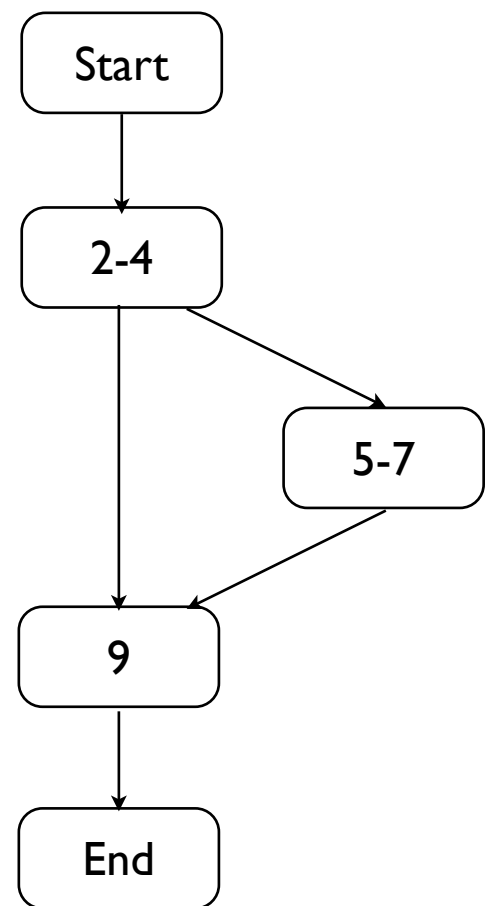
Basic Blocks

```
1 public int foo(int y) {  
2     int x = 0;  
3     int z = y*2;  
4     if(y == x + z) {  
5         System.out.println(y);  
6         z++;  
7         bar();  
8     }  
9     return z;  
10 }
```



Basic Blocks

```
1 public int foo(int y) {  
2     int x = 0;  
3     int z = y*2;  
4     if(y == x + z) {  
5         System.out.println(y);  
6         z++;  
7         bar();  
8     }  
9     return z;  
10 }
```



Identifying Basic Blocks

1. Identify leaders

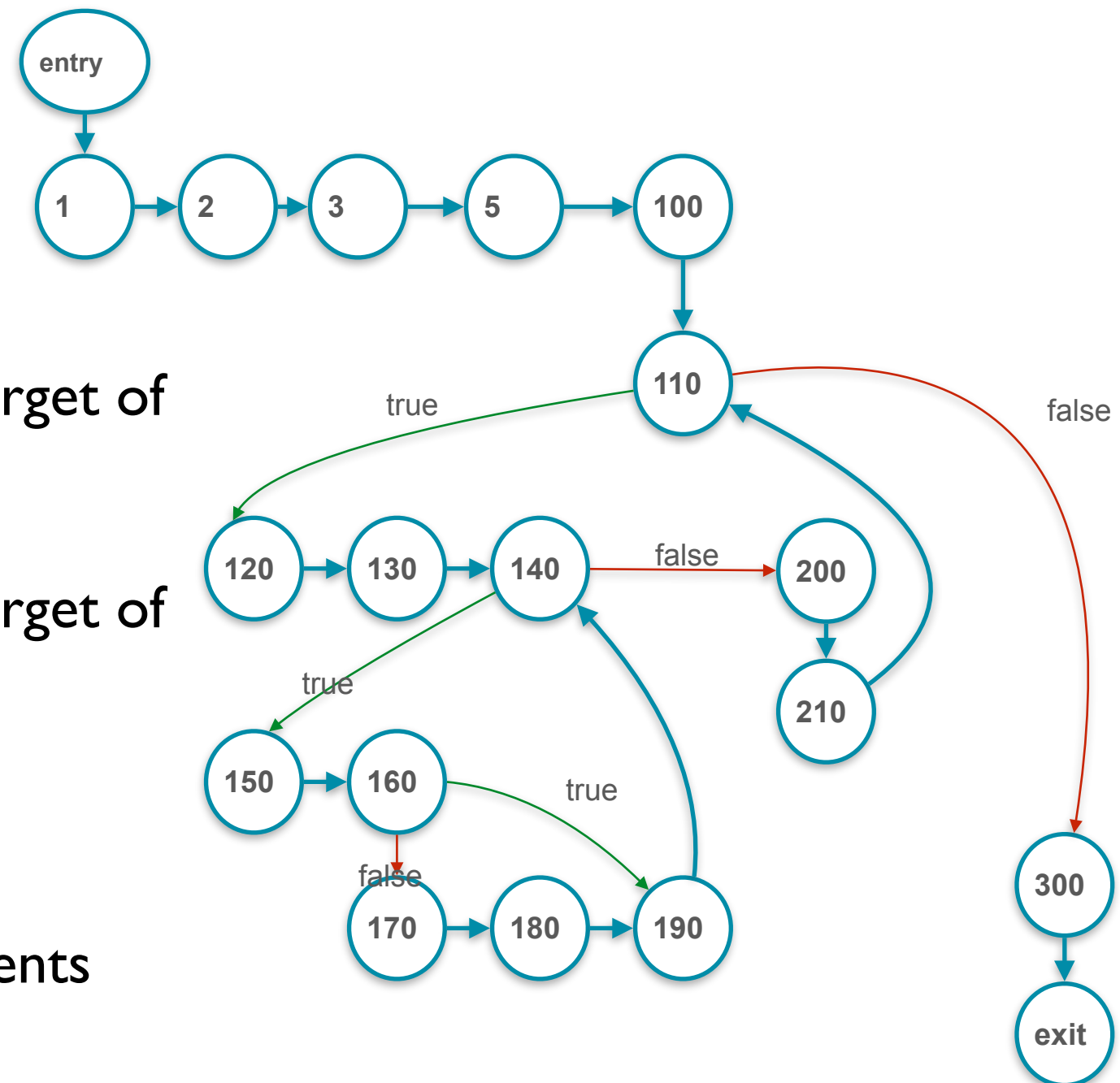
1. First statement

2. Every statement that is the target of a branch

3. Every statement that is the target of an implicit goto

2. Identify the basic blocks

1. Leader + sequence of statements until next leaders



Identifying Basic Blocks

1. Identify leaders

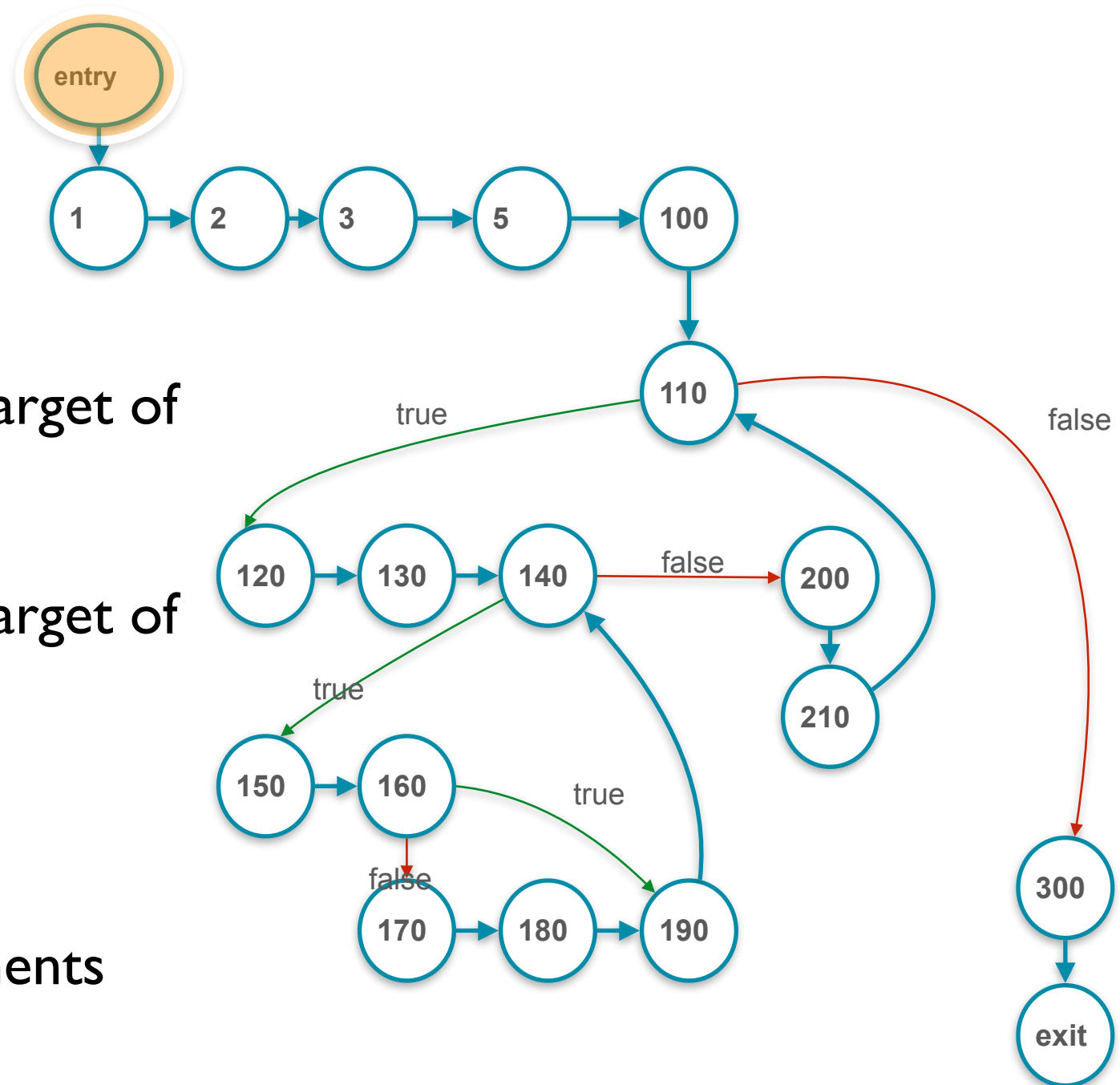
1. First statement

2. Every statement that is the target of a branch

3. Every statement that is the target of an implicit goto

2. Identify the basic blocks

1. Leader + sequence of statements until next leaders



Identifying Basic Blocks

1. Identify leaders

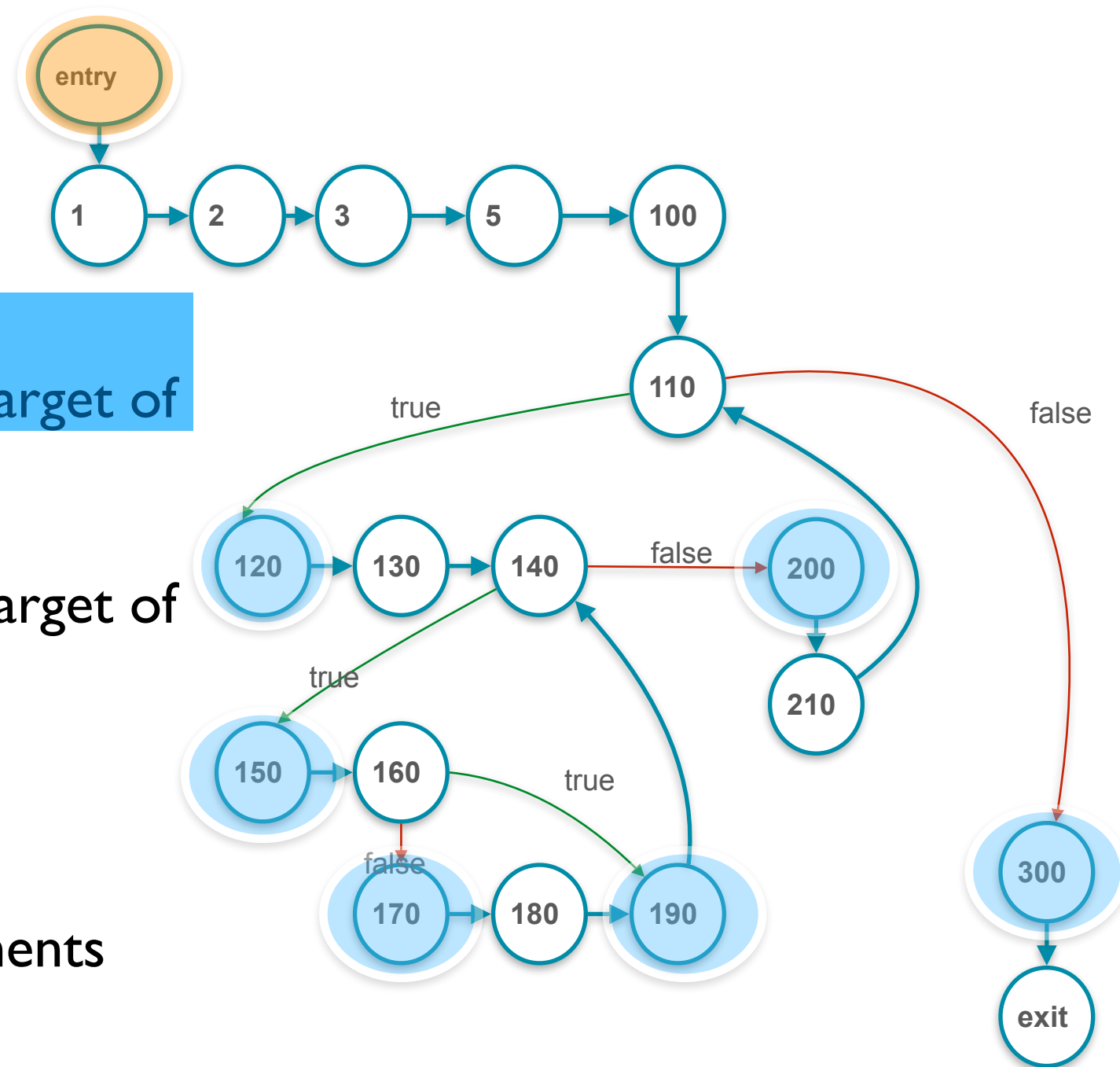
1. First statement

2. Every statement that is the target of a branch

3. Every statement that is the target of an implicit goto

2. Identify the basic blocks

1. Leader + sequence of statements until next leaders



Identifying Basic Blocks

1. Identify leaders

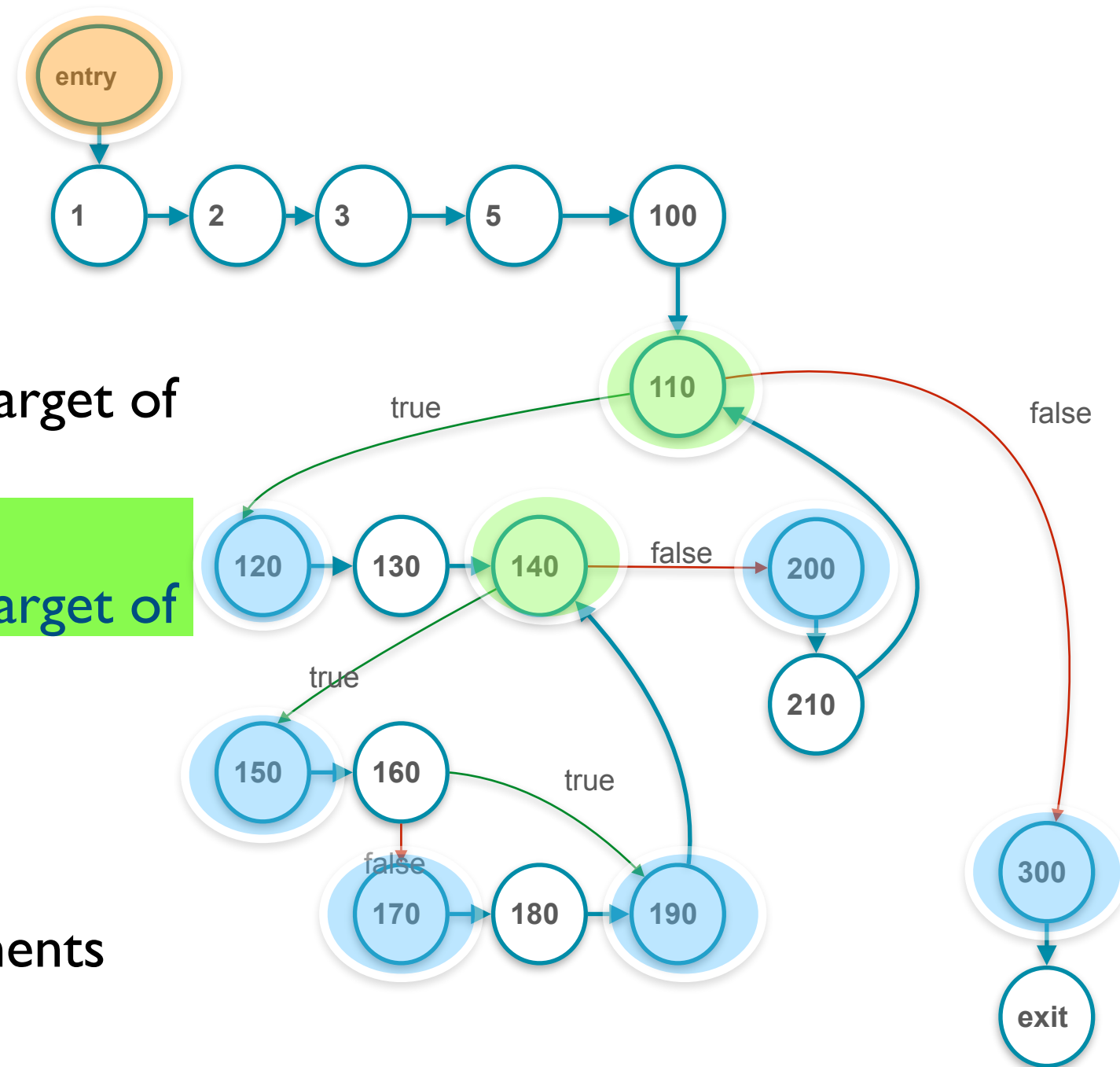
1. First statement

2. Every statement that is the target of a branch

3. Every statement that is the target of an implicit goto

2. Identify the basic blocks

1. Leader + sequence of statements until next leaders



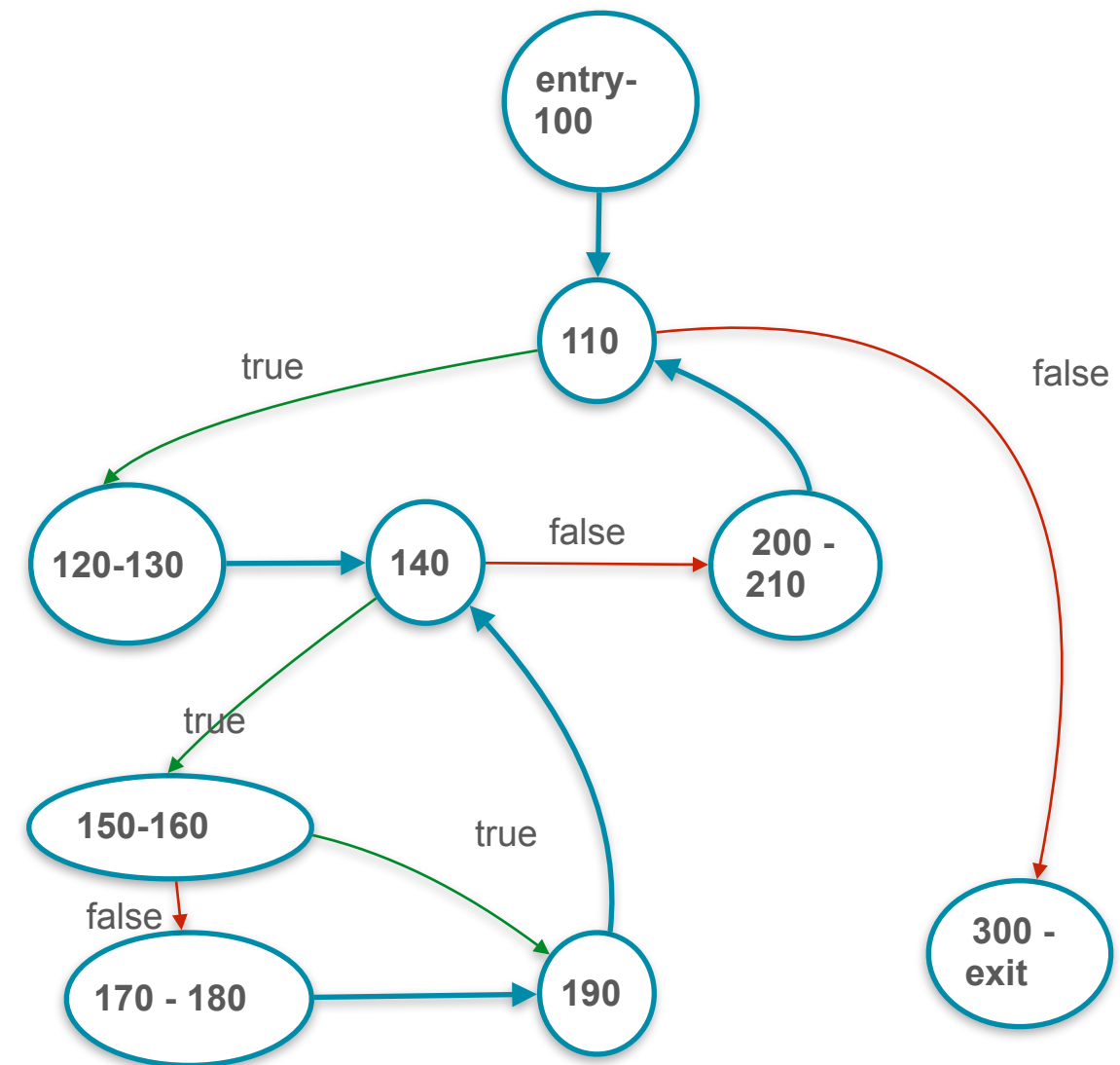
Identifying Basic Blocks

1. Identify leaders

1. First statement
2. Every statement that is the target of a branch
3. Every statement that is the target of an implicit goto

2. Identify the basic blocks

1. Leader + sequence of statements until next leaders

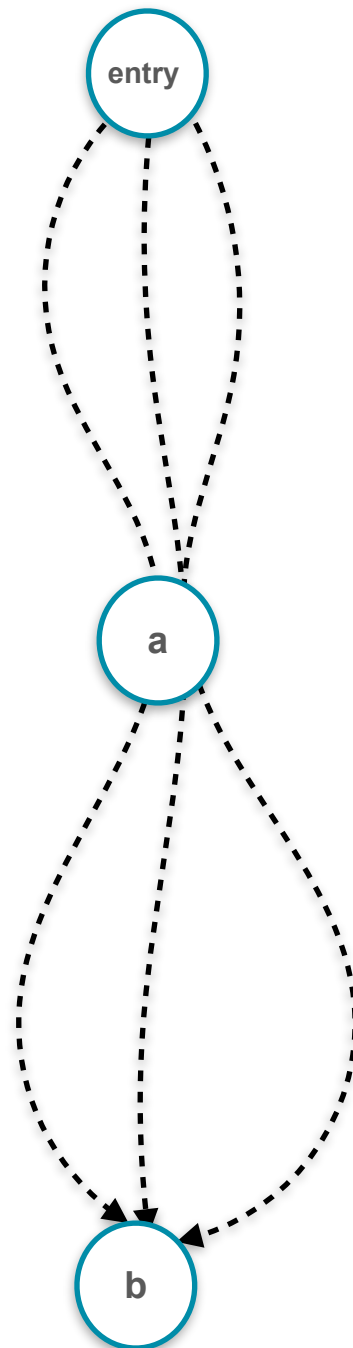


Outline

1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

Dominance

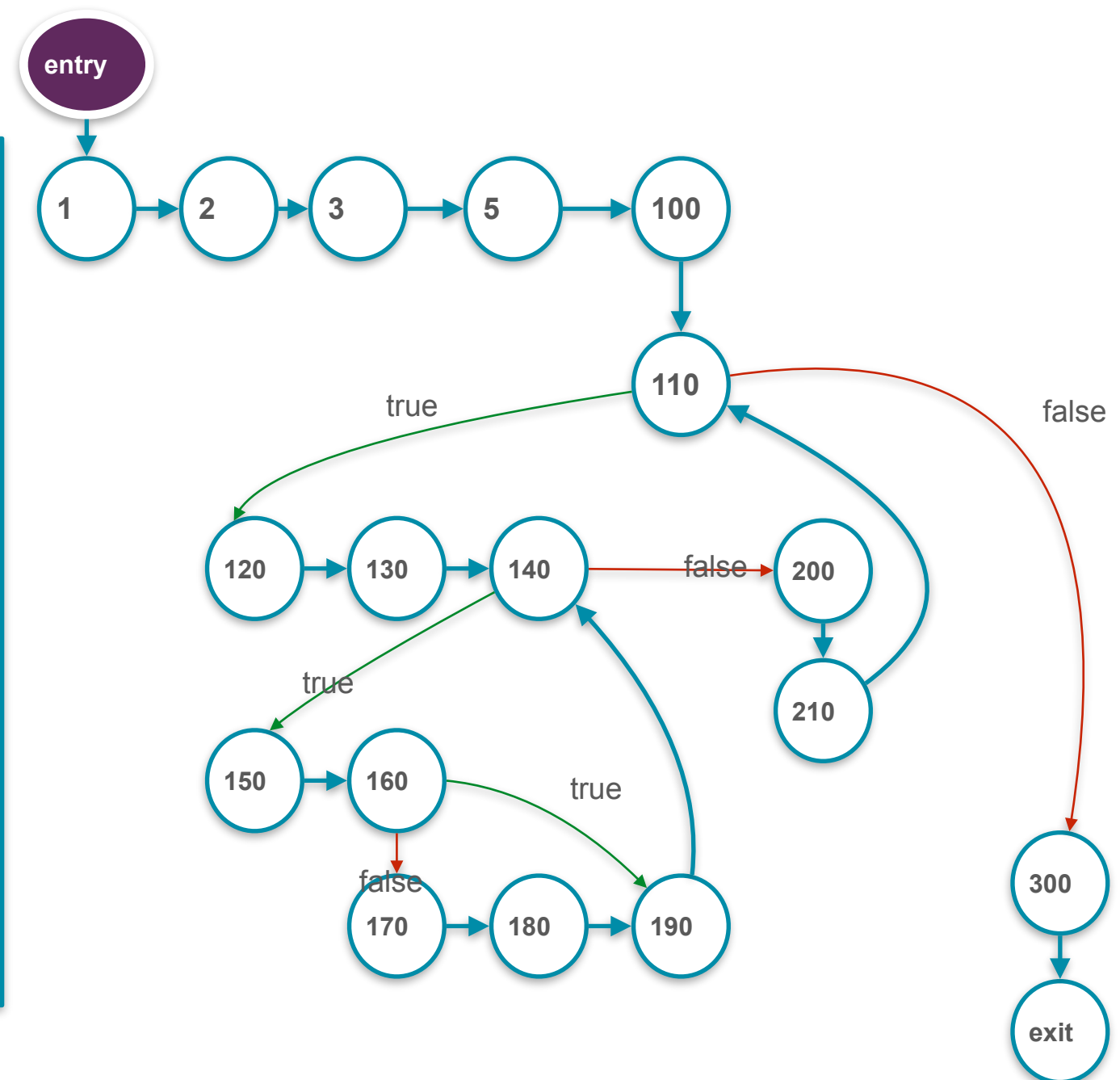
Node a **dominates** b if every path from entry to b must pass through a.



Entry node dominates all other nodes.
Every node dominates itself.
Entry node to a loop dominates all other nodes in the loop.

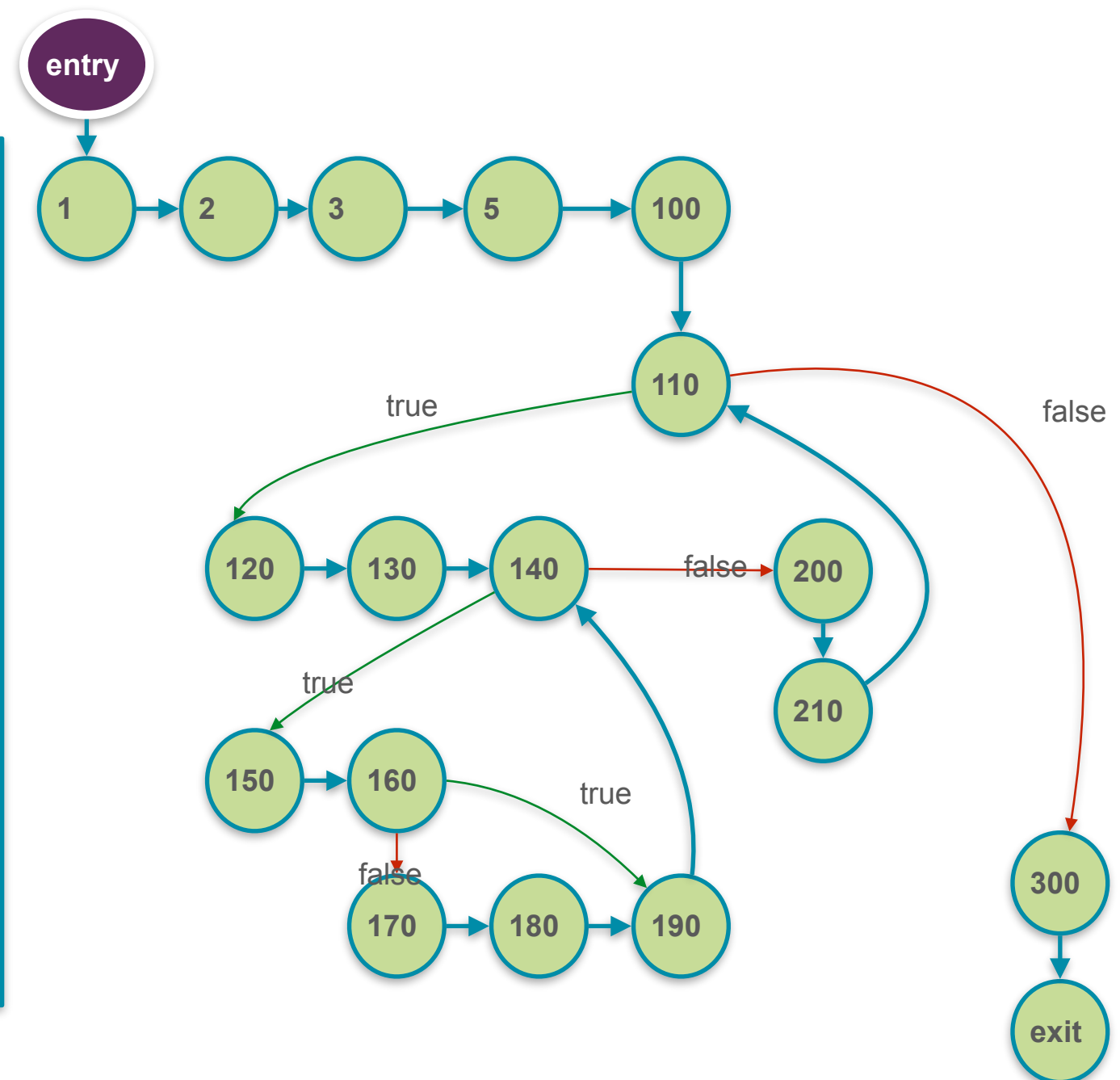
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



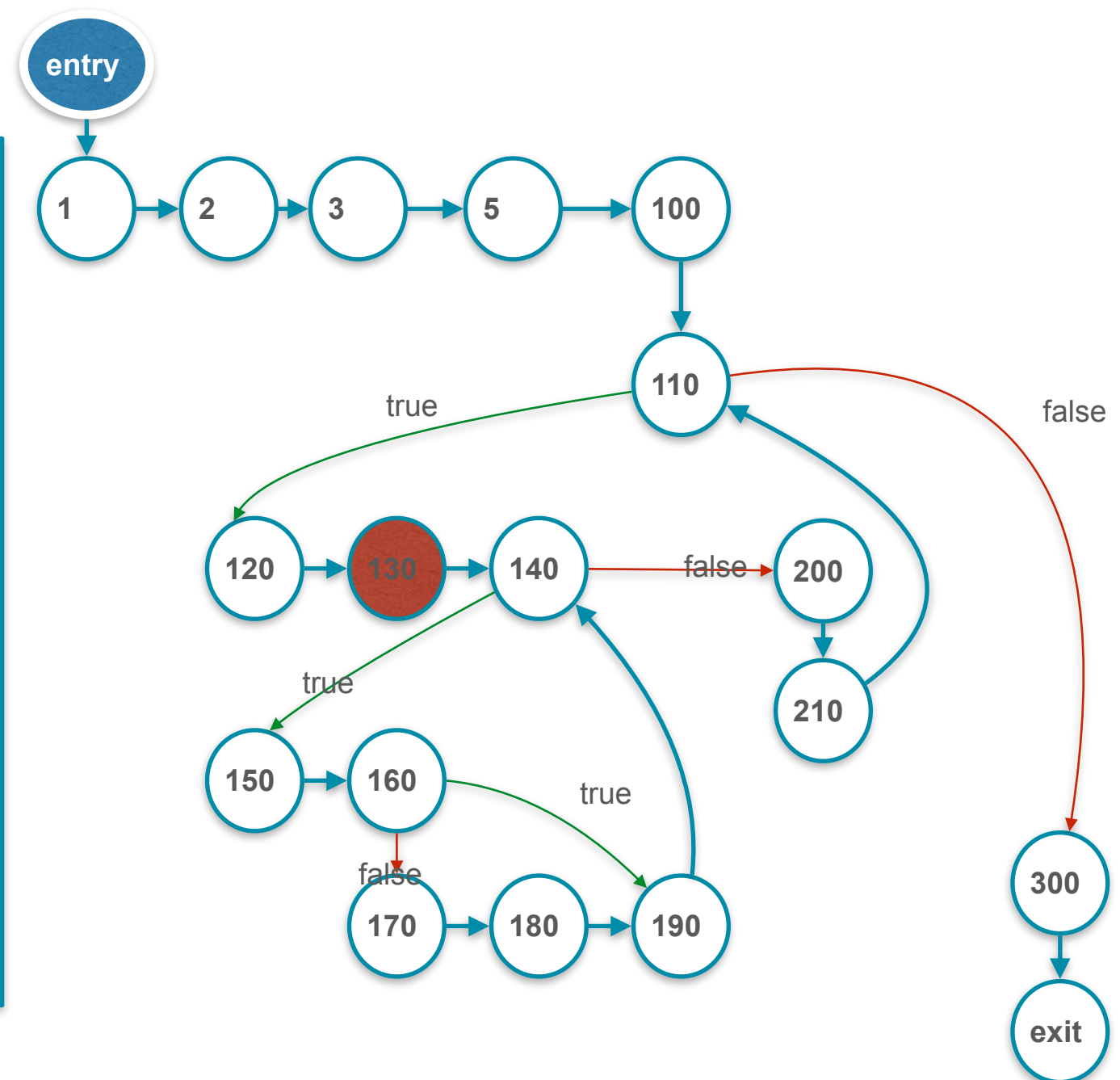
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



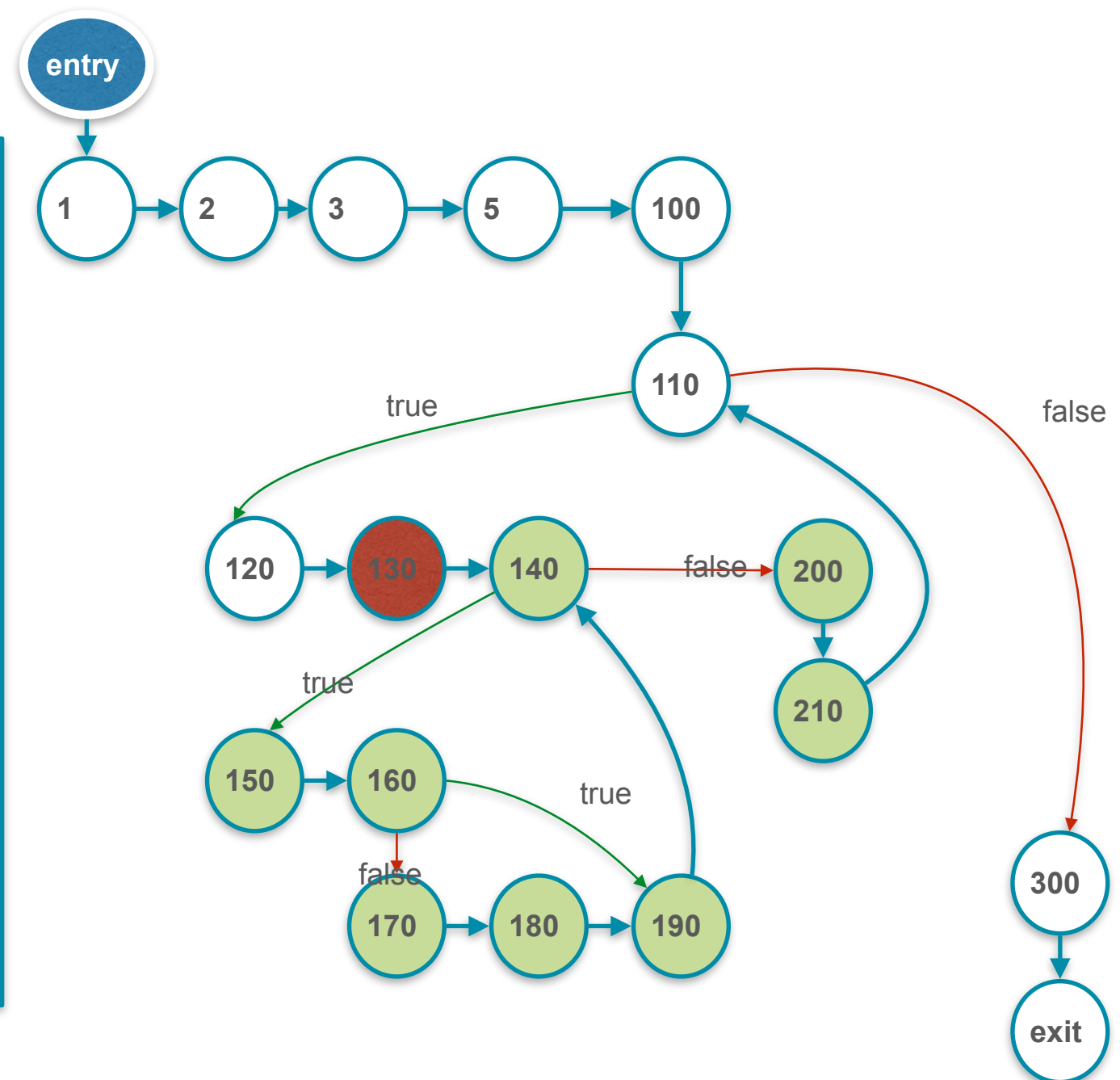
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



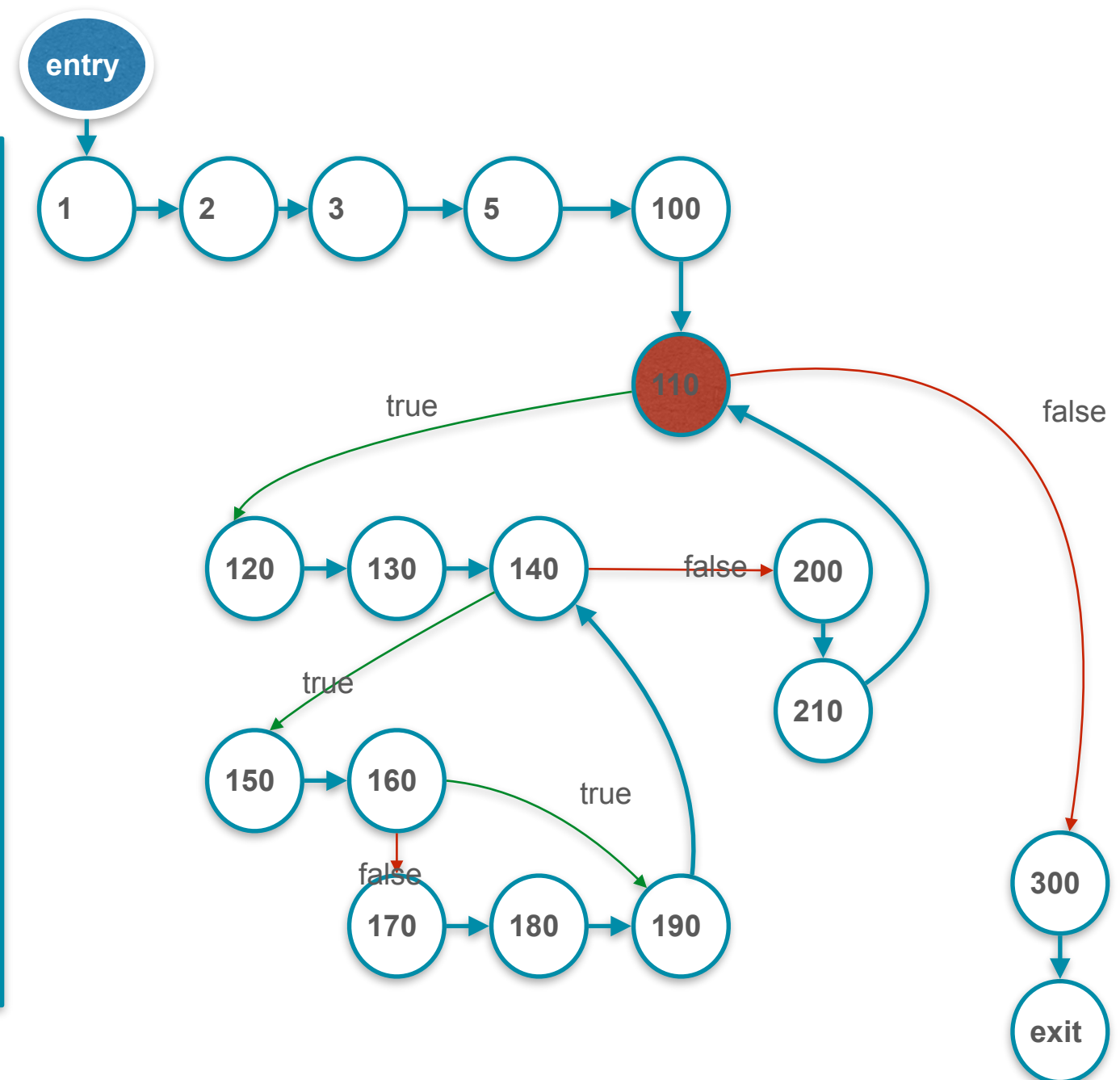
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



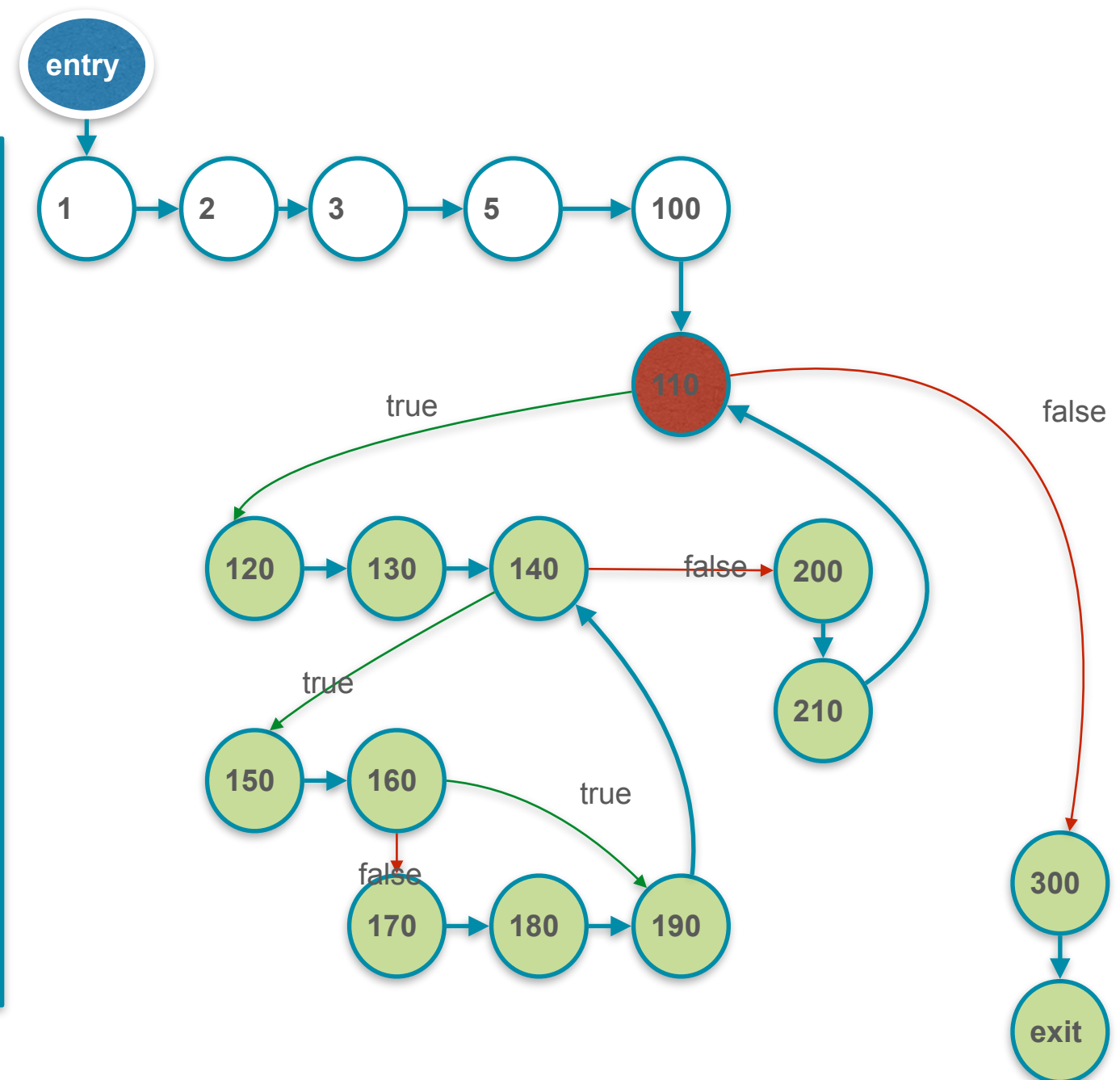
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



Rules

Node **a dominates b** if and only if:

a = b or

a is the unique predecessor to b or

a is the unique predecessor to any of b's predecessors

The dominator relation is:

Reflexive: *a dominates a* is true

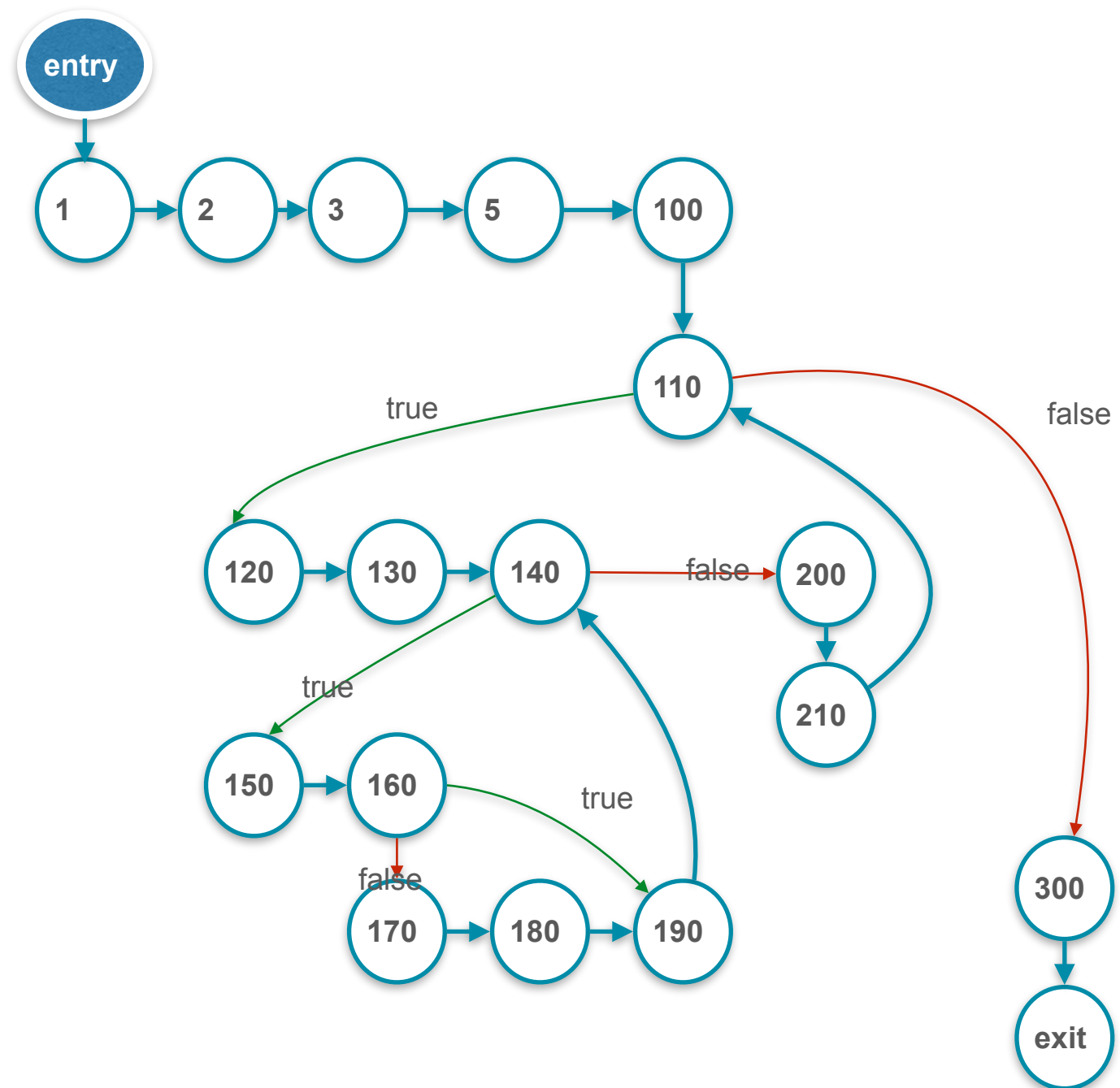
Transitive: If a dominates b and b dominates c, then a dominates c

Antisymmetric: If a dominates b, then b does not dominate a
(if a dom b and b dom a then b=a)

Immediate Dominator

The last dominator of a node n on any path from the entry to n .

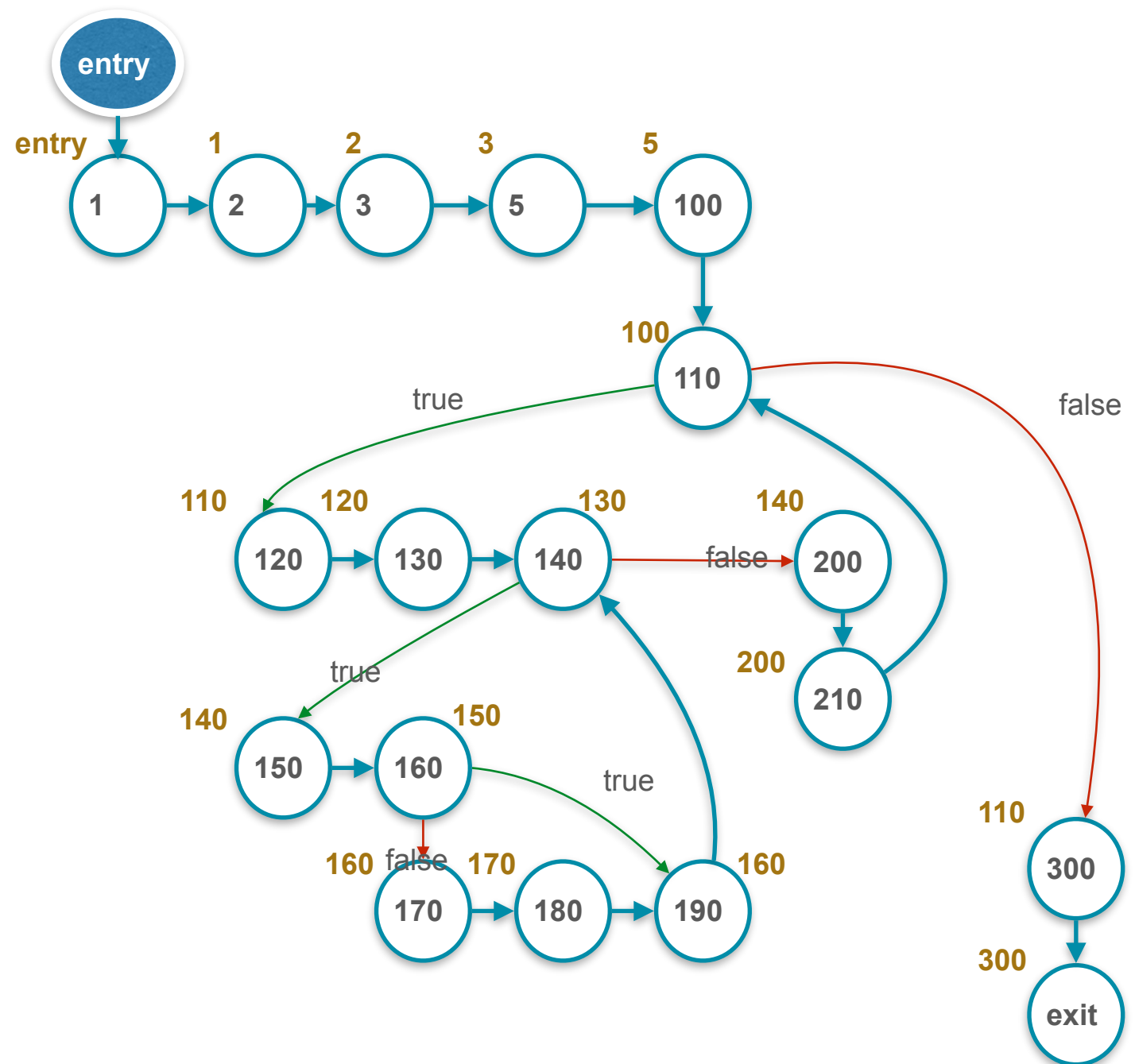
Immediate dominator (*idom*) of a node is **always unique**.



Immediate Dominator

The last dominator of a node n on any path from the entry to n .

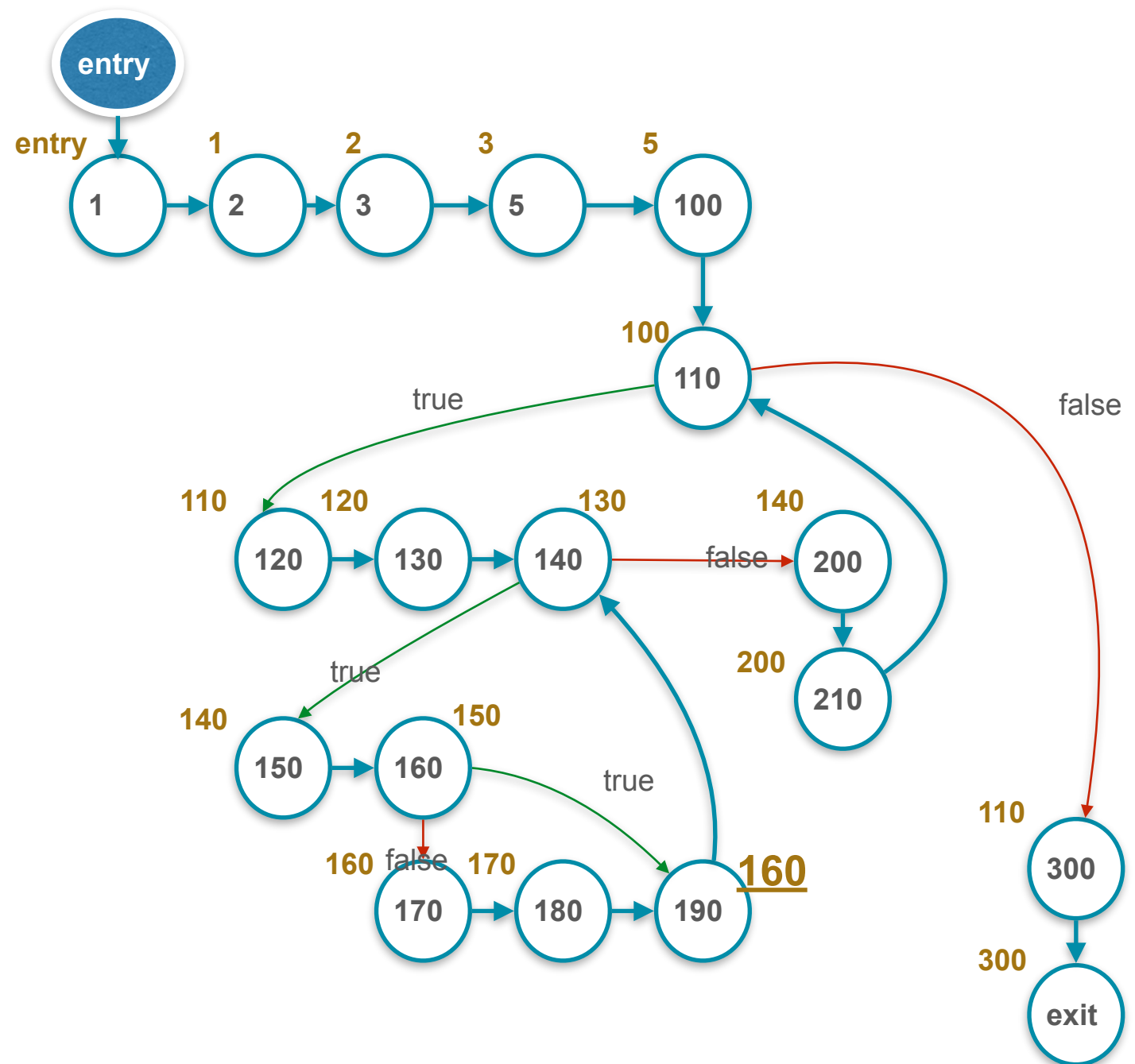
Immediate dominator (*idom*) of a node is **always unique**.



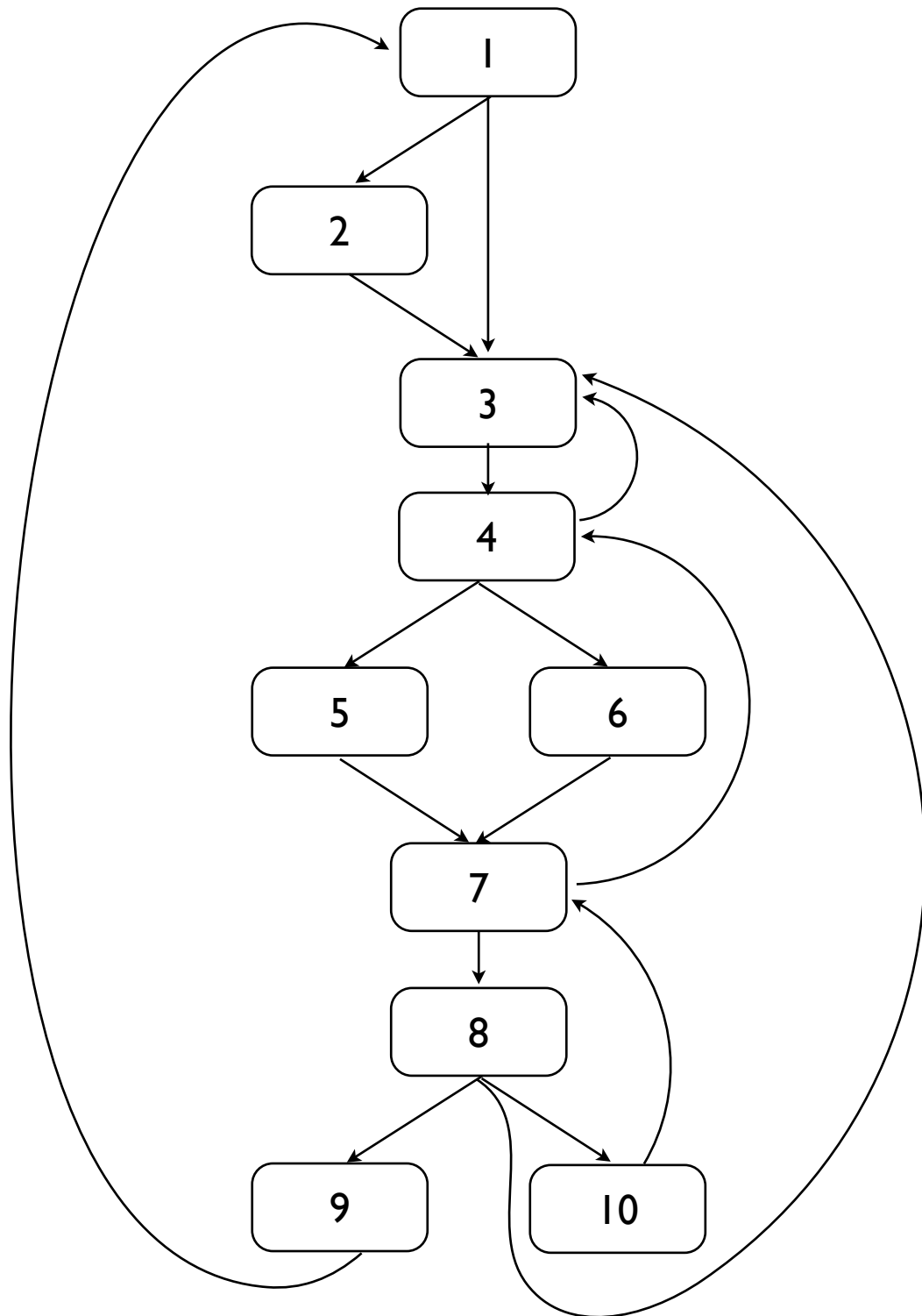
Immediate Dominator

The last dominator of a node n on any path from the entry to n .

Immediate dominator (*idom*) of a node is **always unique**.

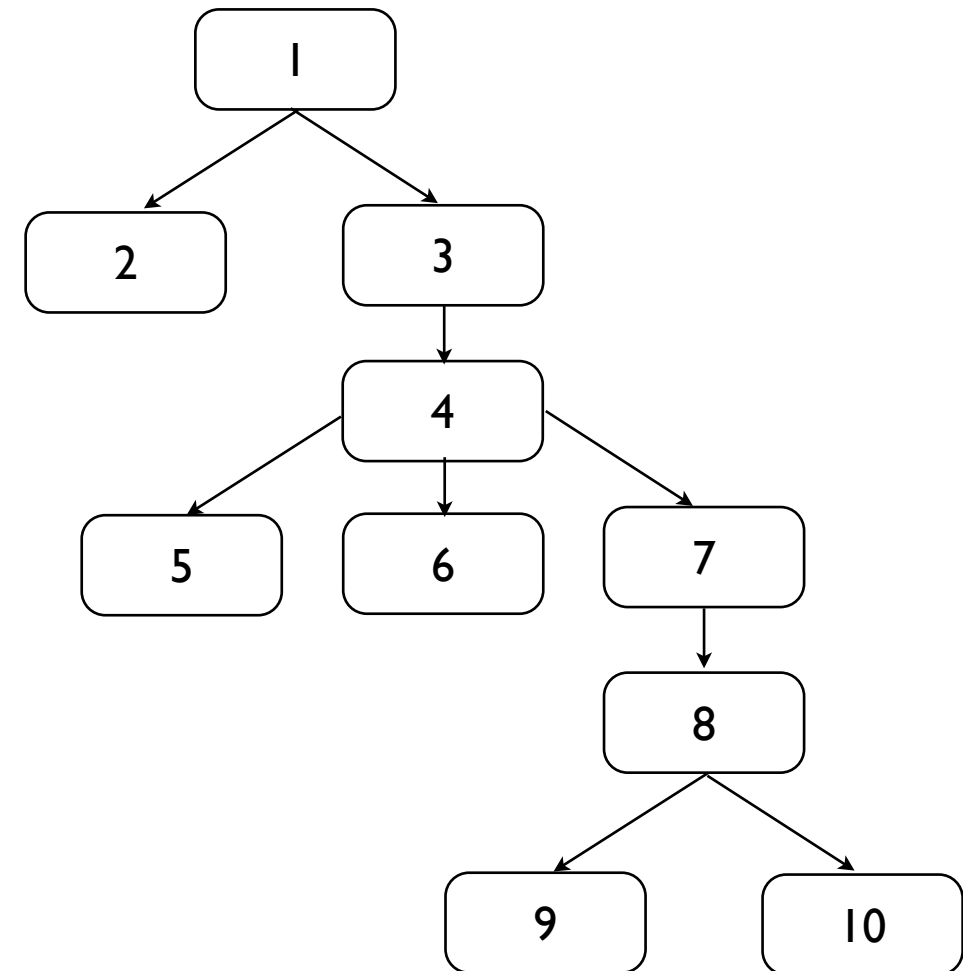
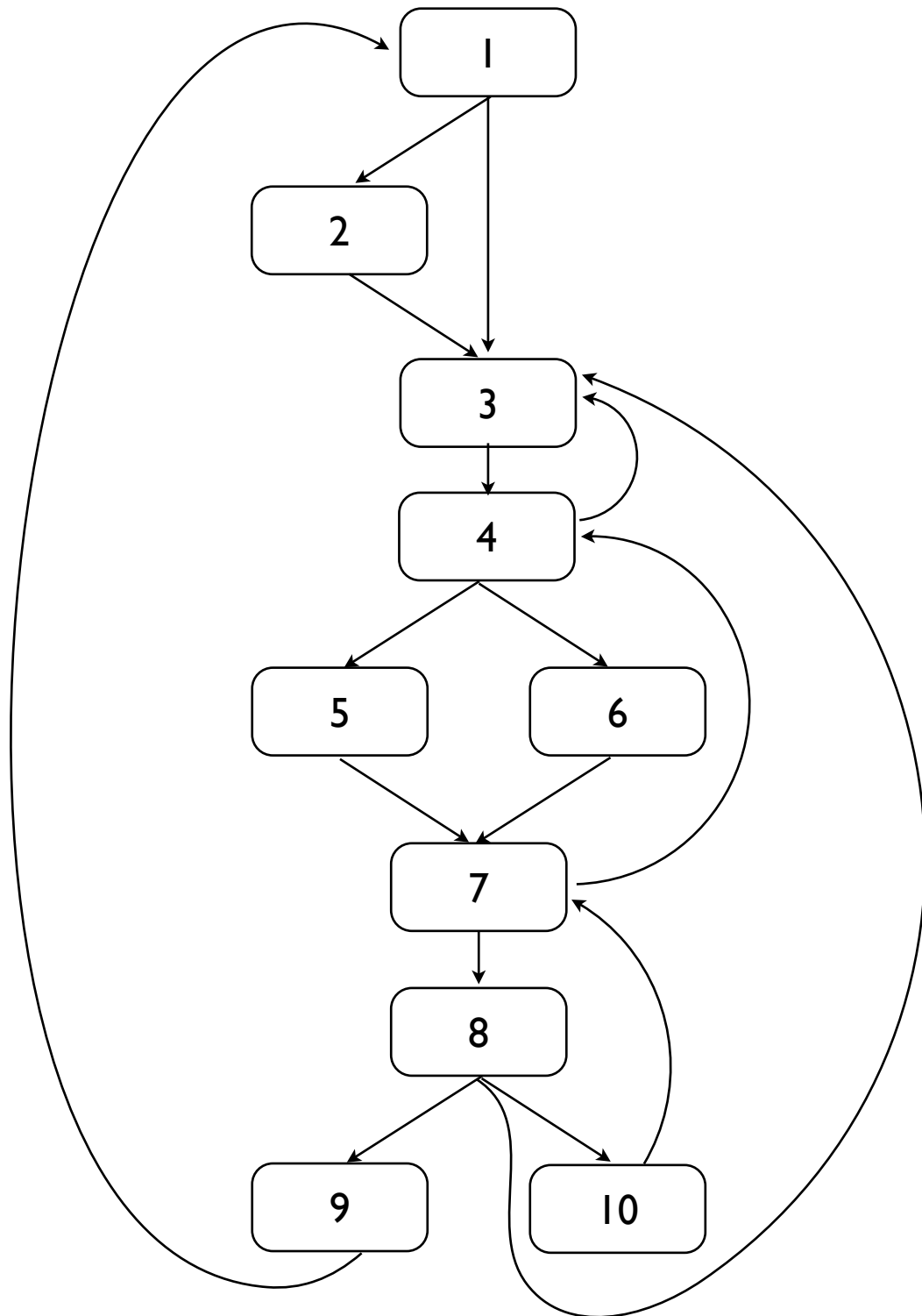


Dominator Example



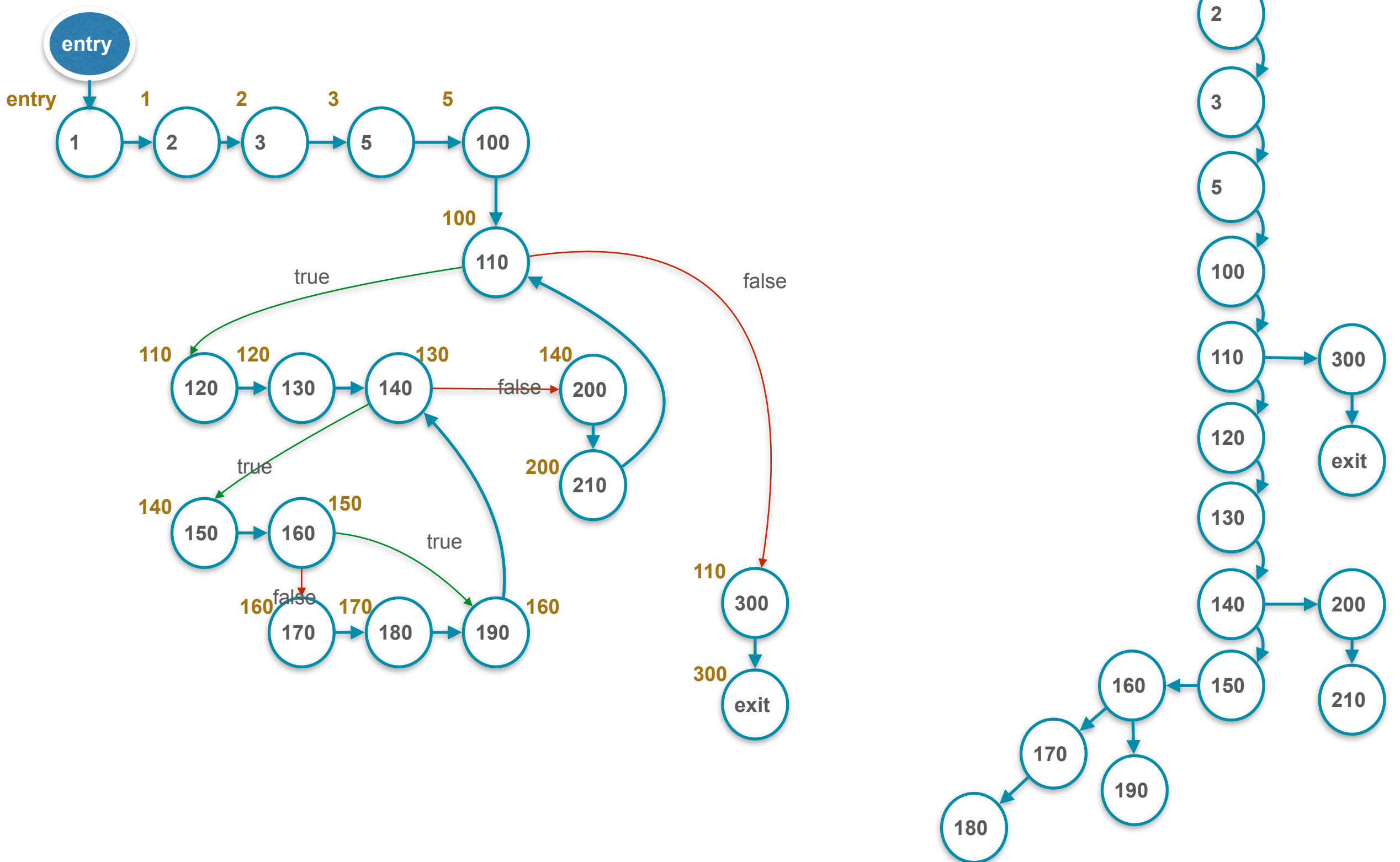
Block	Dom	IDom
1	{1}	-
2	{1,2}	1
3	{1,3}	1
4	{1,3,4}	3
5	{1,3,4,5}	4
6	{1,3,4,6}	4
7	{1,3,4,7}	4
8	{1,3,4,7,8}	7
9	{1,3,4,7,8,9}	8
10	{1,3,4,7,8,10}	8

Dominator Tree

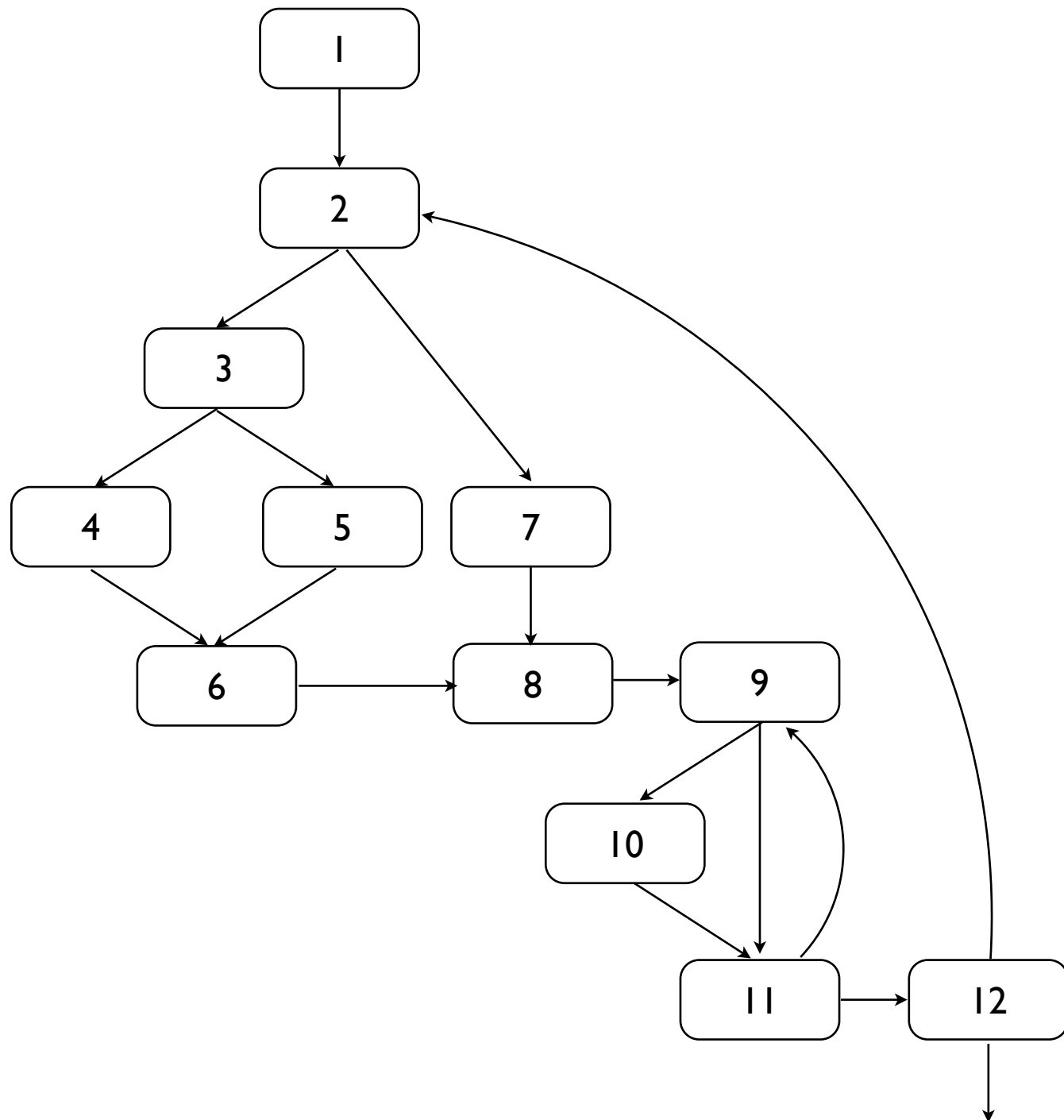


In a dominator tree, a node's parent is its immediate dominator

Dominator Tree



Dominator Example



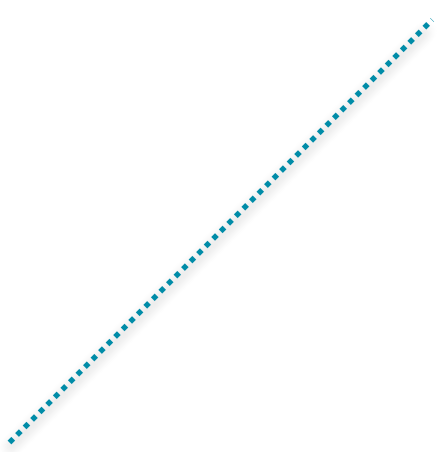
Block	Dom	IDom
1	{1}	-
2	{1,2}	1
3	{1,2,3}	2
4	{1,2,3,4}	3
5	{1,2,3,5}	3
6	{1,2,3,6}	3
7	{1,2,7}	2
8	{1,2,8}	2
9	{1,2,8,9}	8
10	{1,2,8,9,10}	9
11	{1,2,8,9,11}	9
12	{1,2,8,9,11,12}	11

Computing Dominators

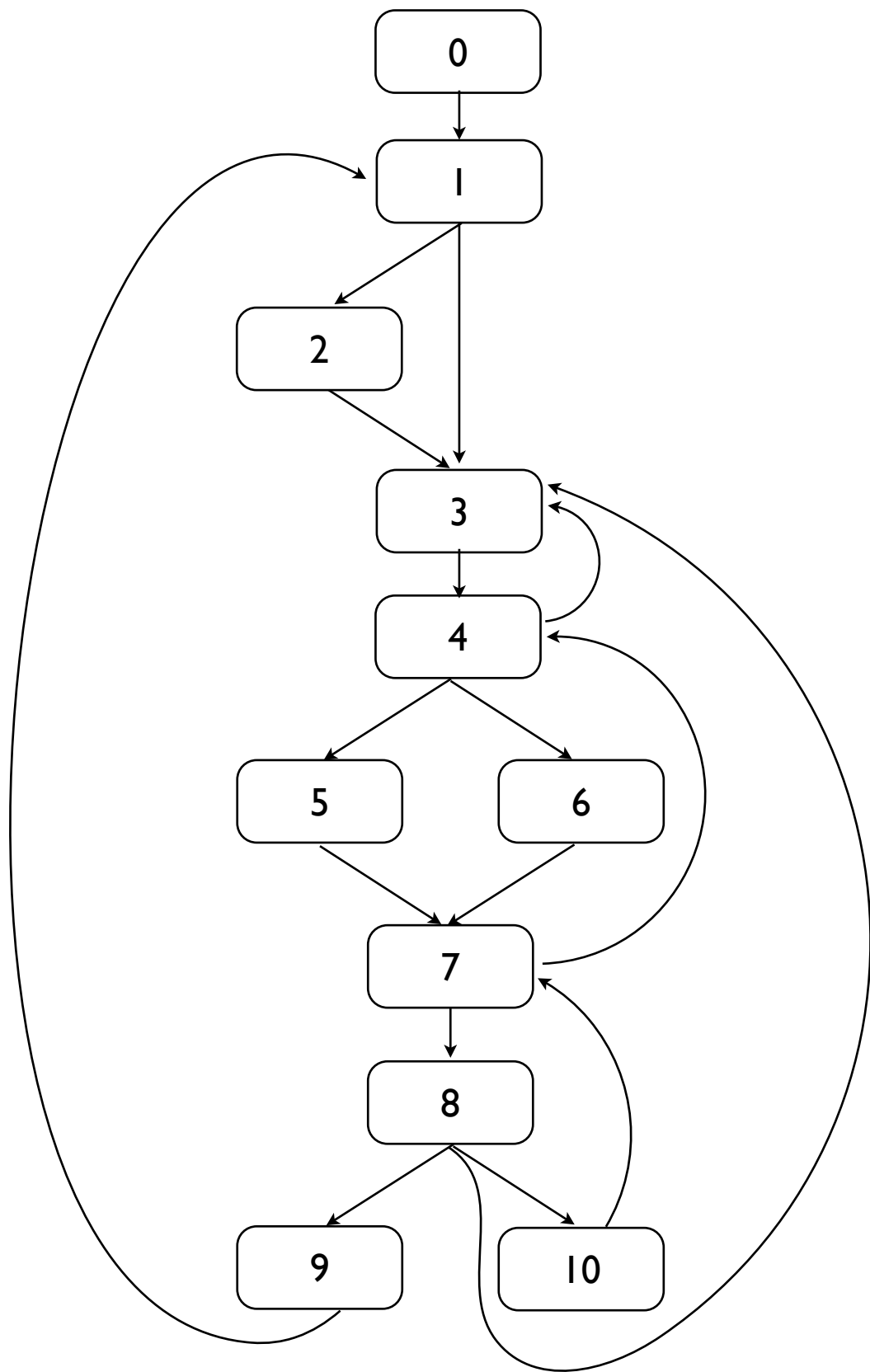
Result: A map of dependence relations D

```
dominance( $CFG$ ) begin
   $entry \leftarrow entryNode(CFG);$ 
   $D(entry) \leftarrow entry;$ 
  for  $n \in CFG.nodes - \{entry\}$  do
     $D(n) \leftarrow CFG.nodes;$ 
  end
   $changed \leftarrow true;$ 
  while  $changed$  do
     $changed \leftarrow false;$ 
    for  $n \in CFG.nodes - \{entry\}$  do
       $currentDominators \leftarrow D(n);$ 
       $newDominators \leftarrow n \cup (\bigcap D(p))$  for all immediate predecessors  $p$  of  $n$ ;
      if  $currentDominators \neq newDominators$  then
         $D(n) \leftarrow newDominators;$ 
         $changed \leftarrow true;$ 
      end
    end
  end
  return  $D$ 
end
```

Set intersection:
only include nodes that
belong to *all* sets $D(p)$

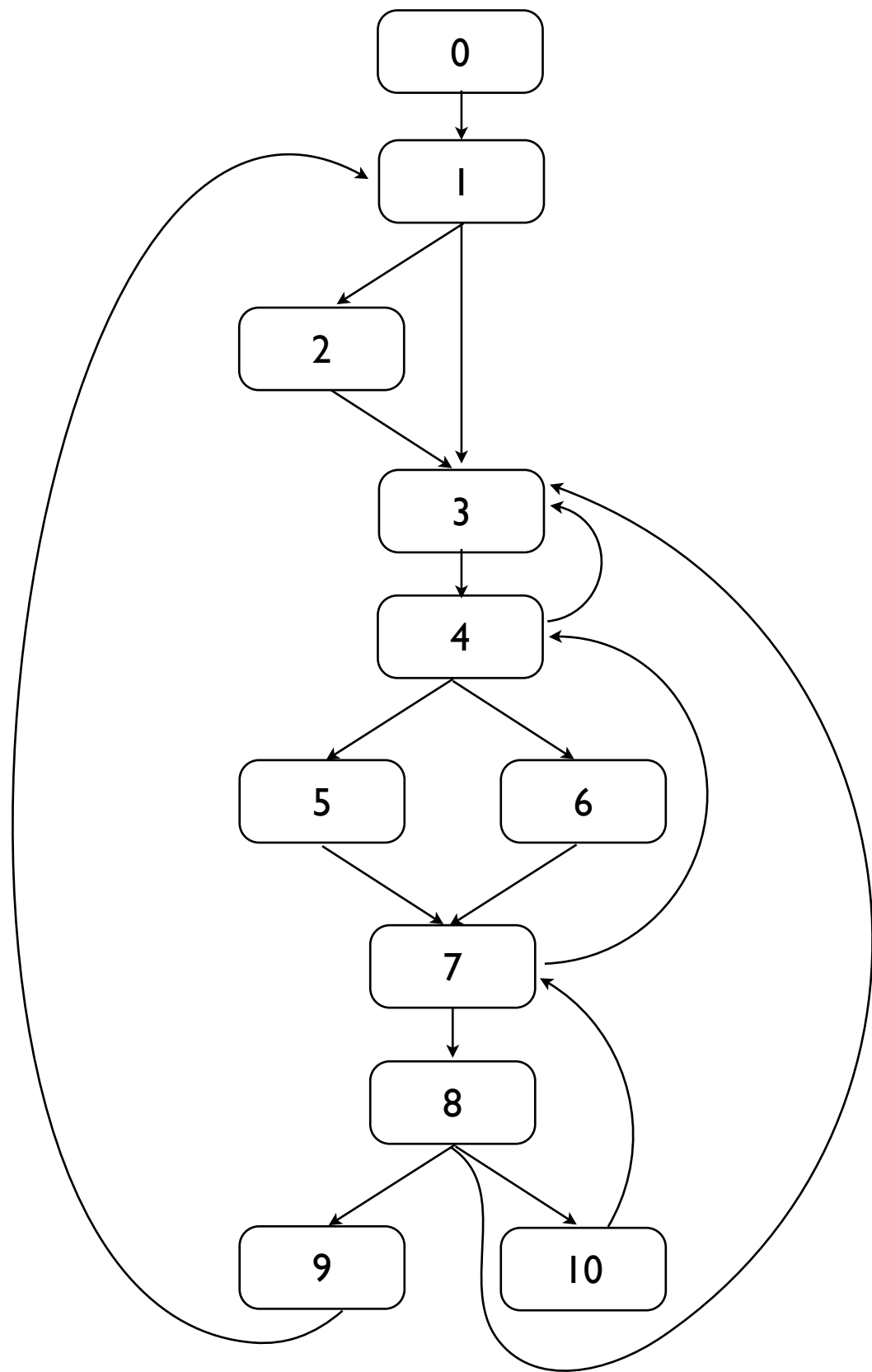


Dominator Example



Block	Initial	Iteration 1
0	{0}	{0}
1	N	{1}+DOM(0)∩DOM(9)
2	N	{2}+DOM(1)
3	N	{3}+DOM(2)∩DOM(1)∩DOM(4)∩DOM(8)
4	N	{4}+DOM(3)∩DOM(7)
5	N	{5}+DOM(4)
6	N	{6}+DOM(4)
7	N	{7}+DOM(5)∩DOM(6)∩DOM(10)
8	N	{8}+DOM(7)
9	N	{9}+DOM(8)
10	N	{10}+DOM(8)

Dominator Example

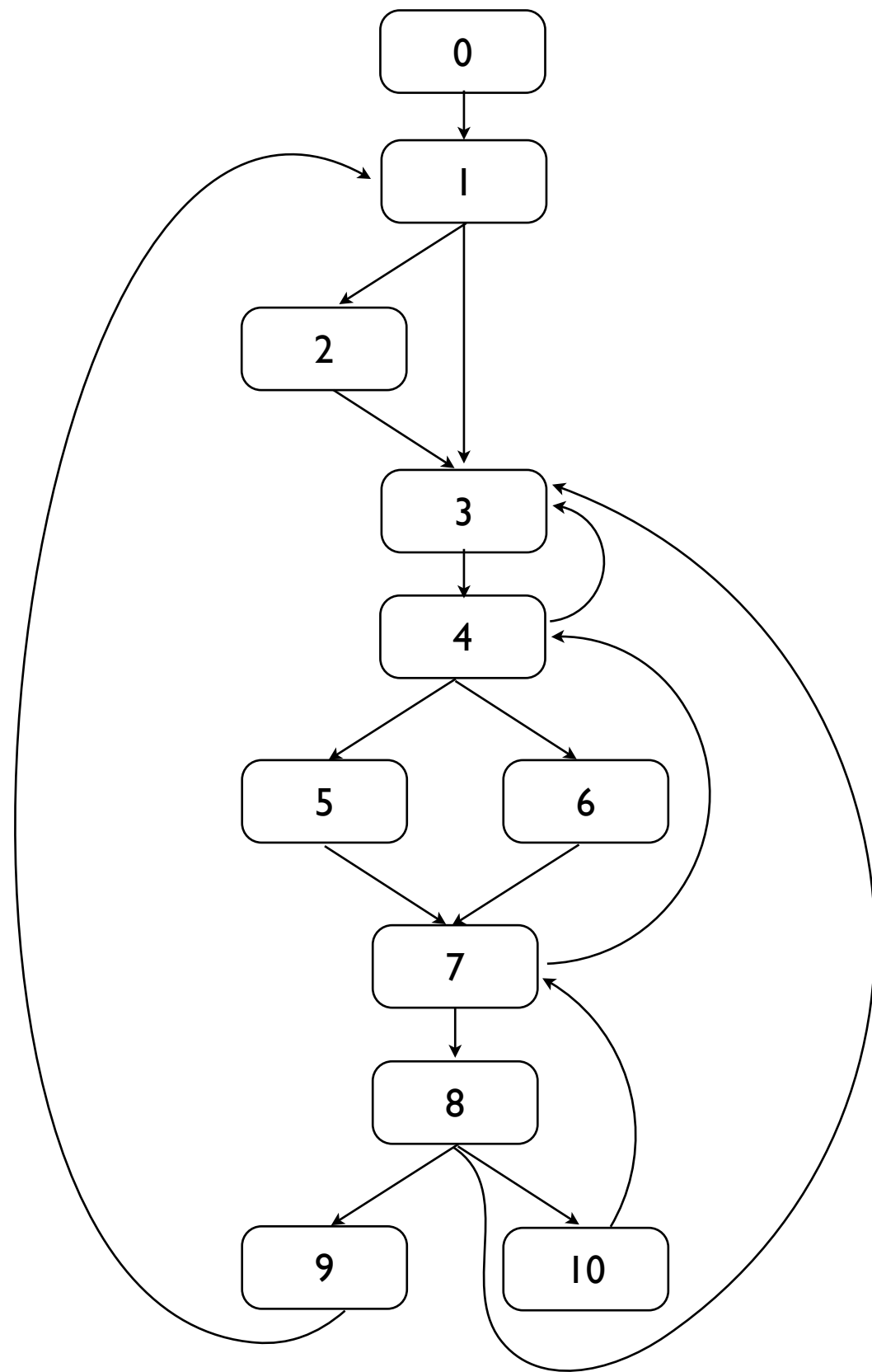


Block	Iteration 1	Iteration 2
0	{0}	{0}
1	{0,1}	{0,1}
2	{0,1,2}	{0,1,2}
3	{0,1,3}	{0,1,3}
4	{0,1,3,4}	{0,1,3,4}
5	{0,1,3,4,5}	{0,1,3,4,5}
6	{0,1,3,4,6}	{0,1,3,4,6}
7	{0,1,3,4,7}	{0,1,3,4,7}
8	{0,1,3,4,7,8}	{0,1,3,4,7,8}
9	{0,1,3,4,7,8,9}	{0,1,3,4,7,8,9}
10	{0,1,3,4,7,8,10}	{0,1,3,4,7,8,10}

Computing IDOM from DOM

- For each node n , initially set $IDOM(n) = DOM(n) - \{n\}$ (SDOM — strict dominators)
- For each node p in $IDOM(n)$, see if p has dominators other than itself also included in $IDOM(n)$: If so, remove it from $IDOM(n)$
- The immediate dominator m of n is the strict dominator of n that is closest to n

I-Dominator Example



Block	SDOM	IDOM
0	$\{\}$	$\{\}$
1	$\{0\}$	$\{0\}$
2	$\{0,1\}$	$\{1\}$
3	$\{0,1\}$	$\{1\}$
4	$\{0,1,3\}$	$\{3\}$
5	$\{0,1,3,4\}$	$\{4\}$
6	$\{0,1,3,4\}$	$\{4\}$
7	$\{0,1,3,4\}$	$\{4\}$
8	$\{0,1,3,4,7\}$	$\{7\}$
9	$\{0,1,3,4,7,8\}$	$\{8\}$
10	$\{0,1,3,4,7,8\}$	$\{8\}$

Outline

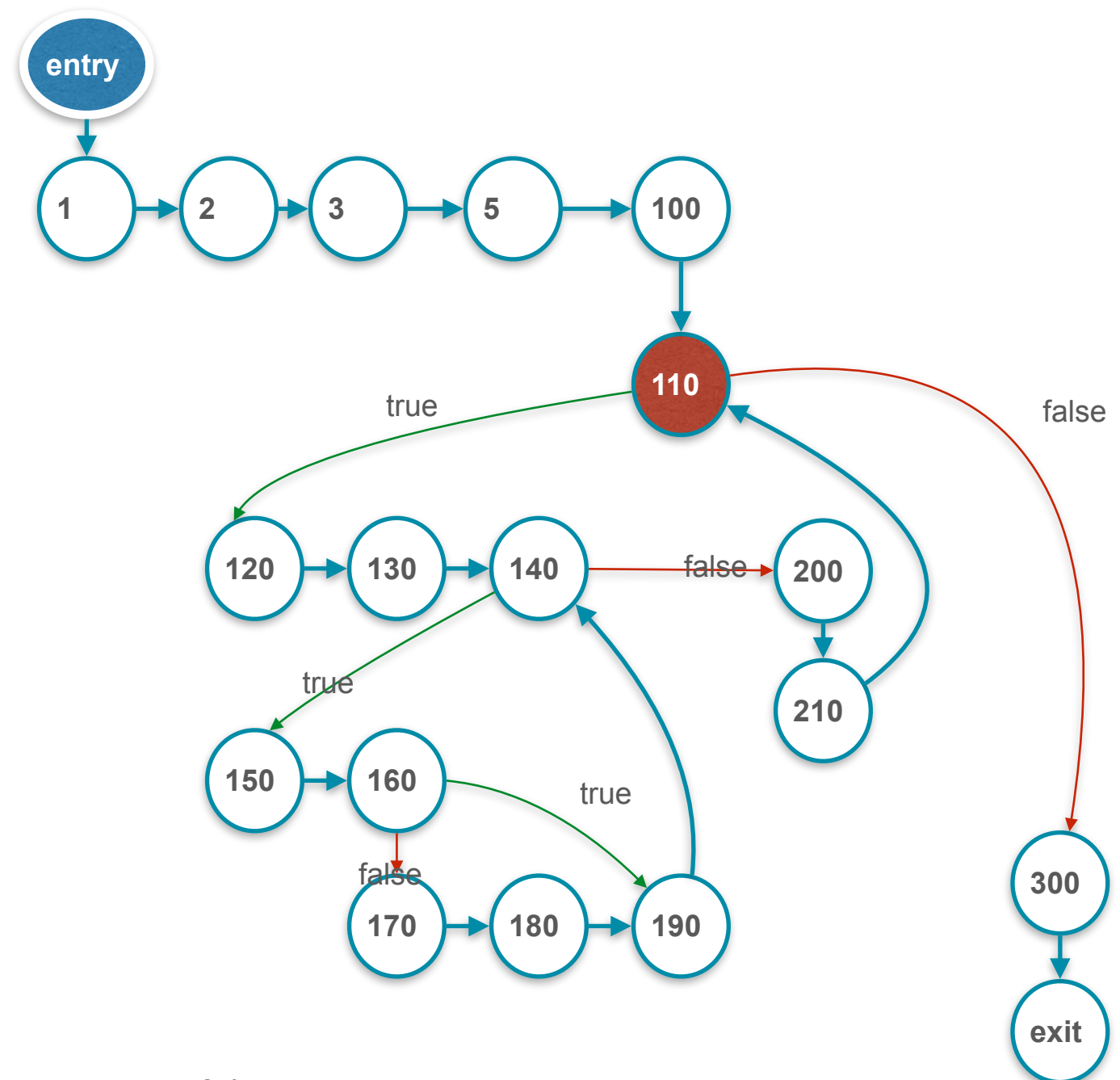
1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

Post-Dominance

- Node d of a CFG **post-dominates** node n if every path from n to the exit node passes through d ($d \text{ pdom } n$)
- $\text{Pdom}(n)$: the set of post-dominators of node n
- Every node post-dominates itself: $n \in \text{Pdom}(n)$
- Each node n has a unique immediate post dominator m

Example

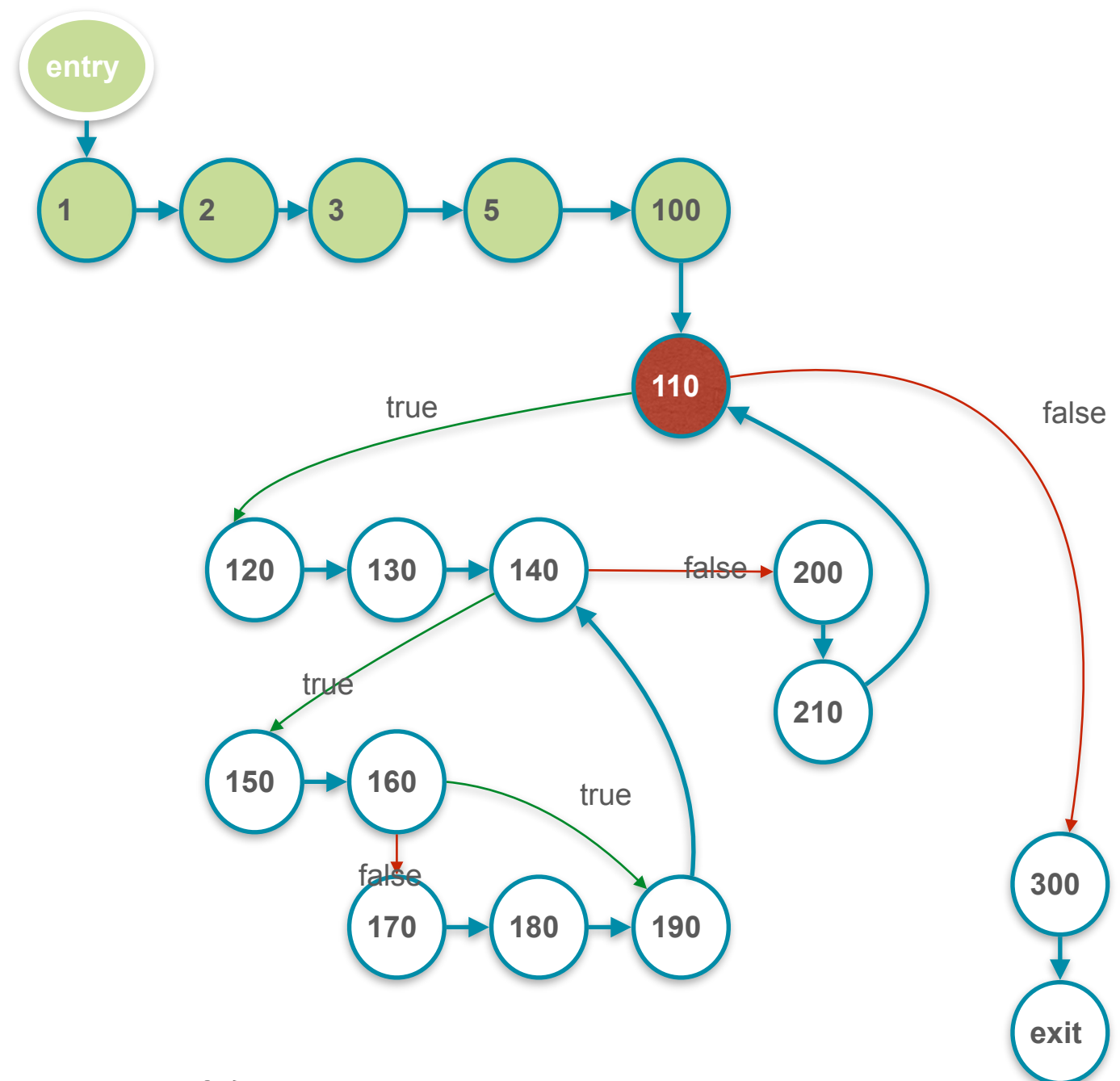
Node b **post-dominates** a if every path from a to exit node passes through b.



Which nodes are post-dominated by 110?

Example

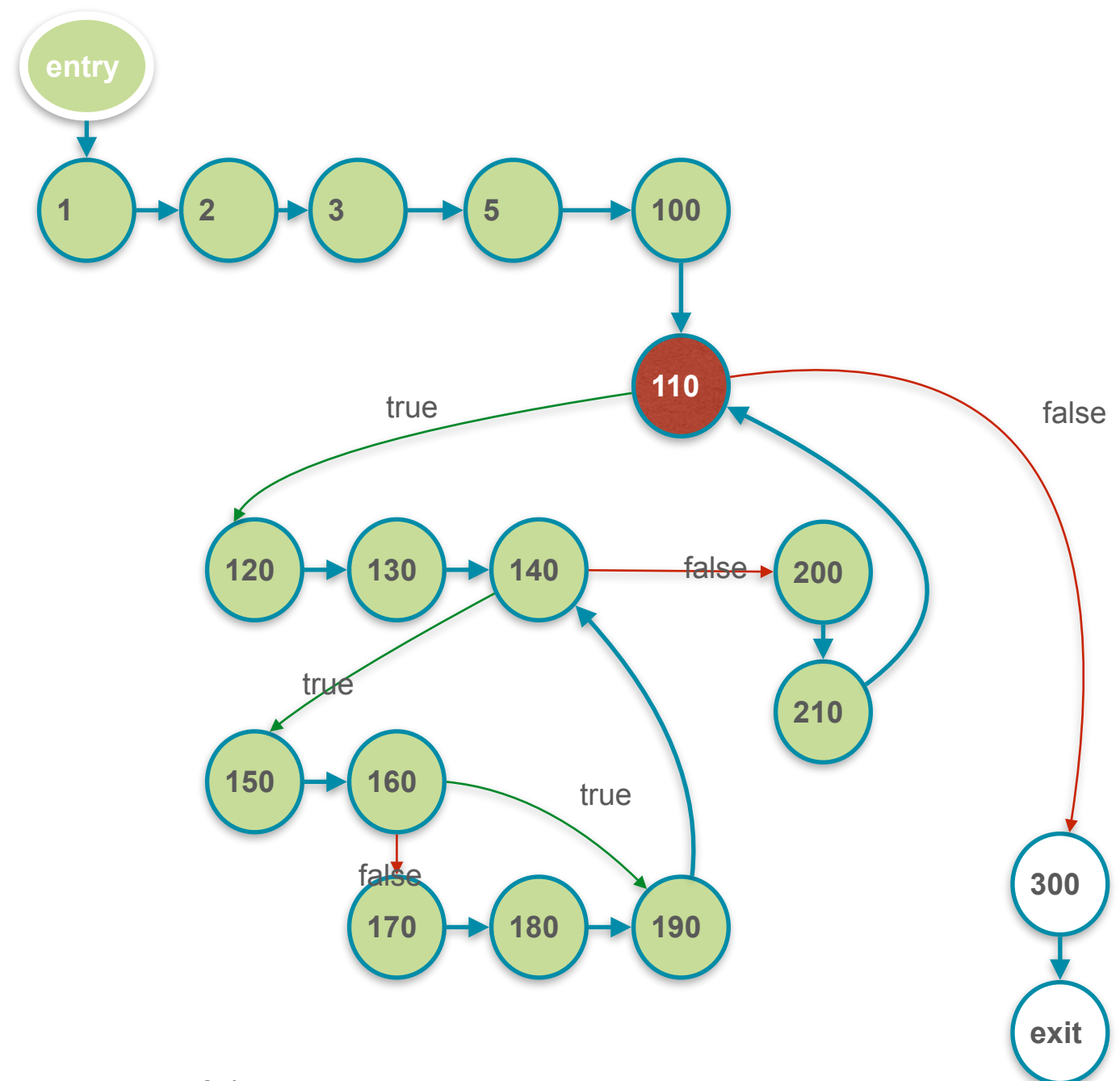
Node b **post-dominates** a if every path from a to exit node passes through b.



Which nodes are post-dominated by 110?

Example

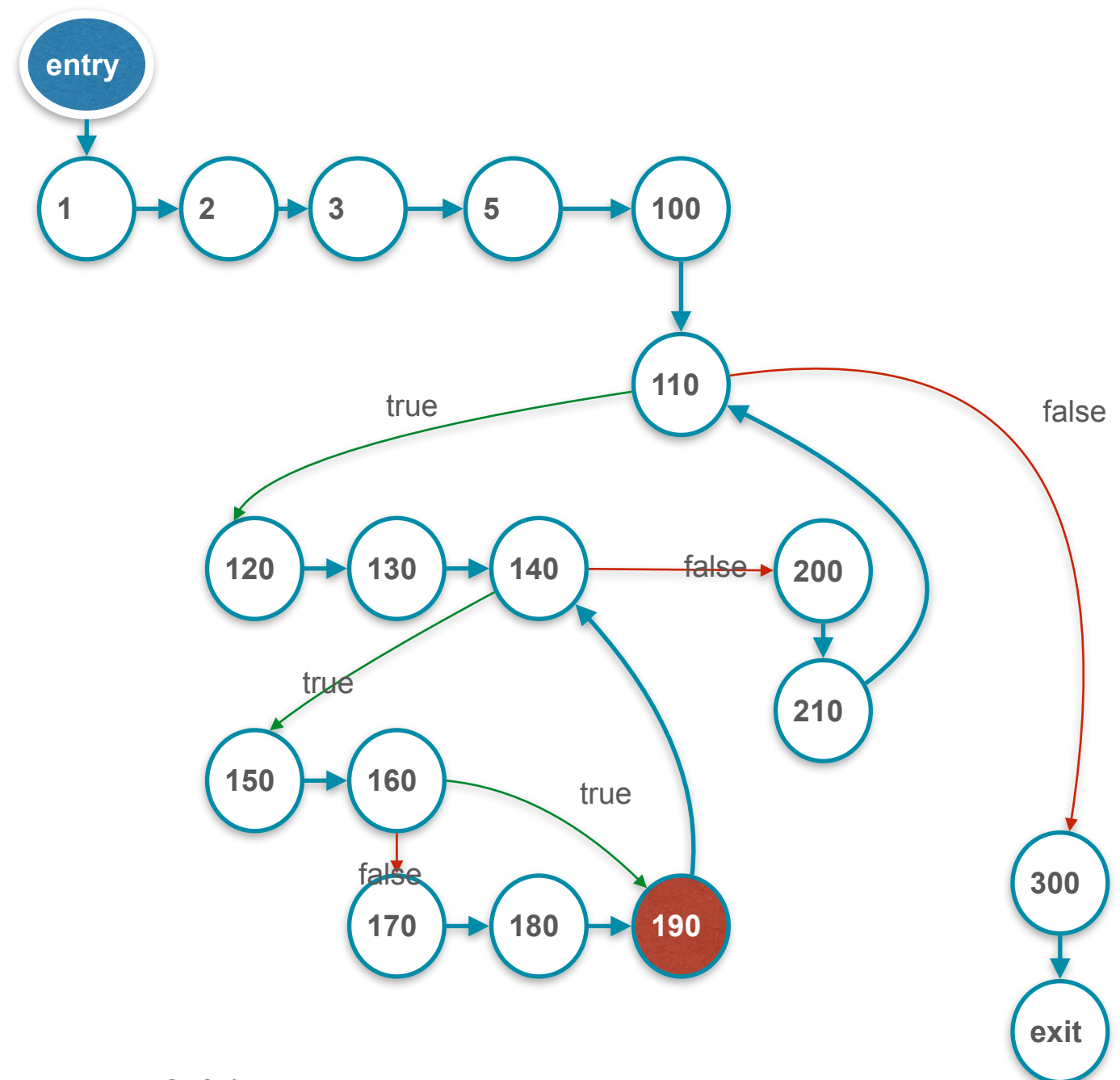
Node b **post-dominates** a if every path from a to exit node passes through b.



Which nodes are post-dominated by 110?

Example (2)

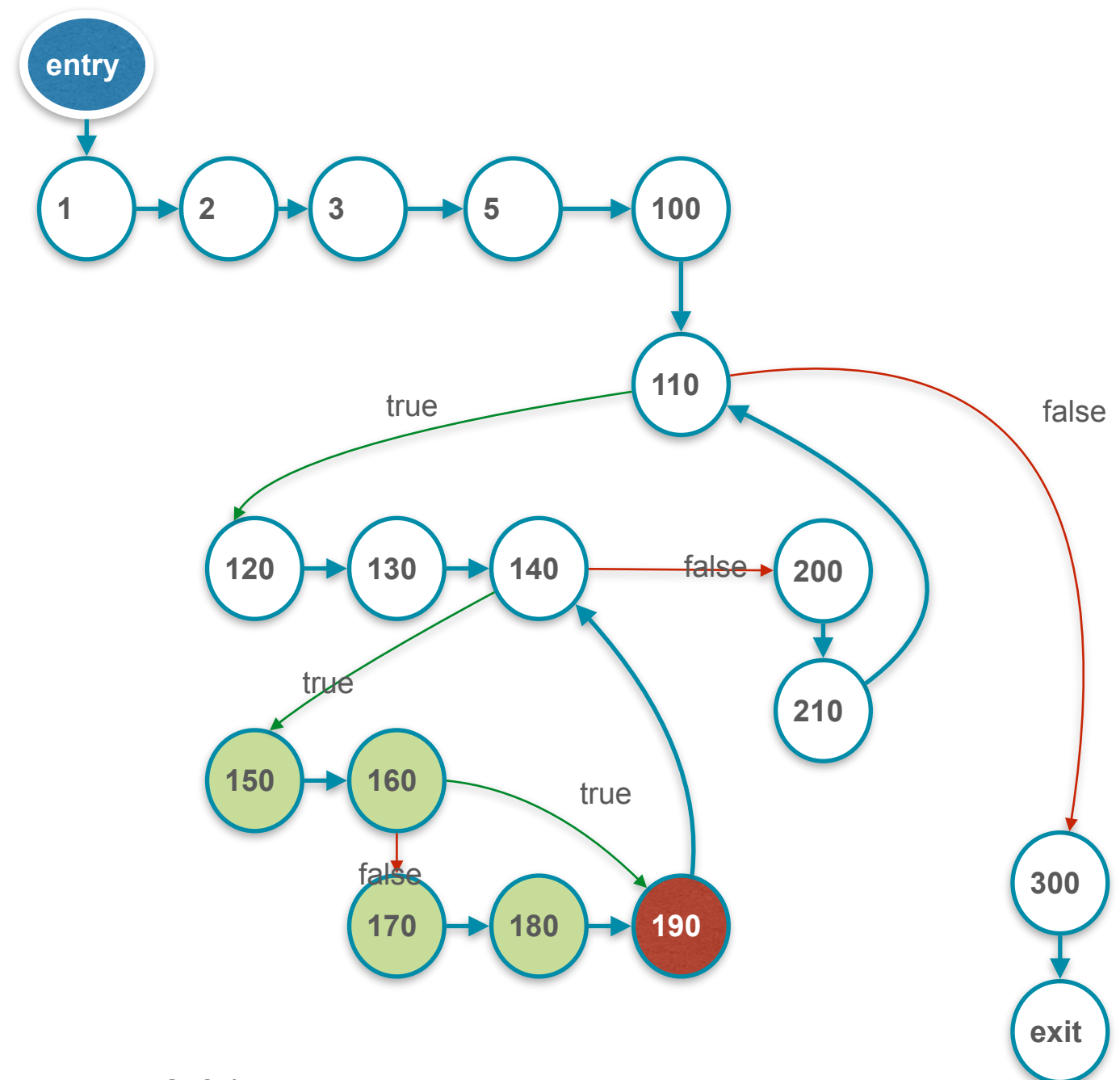
Node b **post-dominates** a if every path from a to exit node passes through b.



Which nodes are post-dominated by 190?

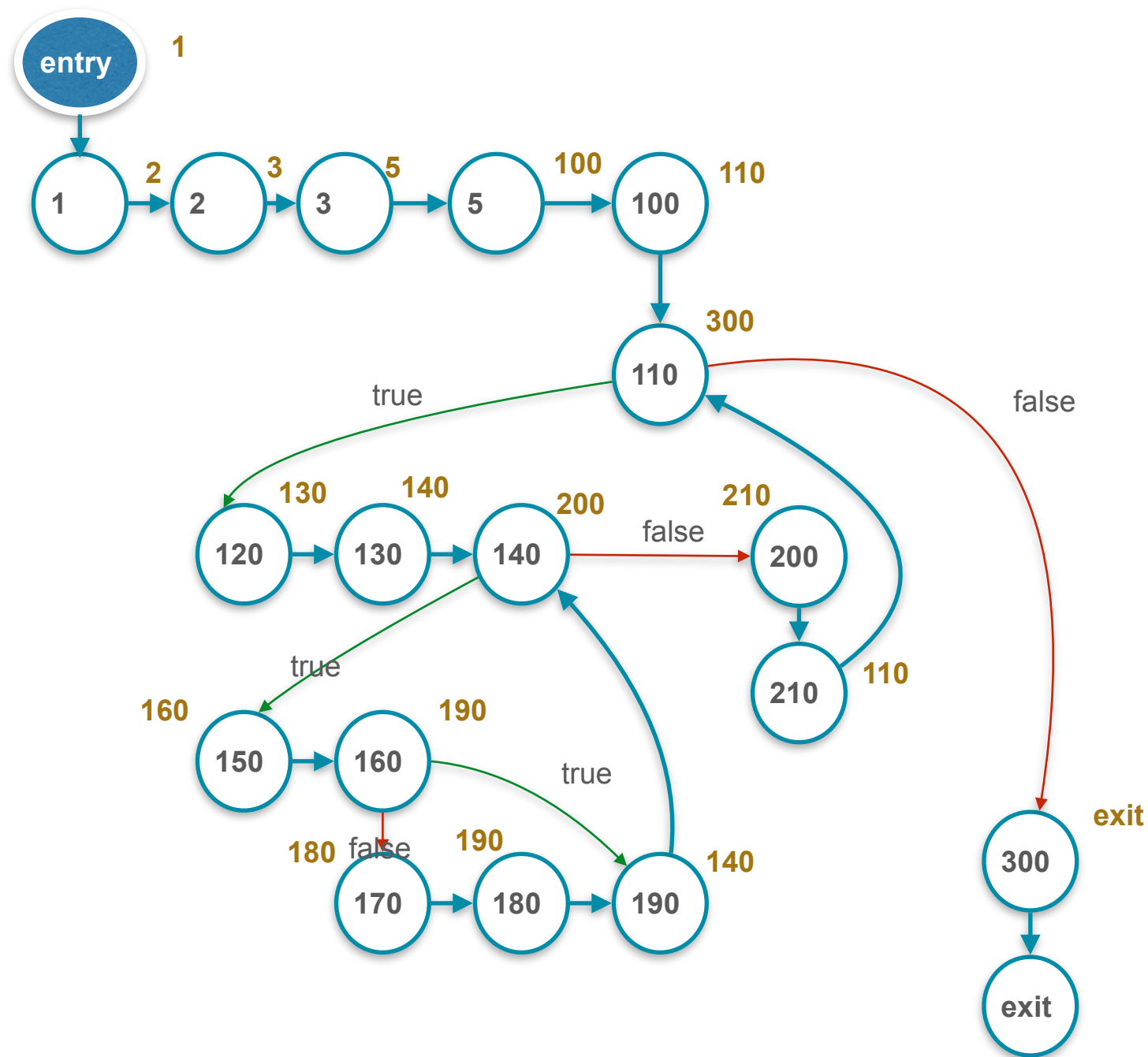
Example (2)

Node b **post-dominates** a if every path from a to exit node passes through b.

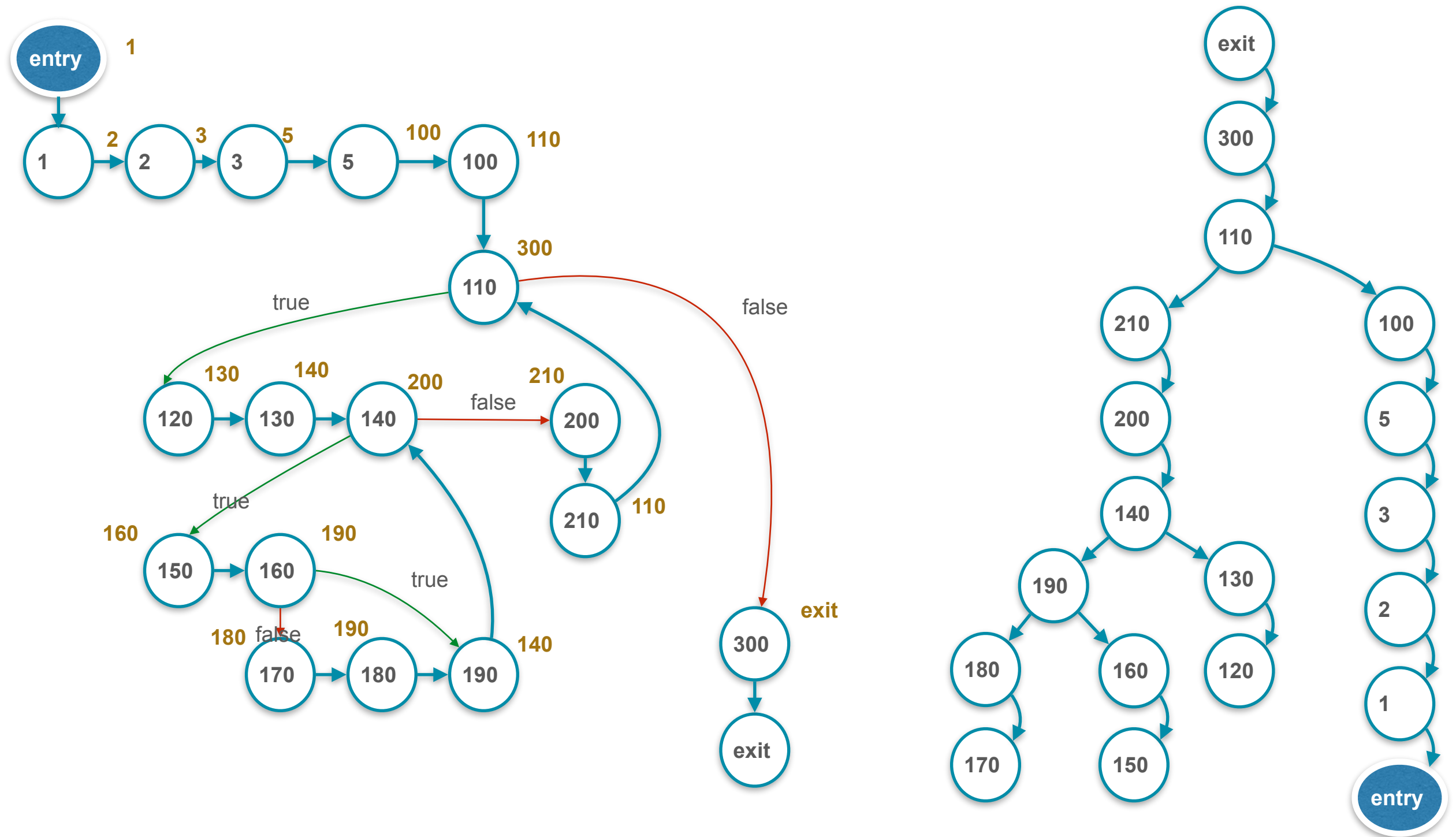


Which nodes are post-dominated by 190?

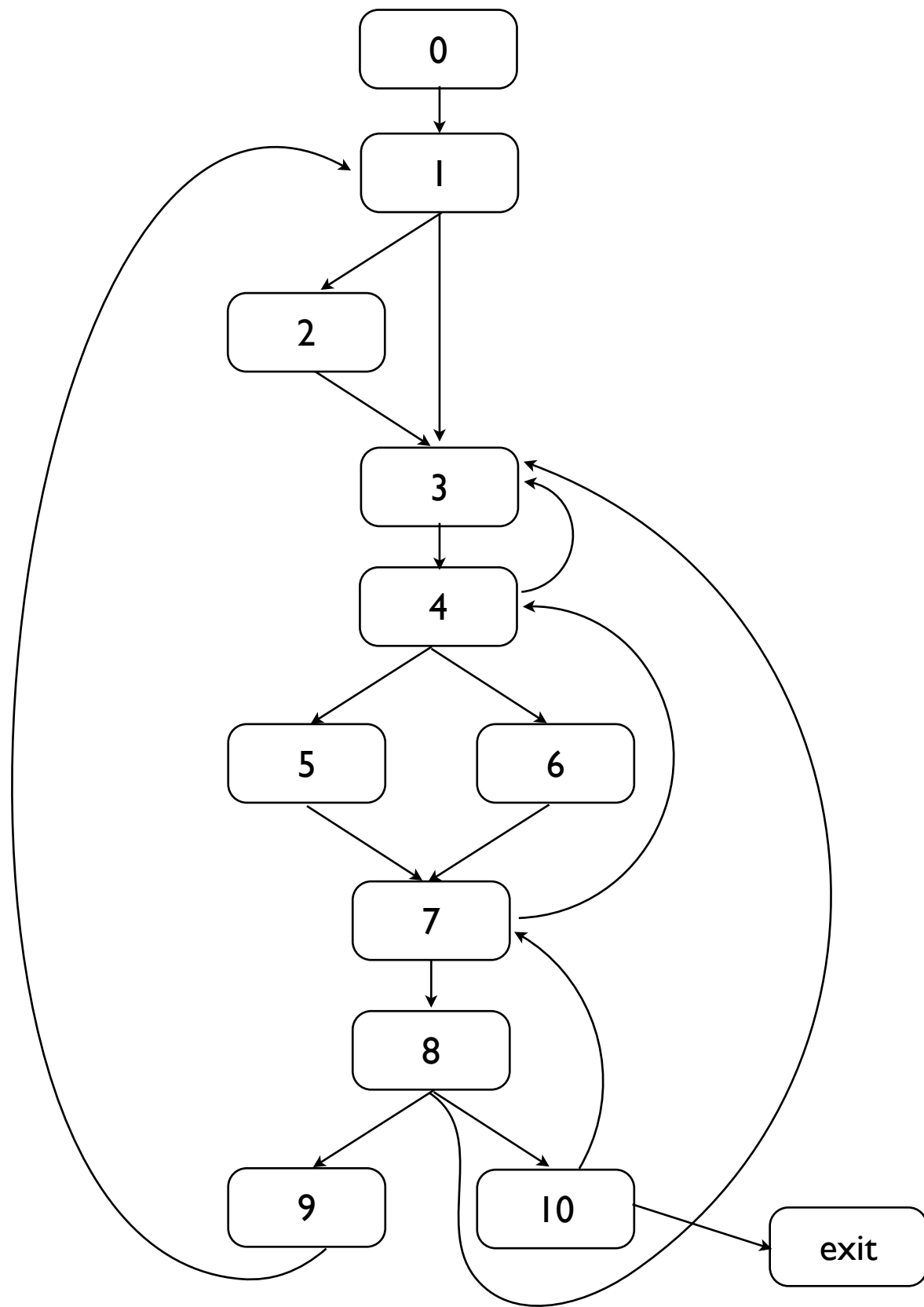
Immediate Post-Dominators



Immediate Post-Dominators



Post-Dominator Example



Block	PDom	IPDom
1	{1,3,4,7,8,10,exit}	3
2	{2,3,4,7,8,10,exit}	3
3	{3,4,7,8,10,exit}	4
4	{4,7,8,10,exit}	7
5	{5,7,8,10,exit}	7
6	{6,7,8,10,exit}	7
7	{7,8,10,exit}	8
8	{8,10,exit}	10
9	{1,3,4,7,8,9,10,exit}	1
10	{10,exit}	exit

Computing Dominators

Result: A map of dependence relations D

```
dominance( $CFG$ ) begin
   $entry \leftarrow entryNode(CFG);$ 
   $D(entry) \leftarrow entry;$ 
  for  $n \in CFG.nodes - \{entry\}$  do
     $D(n) \leftarrow CFG.nodes;$ 
  end
   $changed \leftarrow true;$ 
  while  $changed$  do
     $changed \leftarrow false;$ 
    for  $n \in CFG.nodes - \{entry\}$  do
       $currentDominators \leftarrow D(n);$ 
       $newDominators \leftarrow n \cup (\bigcap D(p))$  for all immediate predecessors  $p$  of  $n$ ;
      if  $currentDominators \neq newDominators$  then
         $D(n) \leftarrow newDominators;$ 
         $changed \leftarrow true;$ 
      end
    end
  end
  return  $D$ 
end
```

Computing Post-Dominators

Result: A map of dependence relations D

```
dominance(CFG) begin  
  entry  $\leftarrow$  entryNode(CFG);  
   $D(\text{entry}) \leftarrow \text{entry}$ ;  
  for  $n \in \text{CFG.nodes} - \{\text{entry}\}$  do  
     $D(n) \leftarrow \text{CFG.nodes}$ ;  
  end  
  changed  $\leftarrow$  true;  
  while changed do  
    changed  $\leftarrow$  false;  
    for  $n \in \text{CFG.nodes} - \{\text{entry}\}$  do  
      currentDominators  $\leftarrow D(n)$ ;  
      newDominators  $\leftarrow n \cup (\bigcap D(p))$  for all immediate predecessors  $p$  of  $n$ ;  
      if currentDominators  $\neq$  newDominators then  
         $D(n) \leftarrow \text{newDominators}$ ;  
        changed  $\leftarrow$  true;  
      end  
    end  
  end  
  return  $D$   
end
```

Reverse the CFG!

Pick the node without any incoming edges. In the reversed graph would originally have been the “exit” node.

Best to start from nodes at the end of the original control flow graph (closest to exit node), and work backwards.

Same as dominance algorithm!

Outline

1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

Control Dependence

Statement b is **control dependent** on statement a if a directly determines whether b will be executed.

b is control dependent on a if:

- There exists a directed path from a to b in the control flow graph where all nodes (excl. a) are post-dominated by b .
- a is not post-dominated by b .

For b to be control dependent on a , a must have two exits, one of which goes through b .

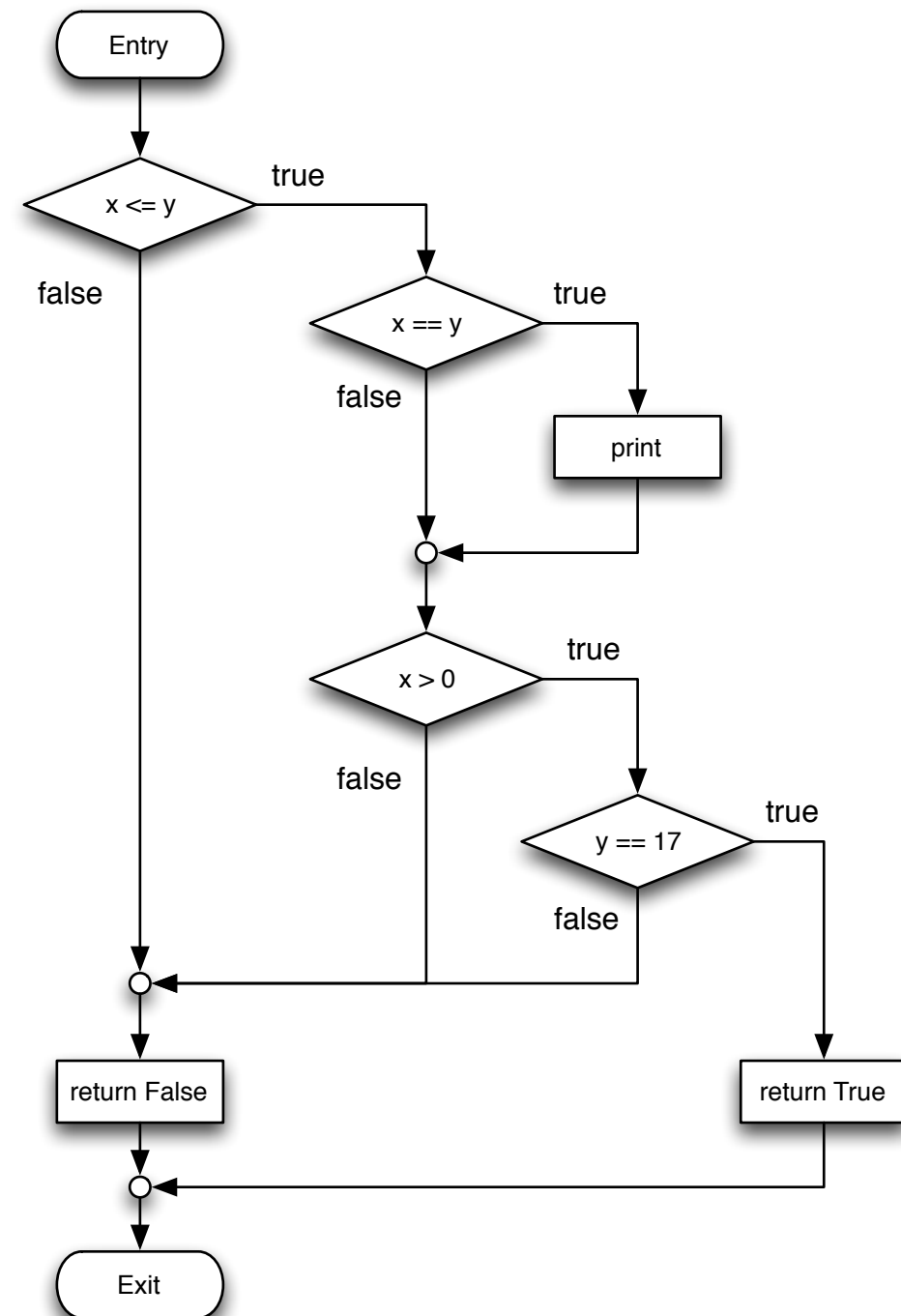
```
def testMe(x, y):  
    if x <= y:  
        if x == y:  
            print("Some output")  
        if x > 0:  
            if y == 17:  
                return True  
    return False
```

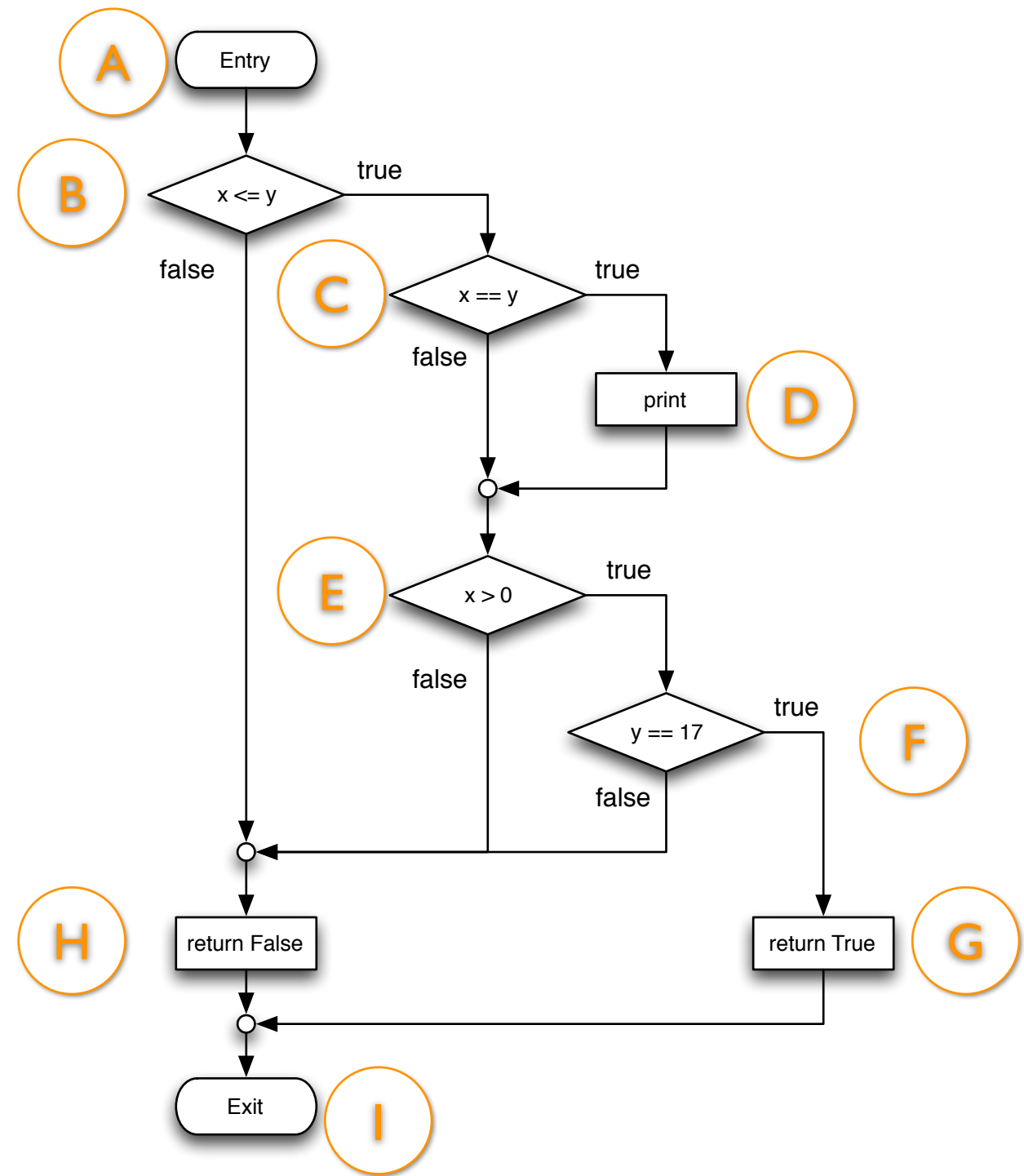


```

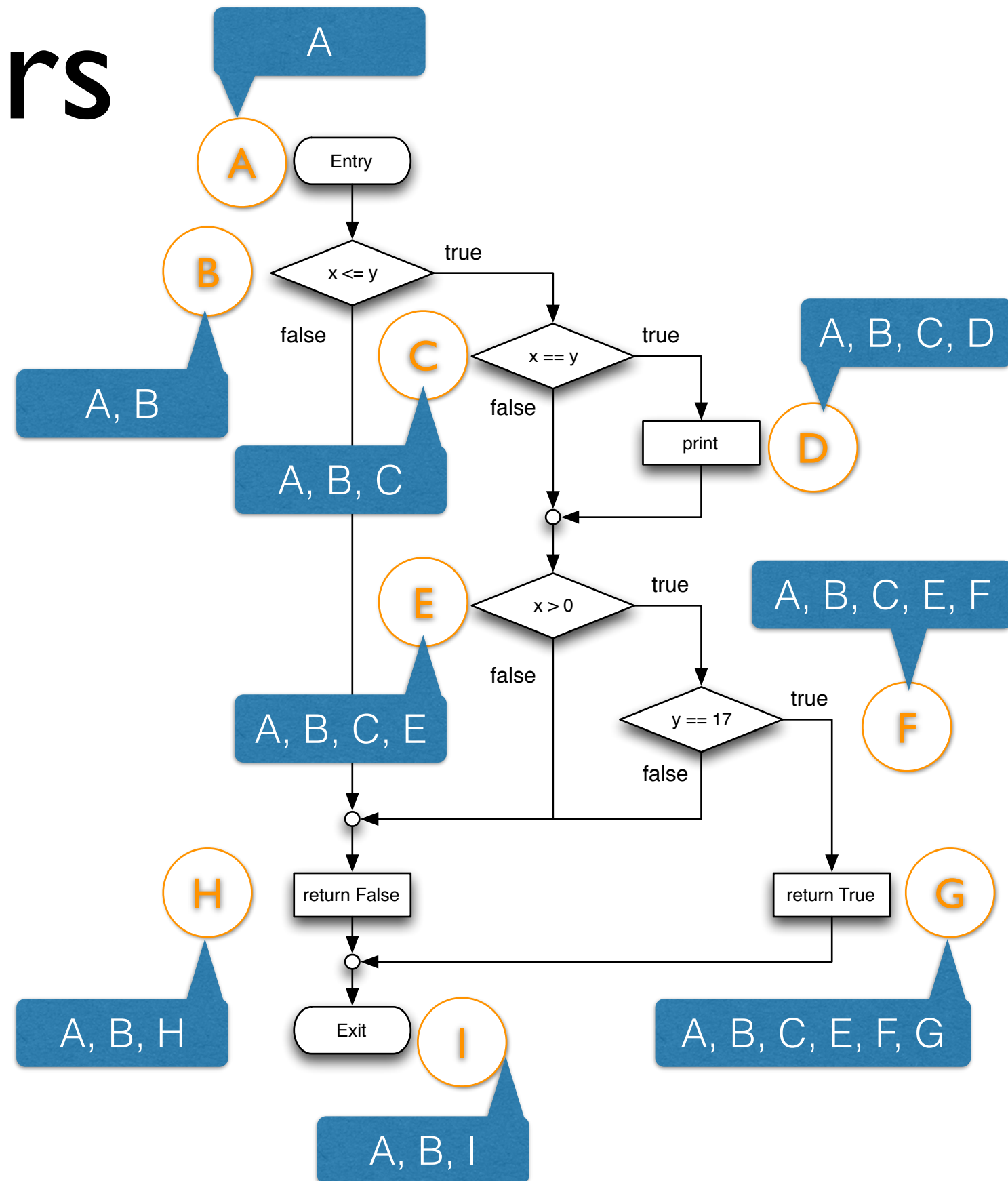
def testMe(x, y):
    if x <= y:
        if x == y:
            print("Some output")
        if x > 0:
            if y == 17:
                return True
    return False

```

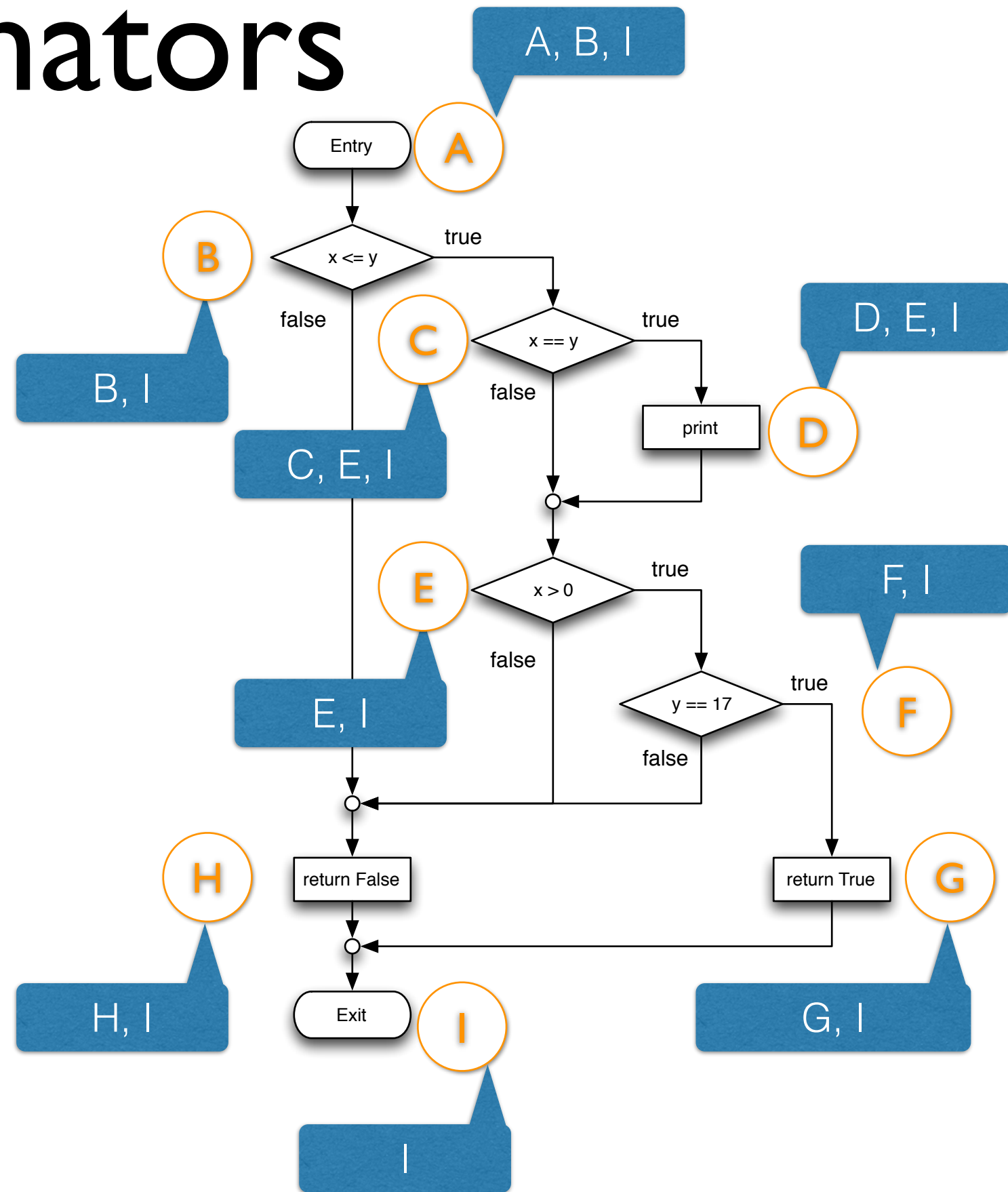




Dominators

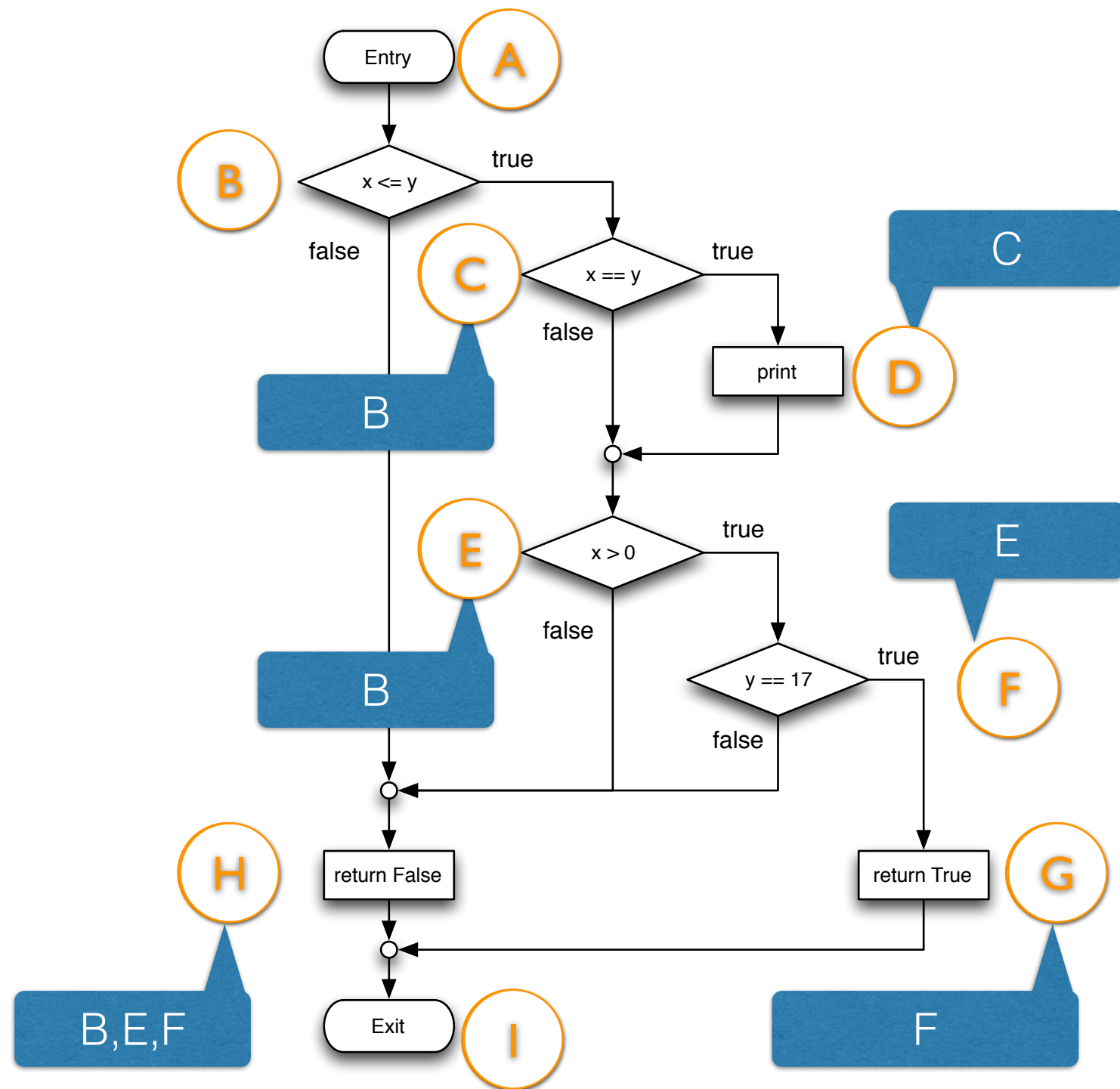


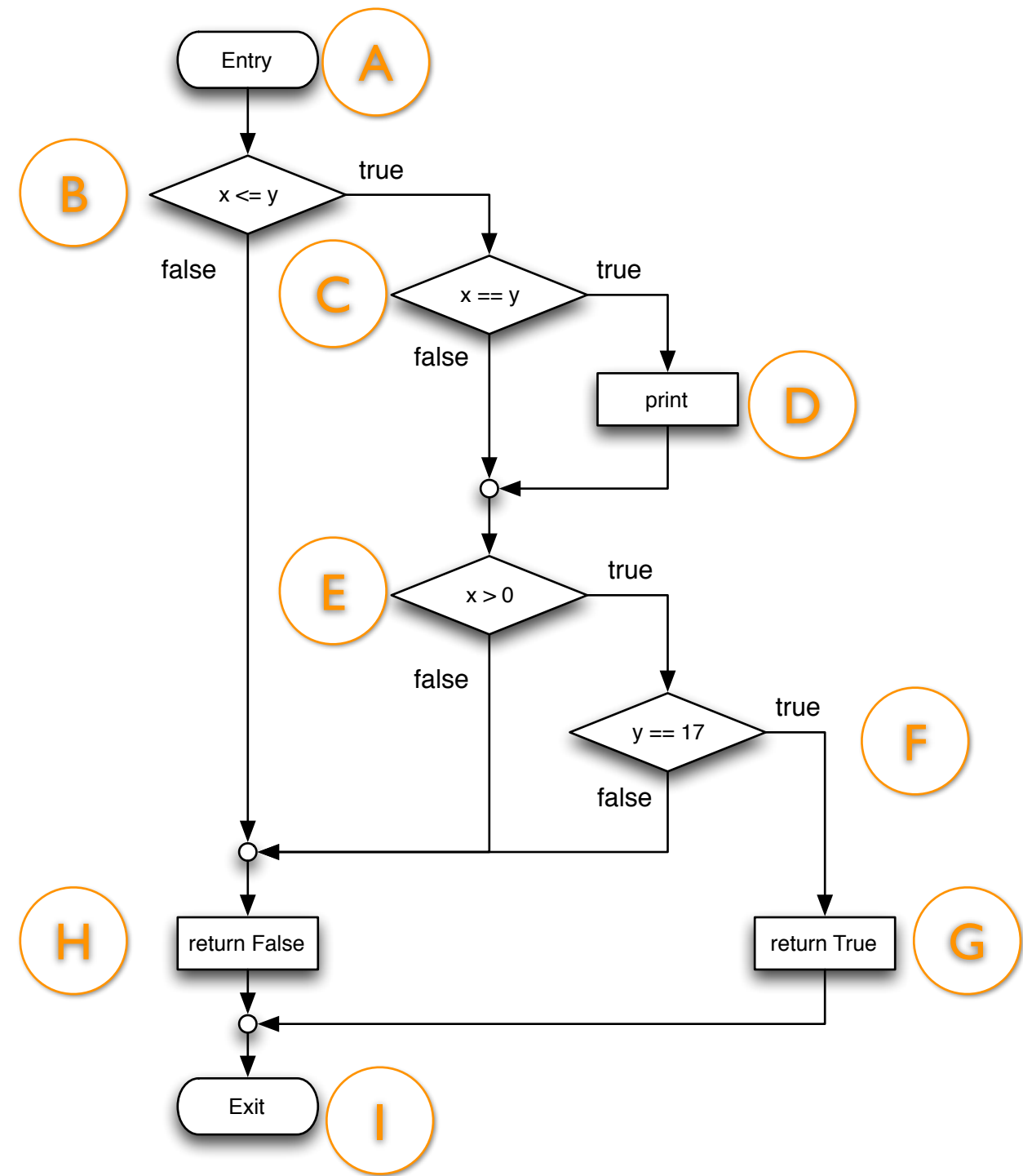
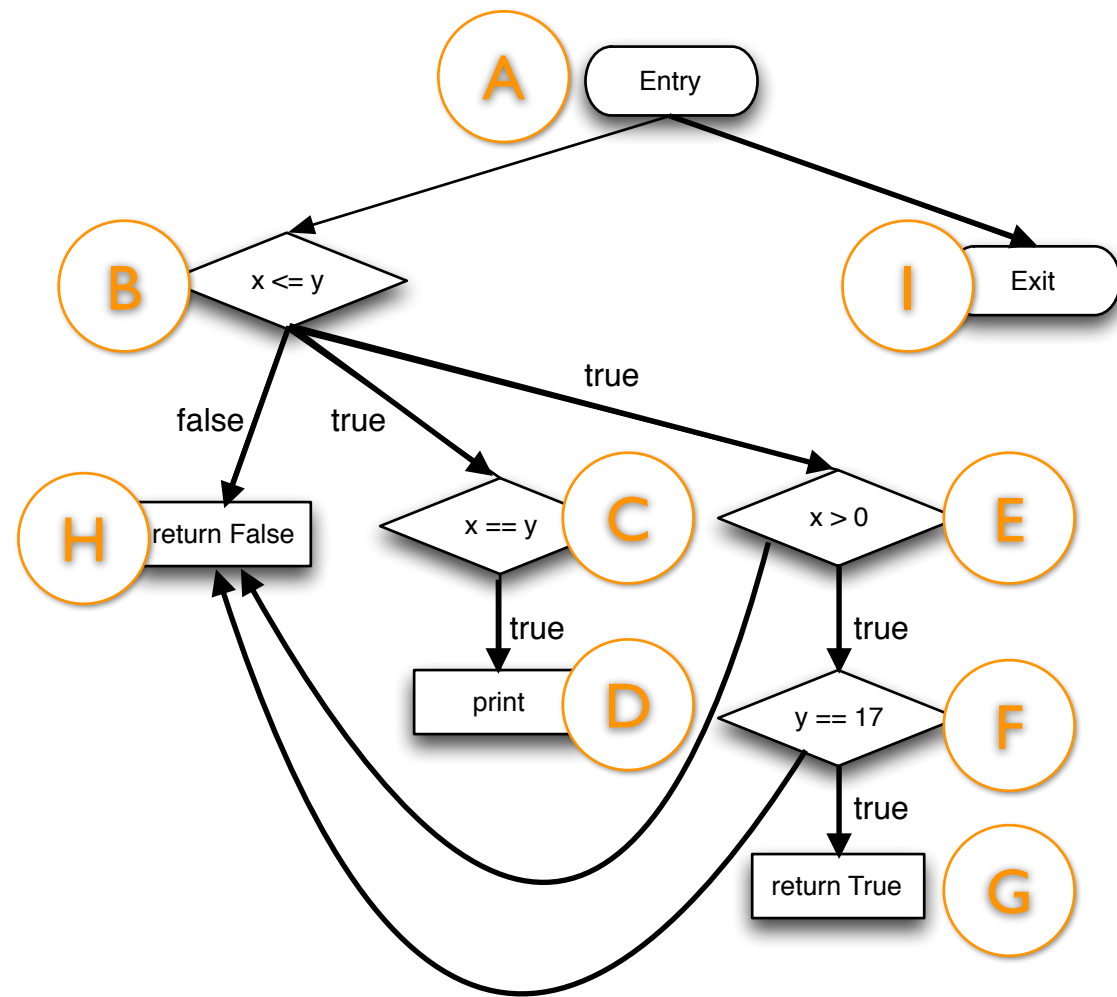
Post-Dominators



Control Dependence

- b is control dependent on a if:
- There exists a directed path from a to b in the control flow graph where all nodes are post-dominated by b (excl. a).
- a is not post-dominated by b .



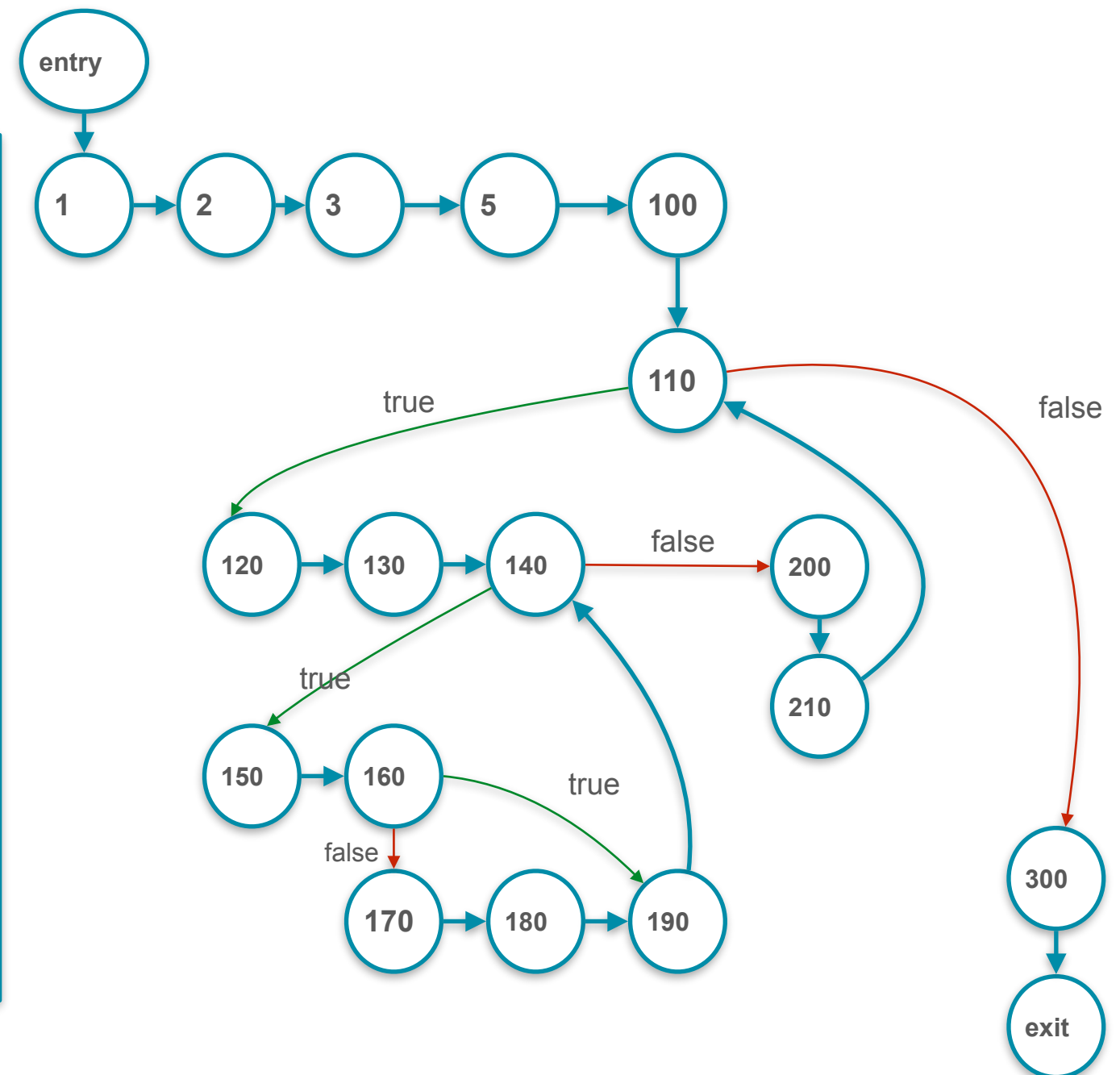


Constructing the CDG

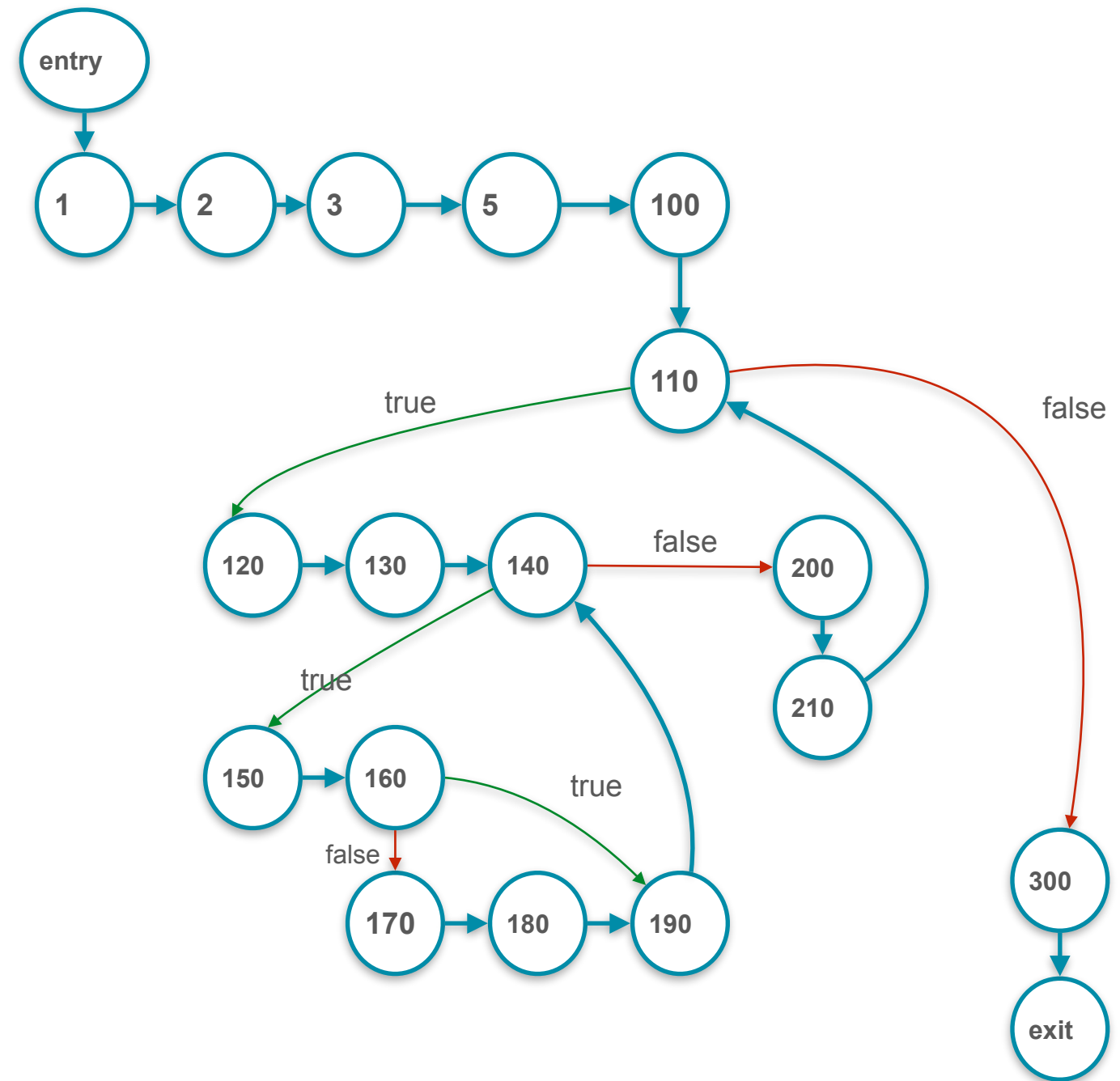
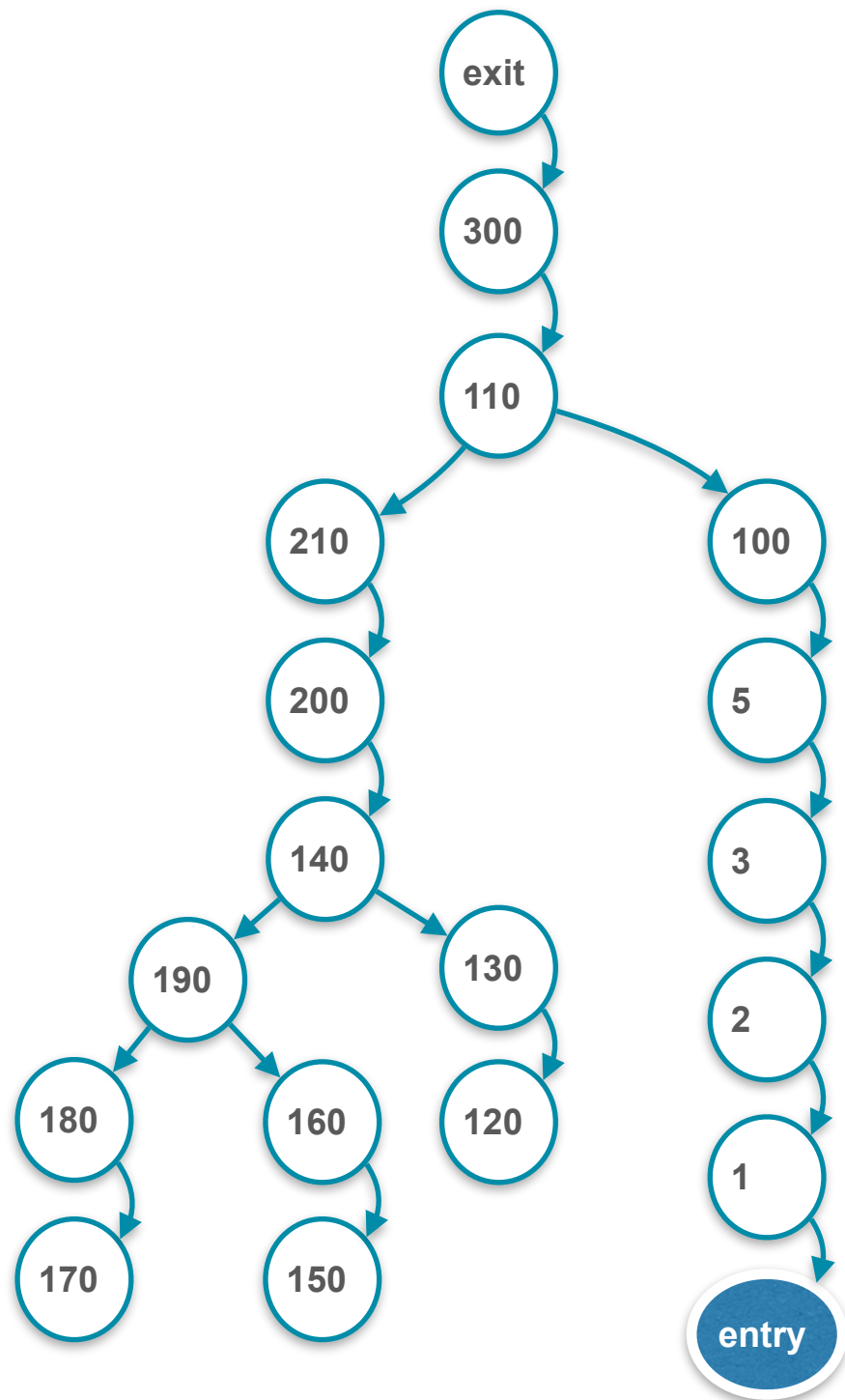
- Determine the control dependencies for the program, using the following steps:
 1. Find S , a set of edges (A,B) in the CFG such that B is not an ancestor of A in the **post-dominator tree**.
 2. For each edge (A,B) in S , find L , the **least common ancestor** of A and B in the **post-dominator tree**.
 3. Consider each edge (A,B) in S and its corresponding L . Traverse backwards in the post-dominator tree from B to L , marking each node visited; mark L **only** if $L = A$.
 4. Statements representing all marked nodes are control dependent on A .

Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```

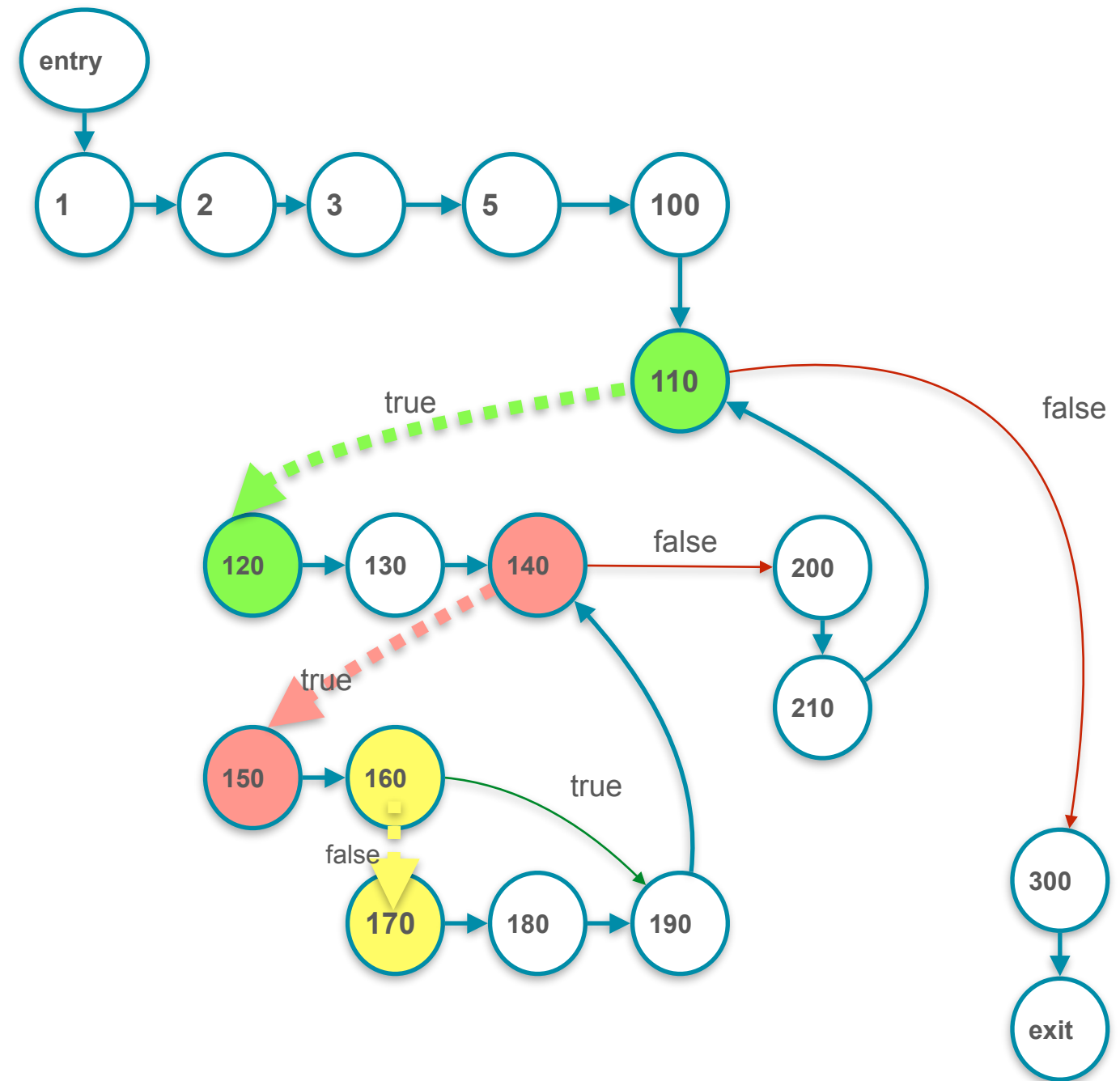
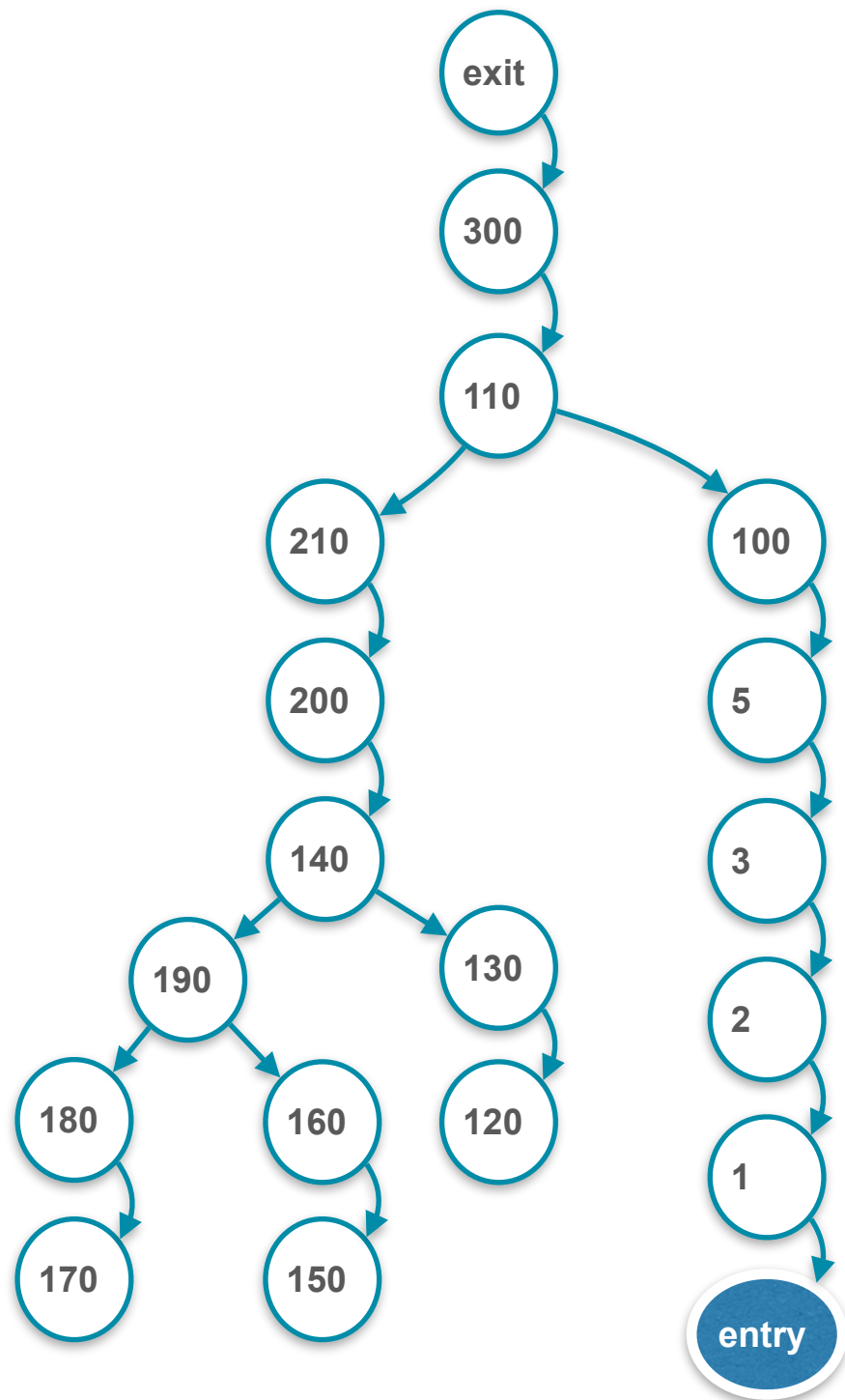


Example



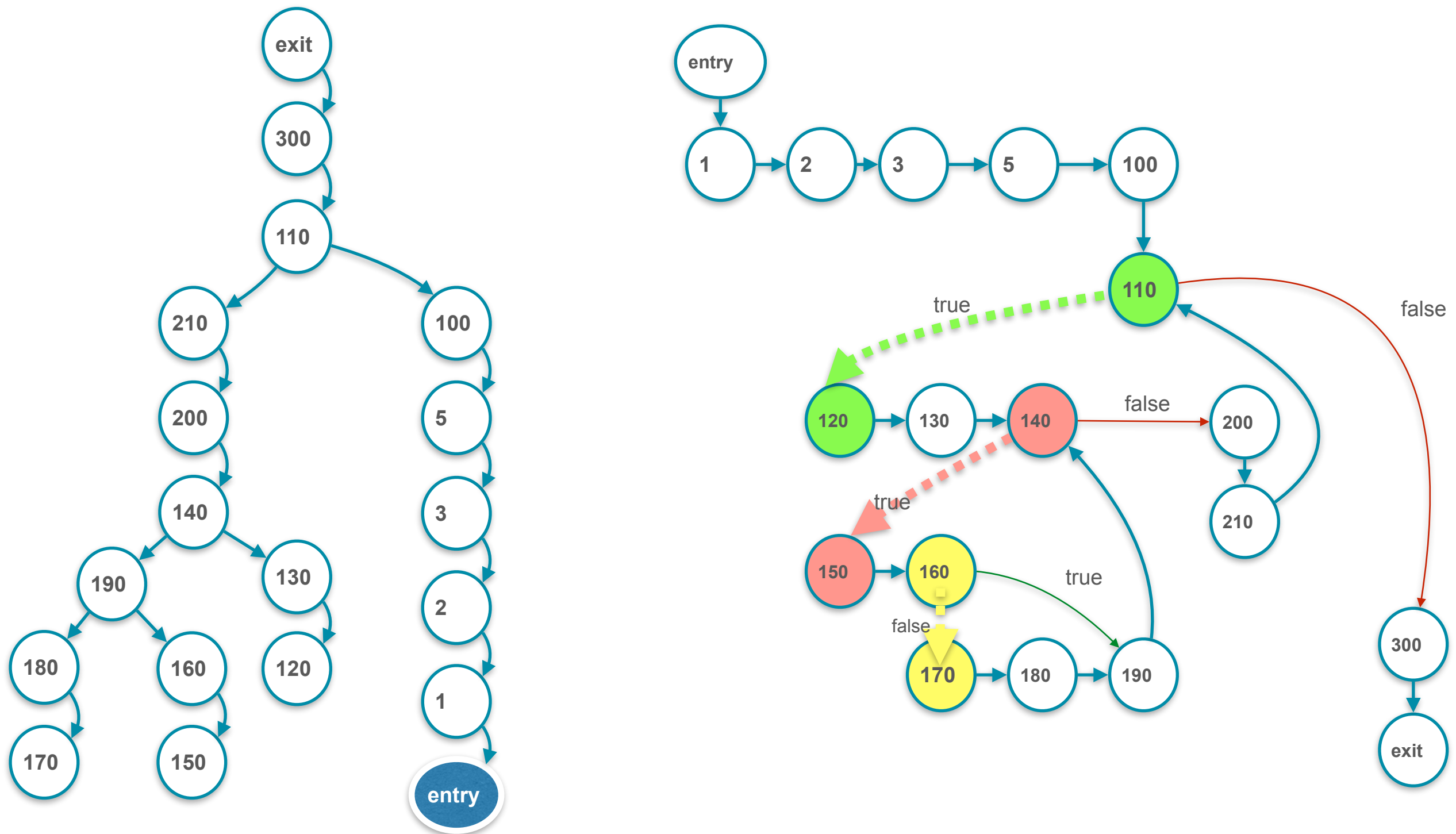
I. Find S , a set of edges (A,B) in the CFG such that B is not an ancestor of A in the post-dominator tree.

Example



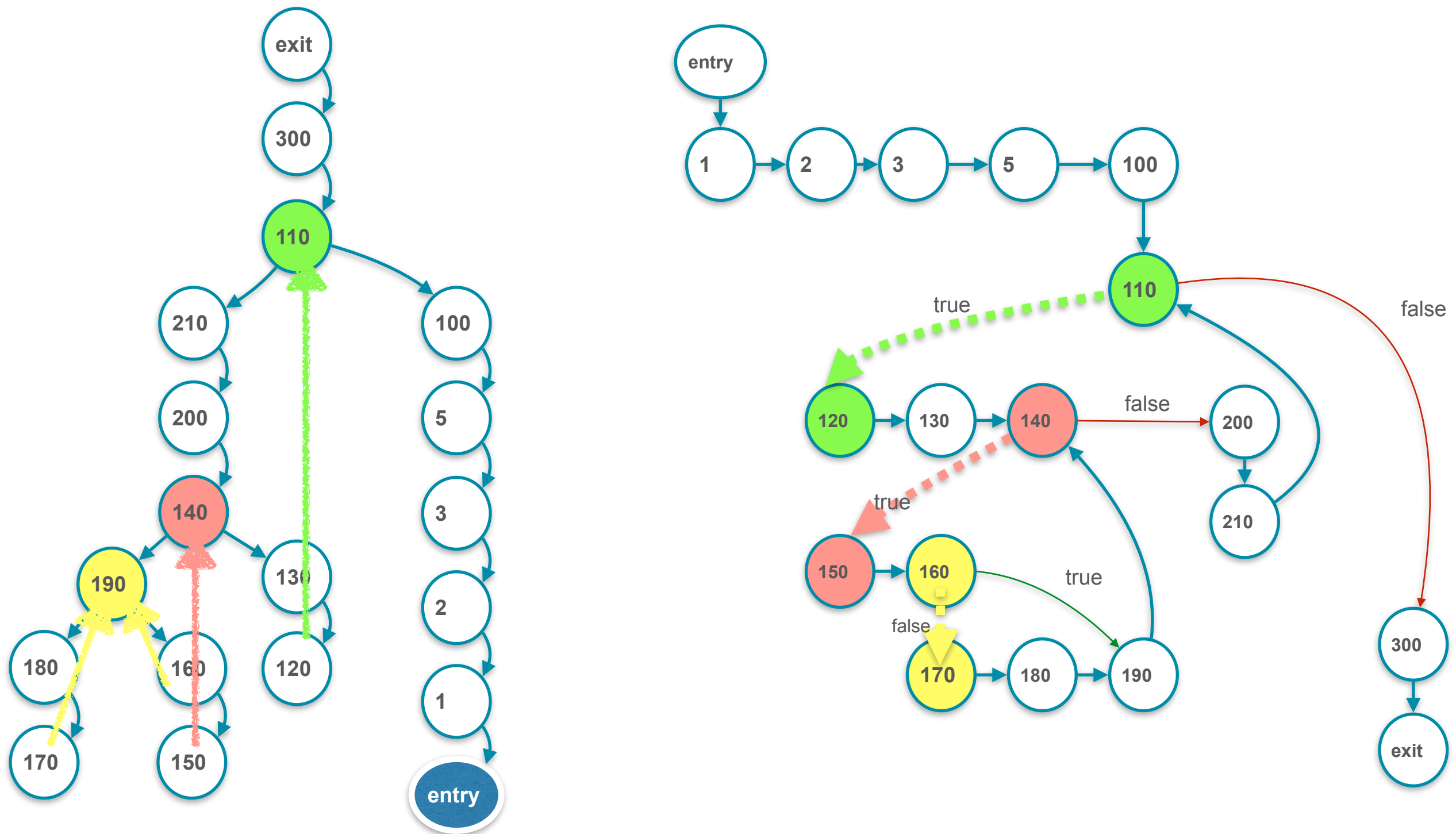
I. Find S , a set of edges (A,B) in the CFG such that B is not an ancestor of A in the post-dominator tree.

Example



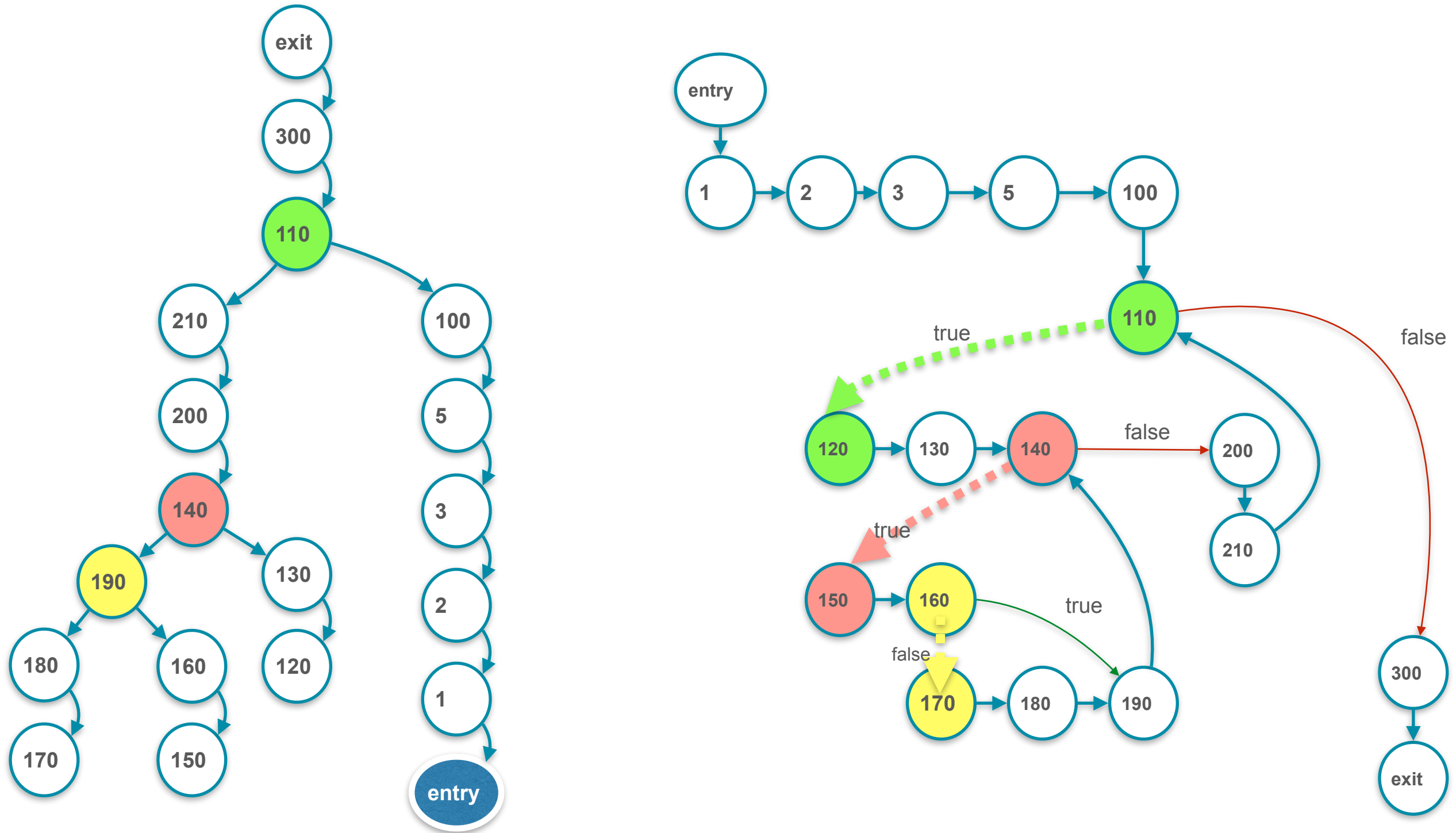
2. For each edge (A,B) in S , find L , the least common ancestor of A and B in the post-dominator tree.

Example



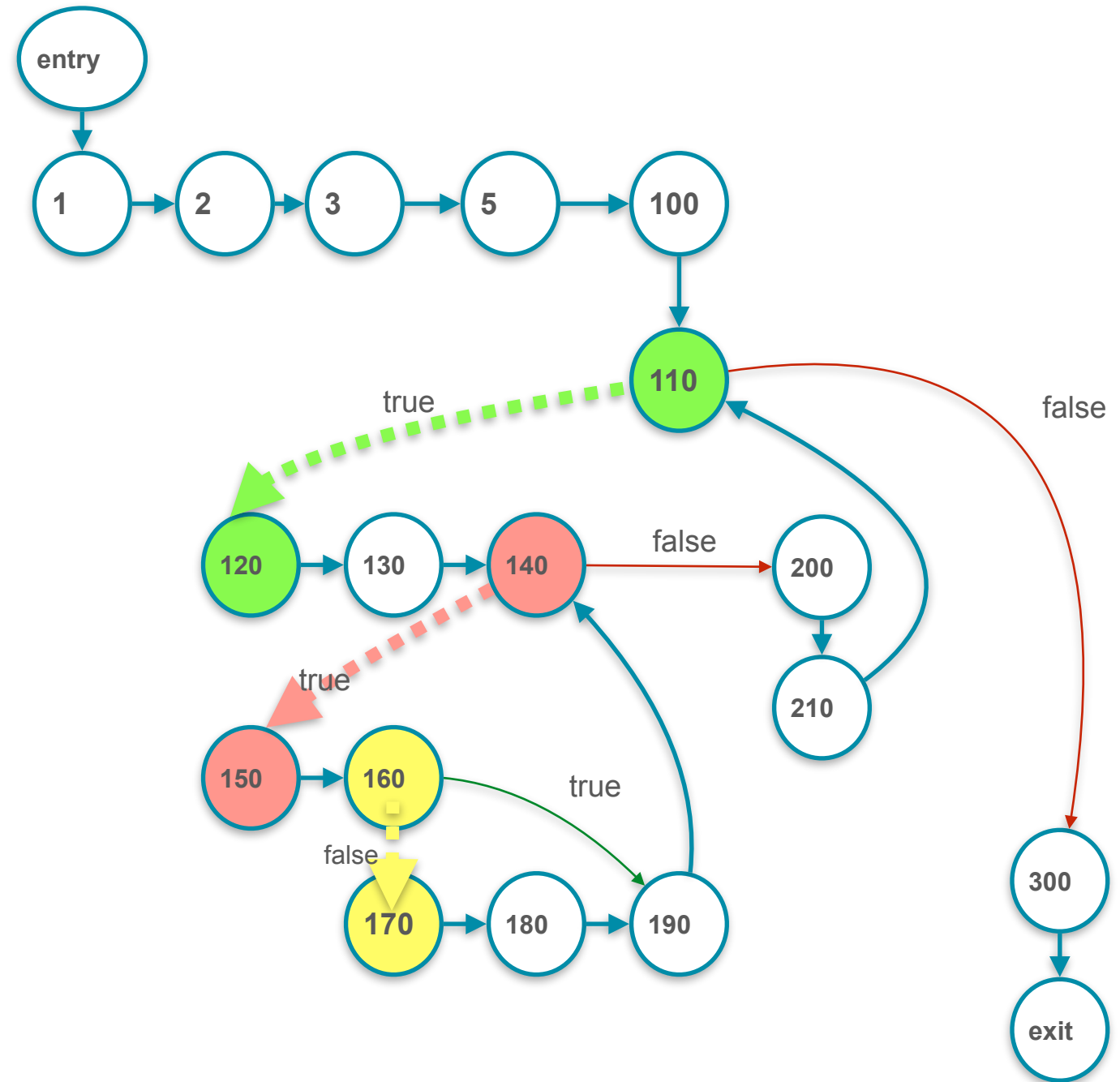
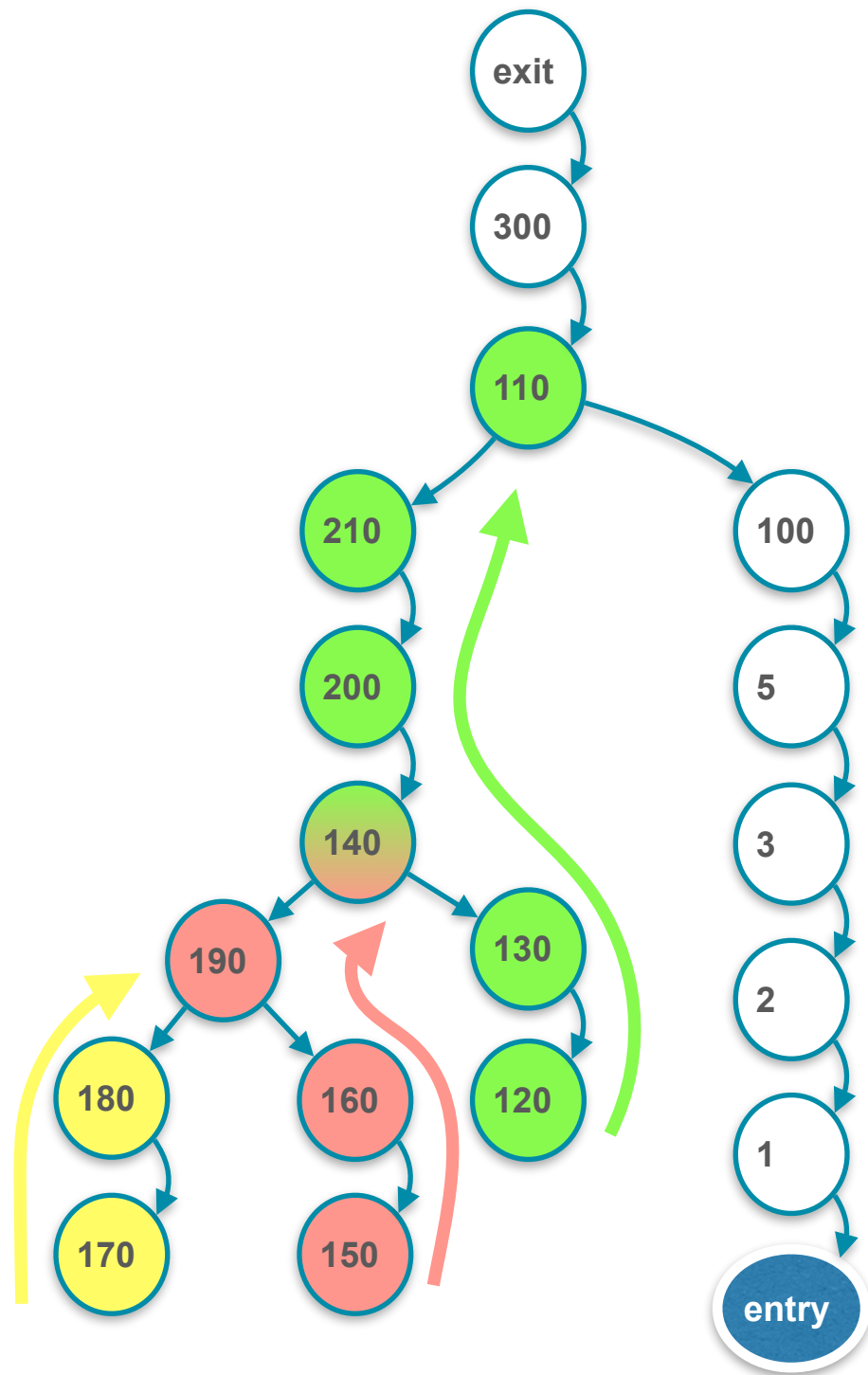
- For each edge (A,B) in S , find L , the least common ancestor of A and B in the post-dominator tree.

Example



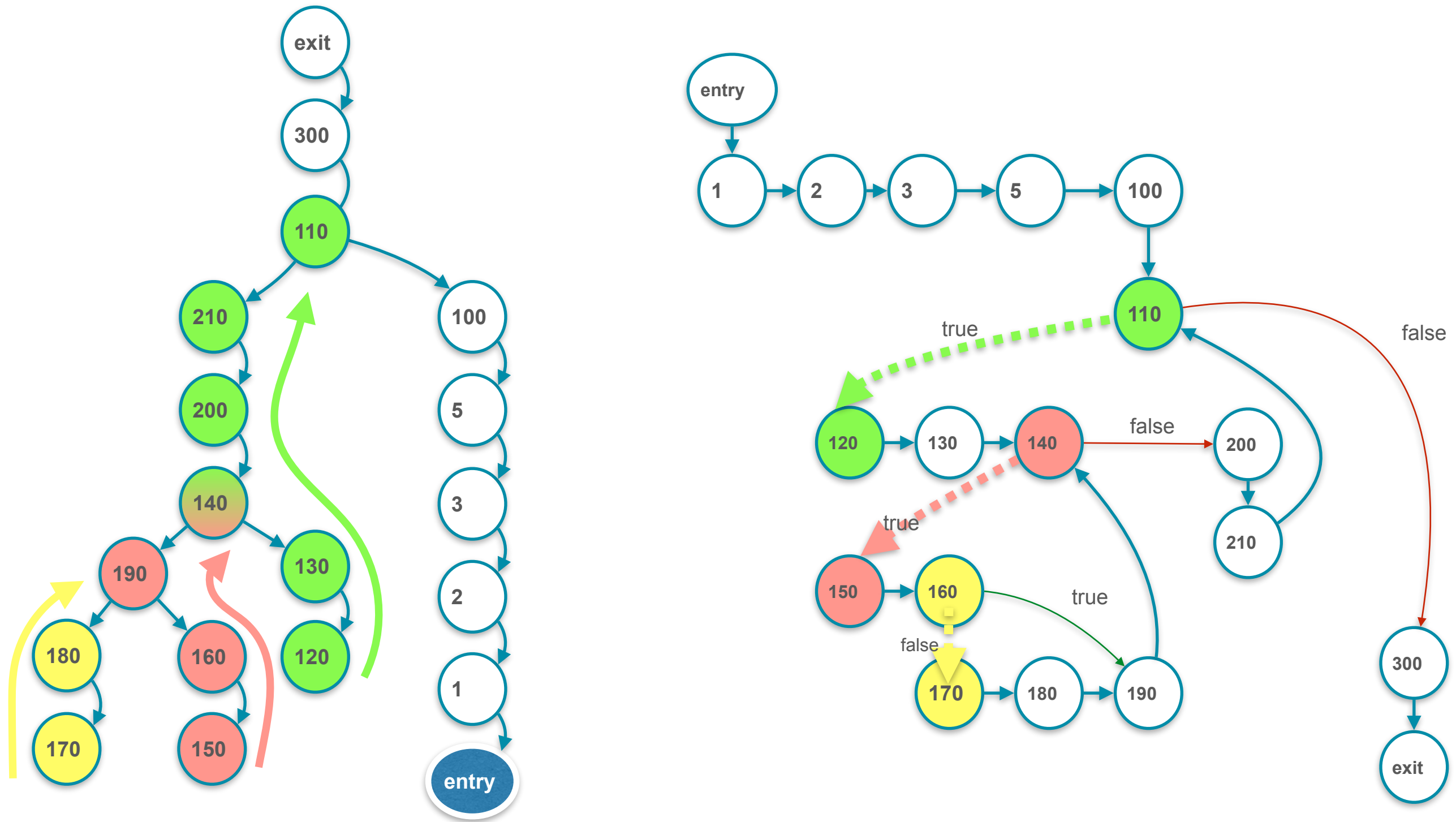
3. Consider each edge (A,B) in S and its corresponding L . Traverse backwards in the post-dominator tree from B to L , marking each node visited; mark L only if $L = A$.

Example



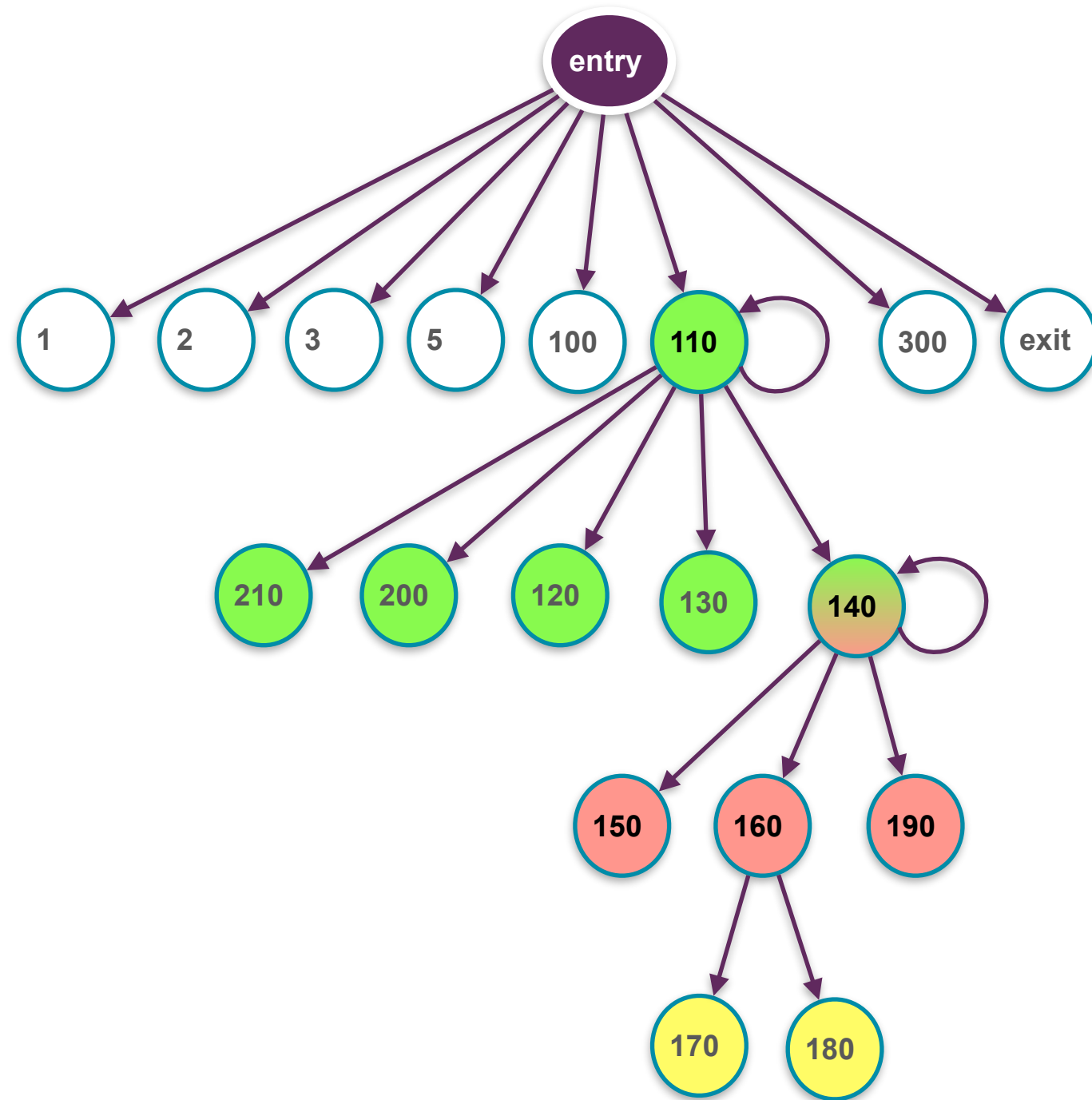
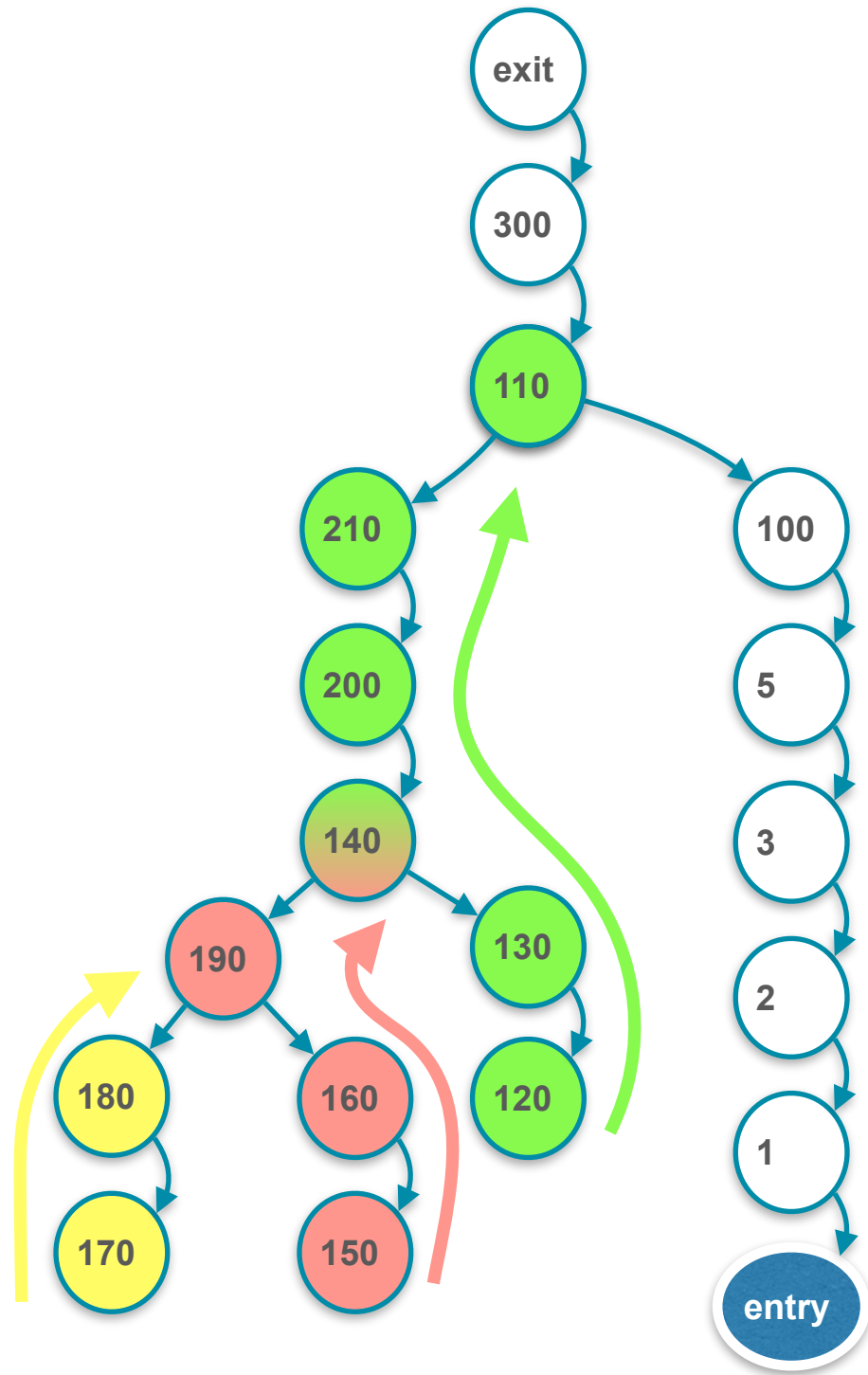
- Consider each edge (A,B) in S and its corresponding L . Traverse backwards in the post-dominator tree from B to L , marking each node visited; mark L only if $L = A$.

Example



4. Statements representing all marked nodes are control dependent on A.

Example



4. Statements representing all marked nodes are control dependent on A.
(All statements not marked control dependent are dependent on entry)

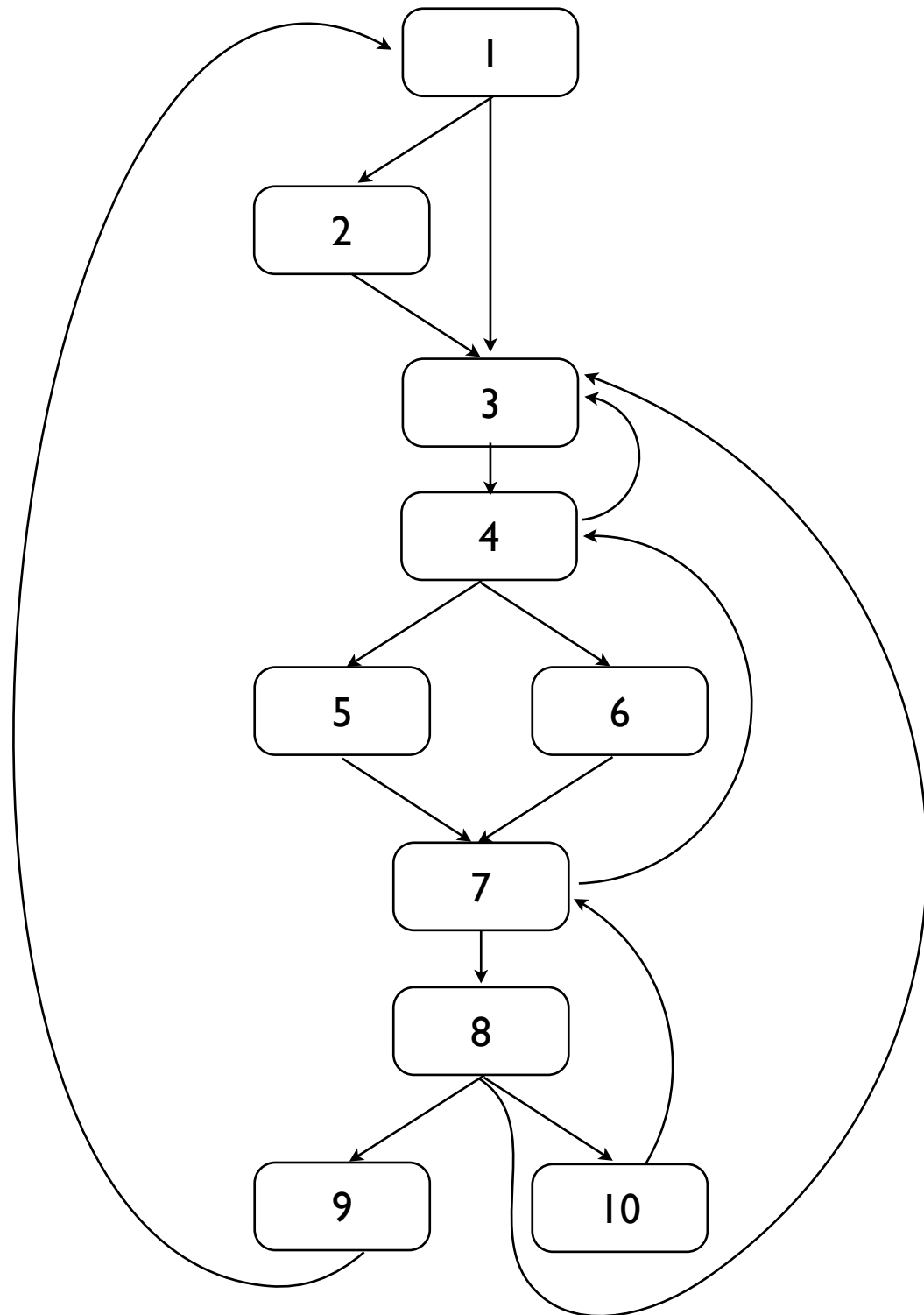
Outline

1. Motivation
2. Control Flow Graphs
3. Basic blocks
4. Dominance
5. Post-Dominance
6. Control Dependence

Natural Loops

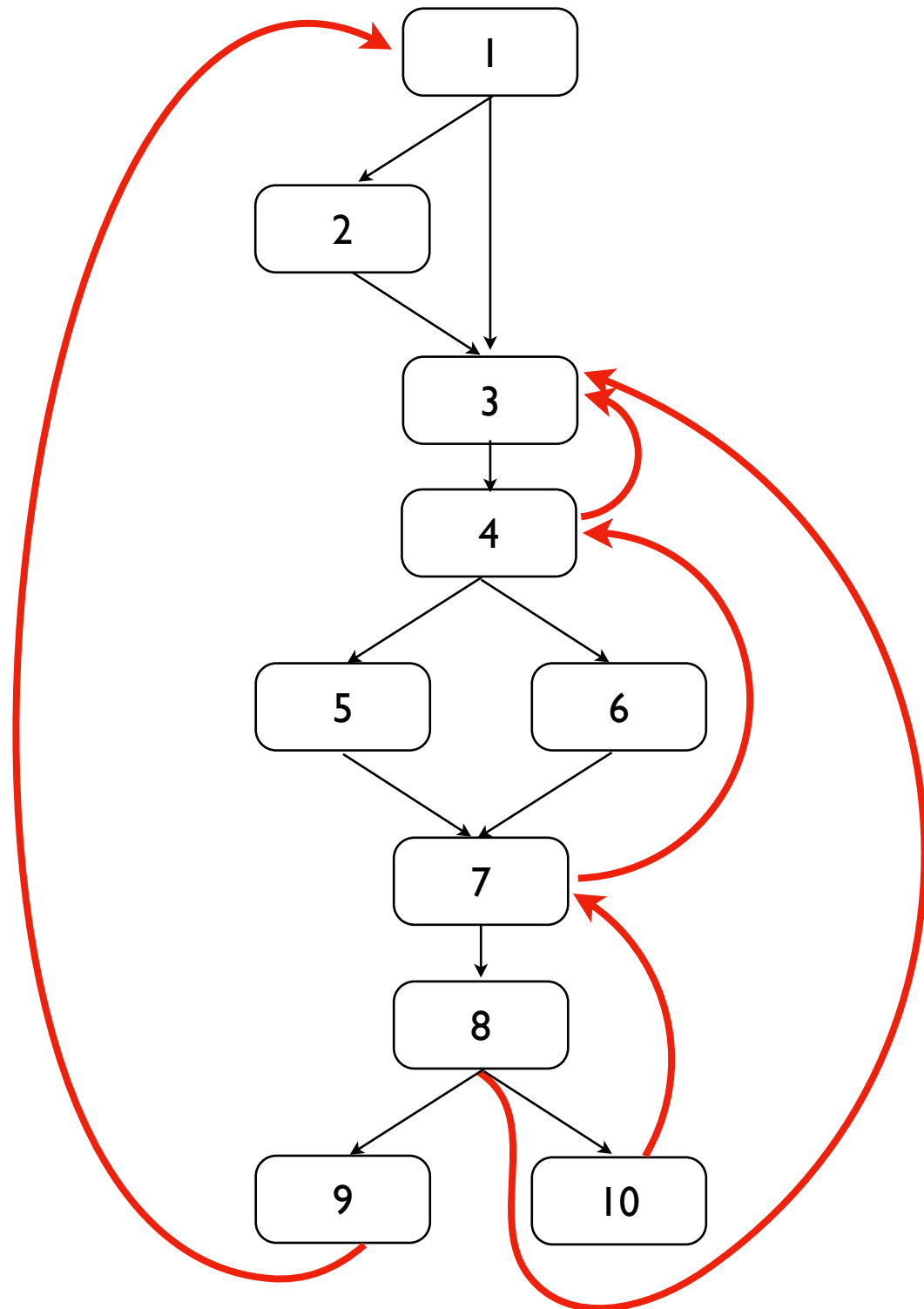
- Natural loops that are suitable for improvement have two essential properties:
 - A loop must have a single entry point called header
 - There must be at least one way to iterate the loop, i.e., at least one path back to the header
- Identifying natural loops
 - Searching for back edges ($n \rightarrow d$) in CFG whose heads dominate their tails
 - For an edge $a \rightarrow b$, b is the head and a is the tail
 - A back edge flows from a node n to one of n 's dominators d
 - The natural loop for that edge is $\{d\}$ +the set of nodes that can reach n without going through d
 - d is the header of the loop

Back Edge Example



Block	Dom	IDom
1	{1}	-
2	{1,2}	1
3	{1,3}	1
4	{1,3,4}	3
5	{1,3,4,5}	4
6	{1,3,4,6}	4
7	{1,3,4,7}	4
8	{1,3,4,7,8}	7
9	{1,3,4,7,8,9}	8
10	{1,3,4,7,8,10}	8

Back Edge Example

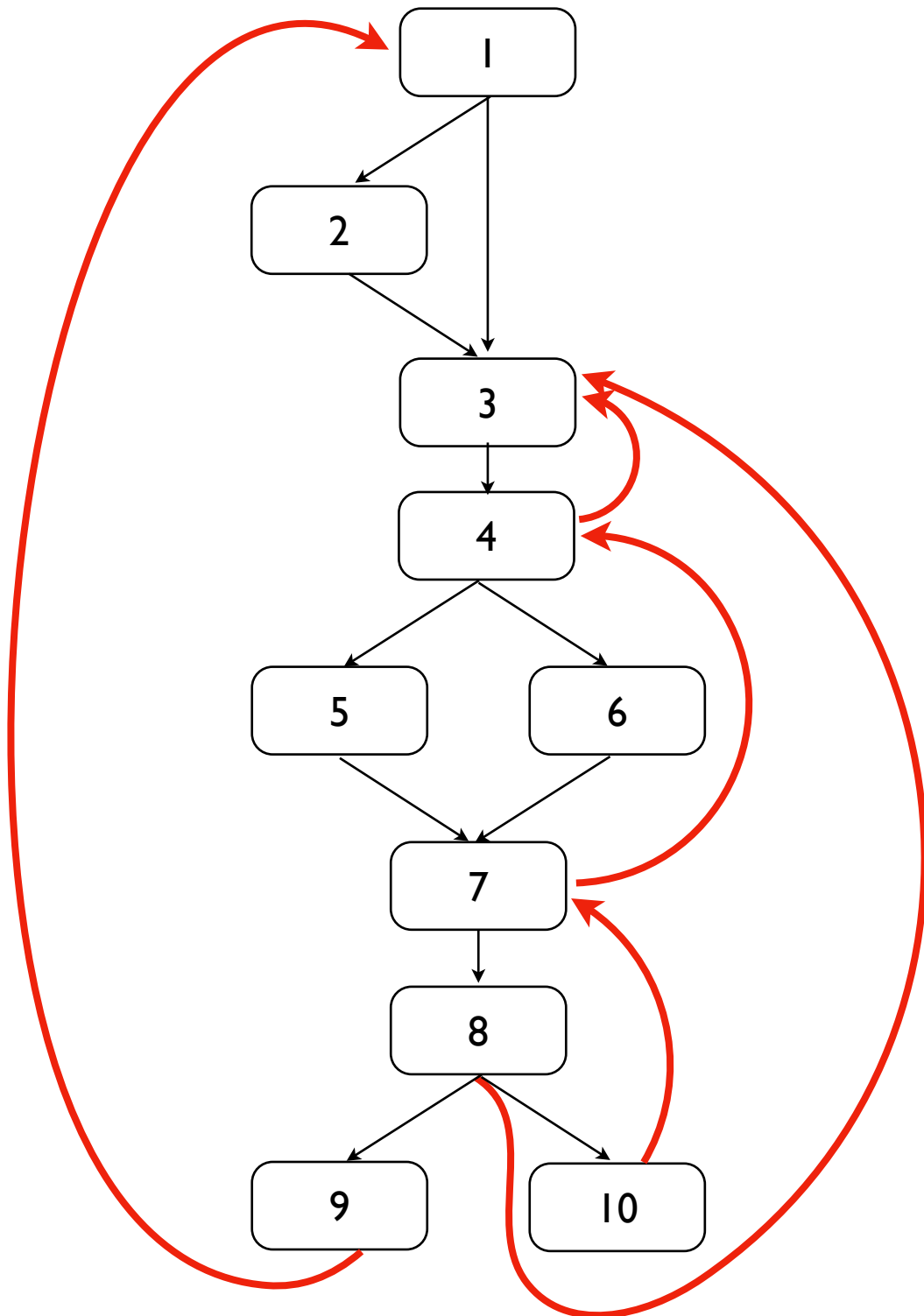


Block	Dom	IDom
1	{1}	-
2	{1,2}	1
3	{1,3}	1
4	{1,3,4}	3
5	{1,3,4,5}	4
6	{1,3,4,6}	4
7	{1,3,4,7}	4
8	{1,3,4,7,8}	7
9	{1,3,4,7,8,9}	8
10	{1,3,4,7,8,10}	8

Identifying Natural Loops

- Given a back edge $n \rightarrow d$, the natural loop of the edge includes
 - Node d
 - Any node that can reach n without going through d

Natural Loops Example



Back Edge	Natural Loop
10 -> 7	{7,10,8}
7 -> 4	{4,7,5,6,10,8}
4 -> 3	{3,4,7,5,6,10,8}
8 -> 3	{3,4,7,5,6,10,8}
9 -> 1	{1,9,8,7,5,6,10,4,3,2}

Inner Loops

- A useful property of natural loops: unless two loops have the same header, they are either disjoint or one is entirely contained (nested within) the other
- An inner loop is a loop that contains no other loops
 - Good optimisation candidate
 - Inner loop of previous example: {7,8,10}